Machine Learning for Music

Module 3: symbolic music generation

Music Generation

Slides mostly based on

“Generative AI for Music and Audio Creation” (Michigan):

https://hermandong.com/teaching/pat498_598_fall2024/

(though lots of other references as we go)

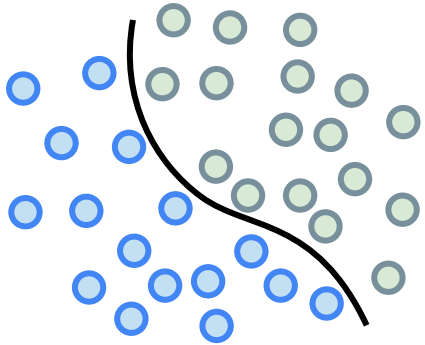
Music Generation

We've spent the last ~four weeks discussing how music can be *represented* for use downstream in predictive pipelines

How can we do the reverse task, i.e., how can we *generate* music?

Recall: *Discriminative* versus *Generative* AI

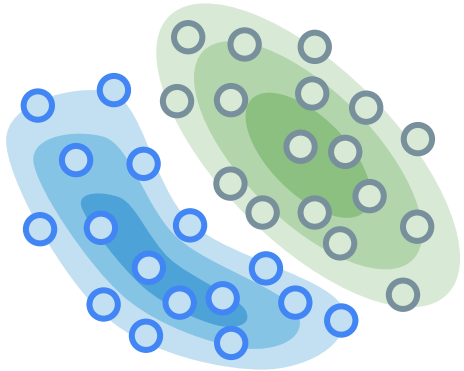
Discriminative



Discriminative models learn the **decision boundary**

$P(y|x)$

Generative



Generative models learn the underlying distribution

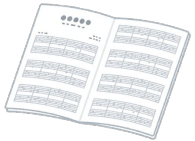
$P(x)$ or $P(x|y)$

Recall: *Discriminative* versus *Generative* AI

Generative AI learns the underlying distribution (x), so can “generate” new samples (x’s)

NVIDIA Fugatto (2024)
(note: not symbolic)

Four paradigms of music generation



Symbolic music generation (Module 3)

- text
  - MIDI

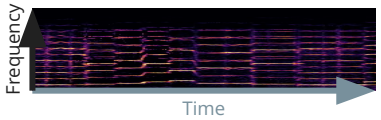
```
Program_change_0,
Note_on_60, Time_shift_2, Note_off_60,
Note_on_60, Time_shift_2, Note_off_60,
Note_on_76, Time_shift_2, Note_off_67,
Note_on_67, Time_shift_2, Note_off_67,
...
```

- image
  - piano roll

Audio-domain music generation (Module 4)

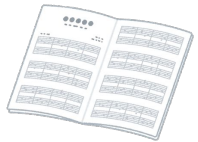
- time series
  - waveform

- image
  - spectrogram



(certainly not a complete picture, but a rough way to characterize things for now)

Symbolic music generation: two main approaches



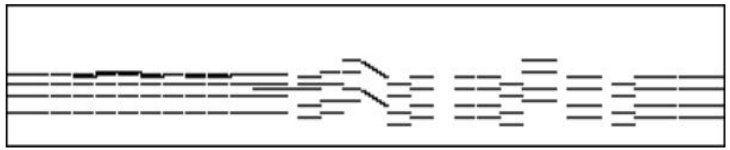
text-based

- Treat music like **text**
- Borrow ideas from **natural language processing (NLP)**
  - RNNs, LSTMs, Transformers, etc.

```
Program_change_0,
Note_on_60, Time_shift_2, Note_off_60,
Note_on_60, Time_shift_2, Note_off_60,
Note_on_76, Time_shift_2, Note_off_67,
Note_on_67, Time_shift_2, Note_off_67, ...
```

image-based

- Treat music like **images** (or 2d objects)
- Borrow ideas from **computer vision (CV)**
  - GANs, VAEs, diffusion models, etc.



Symbolic music generation: several possible tasks

Unconditional

- ∅ –> melody
- ∅ –> chords
- ∅ –> lead sheet
- ∅ –> sheet music

melody & chords

Conditional

Automatic arrangement

- melody → lead sheet
- melody → multitrack
- lead sheet → multitrack
- solo → multitrack
- multitrack → simple version

Performance rendering

- sheet music → performance

Improvisation systems

- performance → performance

Multimodal

- text → music
- video → music
- X → music

Rough outline

- (More) Historical approaches to symbolic generation
  - (from the survey paper, refer as “Survey” later)
- AI Methods in Algorithmic Composition: A Comprehensive Survey (2013)
  - Early survey about automatic music composition
  - https://arxiv.org/pdf/1402.0585
- Introduction to next-token prediction approaches
  - Markov chains for melody & chord generation
  - Hidden Markov Models for melody harmonization
- Language representations of music
  - ABC, MIDI-like, REMI, MMT (all these to be defined!)
- Language models for music generation
  - n-gram, RNN, LSTM, Transformers
- “Image-based” symbolic generation
  - Piano roll generation, MuseGAN

Symbolic music generation

3.1: Historical approaches to symbolic generation

Historical approaches to algorithmic generation

- We have seen a few examples of **stochastic** and **rule-based** music generation systems in Module 1
- Before building our own symbolic music generation systems, let's spend a bit more time on (algorithmic) musical history
- While these are still relatively early attempts, many of them offer important design ideas that have influenced "modern" AI music generation systems

Historical approaches to algorithmic generation

Lots of surveys:

- **(Mostly based on this one)** AI Methods in Algorithmic Composition: A Comprehensive Survey (2013): https://arxiv.org/pdf/1402.0585
- A Survey on Deep Learning for Symbolic Music Generation: Representations, Algorithms, Evaluations, and Challenges: https://dl.acm.org/doi/10.1145/3597493
- Foundation Models for Music: A Survey https://arxiv.org/abs/2408.14340

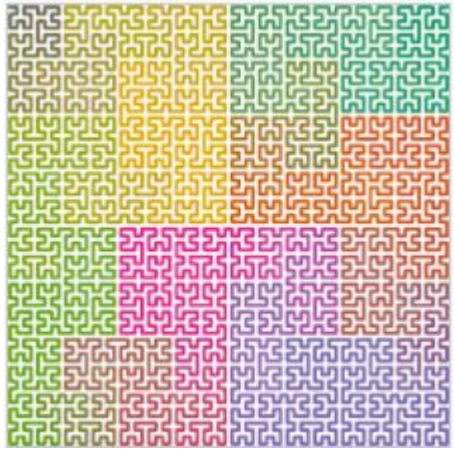
Grammars

- A **grammar** is a set of rules to expand high-level symbols into more detailed sequences of symbols (words) representing elements of formal languages
- The rules of the grammar can be designed by (e.g.) (1) hand; (2) examining a music corpus; (3) using evolutionary algorithms
- Some examples:
  - Evolutionary music composer integrating formal grammar
  - Grammar-based compression and its use in symbolic music analysis
  - Automatic Music Composition with Simple Probabilistic Generative Grammars
  - EvoComposer: An Evolutionary Algorithm for 4-Voice Music Compositions
- (Section 3.1 in highlighted survey)

Example: L-system for score generation

Hilbert curves are an example of a “space-filling” curve

The complex pattern can be drawn using a “rewrite system” where parts of the curve are repeatedly replaced by a production rule (from wikipedia):



Alphabet: A, B
Constants: F + −
Axiom: A
Production rules: A → +BF−AFA−FB+; B → −AF+BFB+FA−

"F" means "draw forward", "+" means "turn left 90°", "-" means "turn right 90°""A" / "B" are ignored during drawing

(animation: https://en.wikipedia.org/wiki/Hilbert_curve)

Przemyslaw Prusinkiewicz: Score Generation with L-Systems. ICMC 1986

Example: L-system for score generation

These rules can be adapted to generate music; here the **height** of each horizontal line becomes a pitch, and its **length** becomes its duration

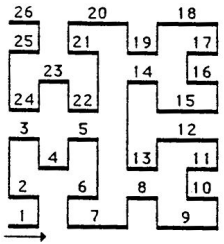


Fig. 3. Traversing the Hilbert curve.

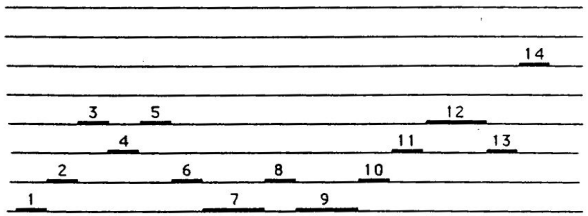


Fig. 4. The score associated with the Hilbert curve in the piano-roll notation.

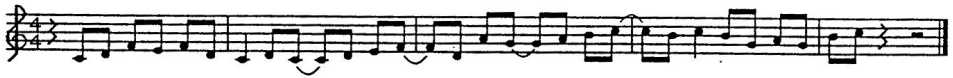


Fig. 5. The score associated with the Hilbert curve in the common musical notation.

Przemyslaw Prusinkiewicz: Score Generation with L-Systems. ICMC 1986

Example: L-system for score generation

Note: there's (probably) no deep association between Hilbert curves and music, though it does happen to have some reasonable properties:

- Successive notes will be close, with occasional jumps
- Most notes are short though some are longer (**Q:** what is the maximum interval and note length?)
- Will have repeated patterns

Still, such approaches arguably "cheat" somewhat e.g. by limiting composition to only notes of the major scale

Similarly: rule-based systems

(Recall some examples in Module 1) Core concepts:

- Fairly similar to a “grammar”: encodes musical theory rules into algorithmic form
- Uses (hierarchical) rule sets for different musical aspects:
  - Harmony rules (no parallel 5ths, etc.)
  - Melody construction
  - Rhythm patterns
- Implements constraints from music theory
- (Survey Section 3.2)

Example: Vivace (Thomas, 1985)

A rule-based AI system for four-part chorale harmonization

- *"The central problem facing student and computer alike is that simply adhering to all the rules is not enough."*
- *"The art of writing music is a **decision making** process."*
- *"A computer system that sets out to compose music as a human might compose needs clearly defined **materials**, appropriate procedures for combining these materials into a meaningful **vocabulary**, and an overall approach to shaping its vocabulary into a **language** and building a closed structure with that language."*

Marilyn Taft Thomas. "Vivace: A rule based AI system for composition", *ICMC*, 1985

Example: Vivace (Thomas, 1985)

A module system containing seven components

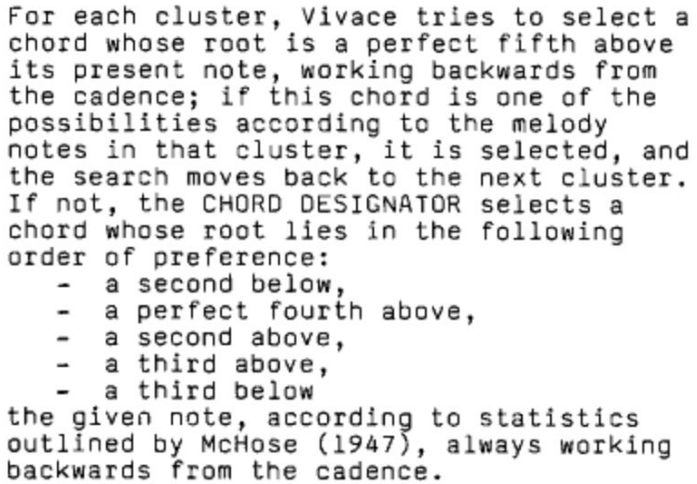
e.g. Melody Writer, Harmonic Rhythm Selector, Phrase Shaper, Chord Designator, ...

e.g.: Phrase shaper:

- Decide the chords for the beginning →establish the tonality
- The chords for the ending →an appropriate cadence

Chord designator:

- Search for the best path from the cadence back to the beginning of the phrase



For each cluster, Vivace tries to select a chord whose root is a perfect fifth above its present note, working backwards from the cadence; if this chord is one of the possibilities according to the melody notes in that cluster, it is selected, and the search moves back to the next cluster. If not, the CHORD DESIGNATOR selects a chord whose root lies in the following order of preference:

- a second below,
- a perfect fourth above,
- a second above,
- a third above,
- a third below

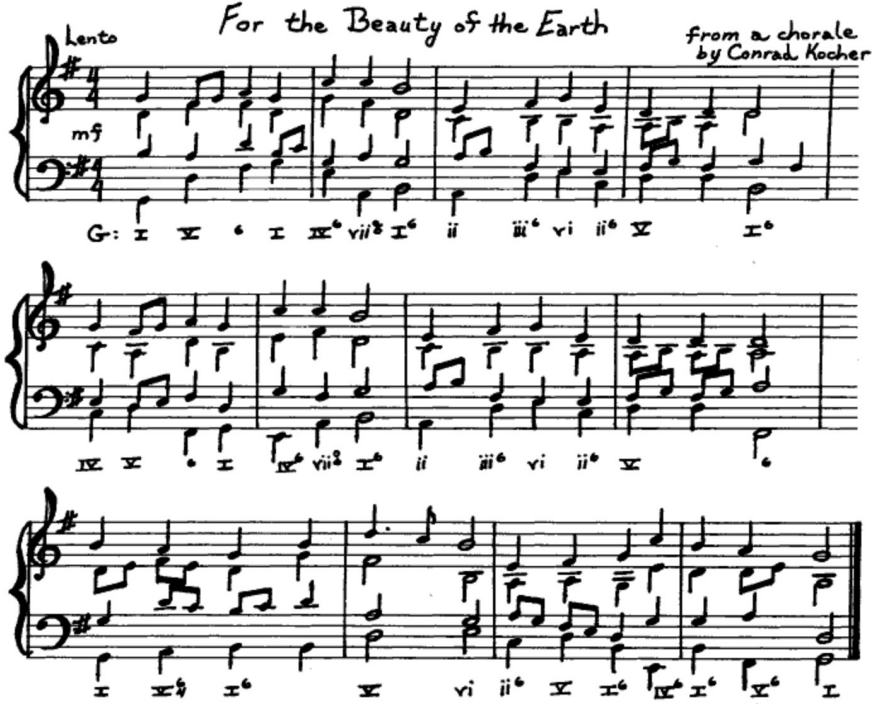
the given note, according to statistics outlined by McHose (1947), always working backwards from the cadence.

(details of chord designator)

Marilyn Taft Thomas. "Vivace: A rule based AI system for composition", *ICMC*, 1985

Example: Vivace (Thomas, 1985)

(**note:** in this example the system produces the harmonization rather than the melody)



Ex. 5. A recent harmonization by Vivace.

Marilyn Taft Thomas. "Vivace: A rule based AI system for composition", *ICMC*, 1985

Rules & grammars

Limitations:

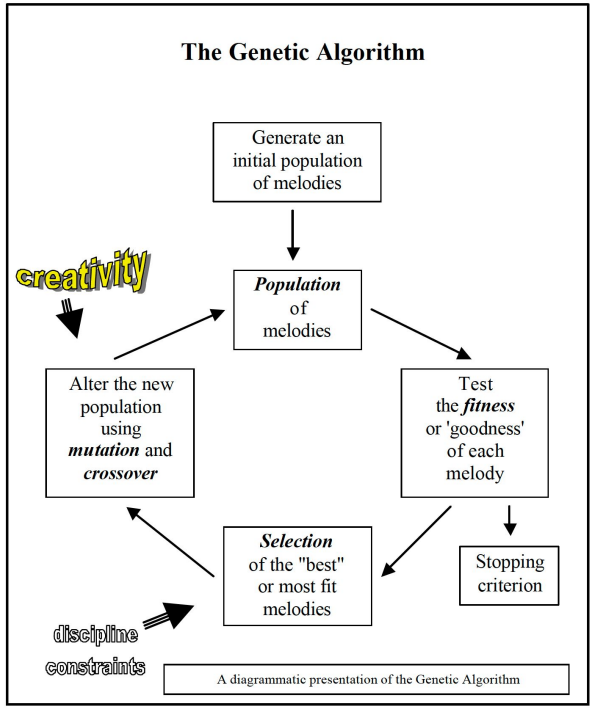
- Grammars fail to capture the internal coherency and subtleties required for music composition (Moorer, J. A. (1972). *Music and computer composition*)
- Difficult to manually define a set of grammatical rules that will produce good compositions
- General grammars seem to be very difficult to implement effectively, except for very simple toy systems

Evolutionary / genetic algorithms

- Have been combined with almost every other method
- a changing set of candidate solutions undergoes a repeated cycle of evaluation, selection and reproduction with variation
- Steps:
  - **Initialization:** generate the candidate solutions of the initial set (user-specified examples or randomly)
  - **Evaluation:** each candidate is then evaluated using a fitness function, a heuristic rule to measure its quality.
  - **Selection:** generate a new set of candidate solutions by copying a number of times probabilistically proportional to its fitness
  - **Reproduction with variation:** mutation or recombination operators
- (Survey Section 3.5)
- See also cellular automata (Survey Section 3.6)

Example: genetic algorithms (Towsey et al., 2001)

- **Initialization**: generate the candidate solutions of the initial set
- **Evaluation**: each candidate is then evaluated using a **fitness function**, a heuristic rule to measure its quality
- **Selection**: generate a new set of candidate solutions by copying a number of times probabilistically proportional to its fitness
- **Reproduction with variation**: mutation or recombination operators



Michael Towsey, Andrew Brown, Susan Wright and Joachim Diederich. "Towards melodic extension using genetic algorithms," *Educational Technology & Society, 4 (2), 54–65*, 2001

Example: genetic algorithms (Towsey et al., 2001)

Fitness function: a weighted sum of **features** of the composition. E.g.:

- **Pitch variety:** The ratio of distinct pitches to notes; this feature is a measure of the diversity of the pitch class set used in writing the melody
- **Key Centered**: The proportion of quanta where the pitch is primary, that is either tonic or dominant; the feature provides an indication of how strongly the melody has a sense of key
- **Contour Stability**: The proportion of intervals for which the following interval is in the same direction; this is a measure of stability in melodic direction
- **Note Density**: The ratio of notes to quanta; this feature indicates the sparseness versus 'business' of a melody
- etc.....

Michael Towsey, Andrew Brown, Susan Wright and Joachim Diederich. "Towards melodic extension using genetic algorithms." *Educational Technology & Society, 4 (2), 54–65*, 2001

Example: genetic algorithms (Towsey et al., 2001)

Note: Fitness functions (pitch variety; key centered; contour stability; note density), etc. are attempts to assess musical quality without having to ask humans to listen to the the music

While evolutionary algorithms don't show up in this class, "fitness functions" like those above have survived as a means of doing offline evaluation of generated music

Michael Towsey, Andrew Brown, Susan Wright and Joachim Diederich. "Towards melodic extension using genetic algorithms." *Educational Technology & Society, 4 (2), 54–65*, 2001

Aleatoric (random) music

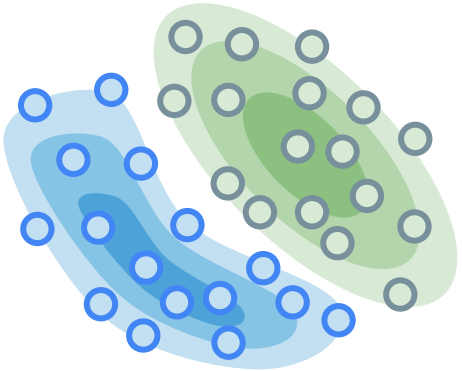
- “Aleatoric” just means “depending on chance”
- We already studied an example in Module 1 (Musikalisches Würfelspiel):
  - Each bar of a piece of music (a waltz) has 11 possible options
  - Roll a pair of dice (2-12) to choose which bar to play
- Also saw some more “truly random” methods, e.g. based on star charts

https://libraries.mit.edu/news/mozart-rolls/22909/

Aleatoric (random) music

Why would we want randomness?

- Generally we won't want "true" randomness as such; rather our goal with generative modeling is to *learn the underlying distribution and then* ***sample*** *from it*
- That is, we want a random process that will be likely to generate "musical" output and unlikely to generate "non-musical" output
- Generative models (LLMs, stable diffusion, etc.) typically involve this type of randomness



Generative models learn the underlying distribution

$P(x)$ or $P(x|y)$

Example: Markov chains

- “What happens next depends only on the state of affairs now.” (wikipedia)
- State (e.g. notes) and transition probability matrix, which is learned (measured) from data
- Transition probabilities can also be learned from multiple steps (i.e., “higher-order” Markov chains)
- More details in next section
- (and in Section 3.3 of the referenced survey)

1st–order matrix

Note	A	C♯	E♭
A	0.1	0.6	0.3
C♯	0.25	0.05	0.7
E♭	0.7	0.3	0

Markov chains

Limitations:

- Low-order Markov chains produce strange, unmusical compositions that wander aimlessly (see workbook 3!)
- High-order chains essentially “rehash” musical segments from the corpus, and are also computationally expensive to train
- **But:** still useful to teach the “token generation based on recent tokens” way of thinking about generative modeling, which will be the basis of most “fancier” methods

Early attempts with Artificial Neural Networks (ANNs)

"*Using a set of examples (input patterns) to train the network in order to use it to recognize or generate similar patterns*." (Fernández & Vico, 2013)

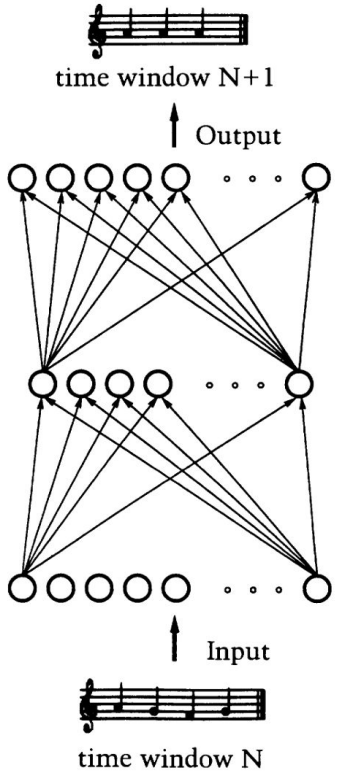
- Todd (1989), who used a three-layered recurrent ANN designed to produce a temporal sequence of outputs encoding a **monophonic melody**
- Instead of absolute pitches (as in Todd's work) in the mapping, for composing music in Bach's Style, Duff (1989) published another early example encoding **relative pitches**
- Mozer (1991) developed a recurrent ANN with a training program devised to capture both **local and global patterns** in the set of training examples

Peter M. Todd. "A connectionist approach to algorithmic composition," *Computer Music Journal*, 1989
M.O. Duff. "Backpropagation and Bach's 5th cello suite (Sarabande)," *IJCNN*, 1989
Michael C. Mozer. "Music and Connectionism, chap. Connectionist music composition based on melodic, stylistic, and psychophysical constraints," *The MIT Press, Cambridge, pp. 195–211*, 1991

Example: monophonic melody generation (Todd, 1989)

Network design (1)

- "*A network design which can learn to associate time windows (e.g. measures) in a piece of music with the following time windows. Here, one measure as input produces the following measure as output.*"
- A sliding window of successive time-periods of fixed size (i.e. measures) are presented to the network simultaneously.

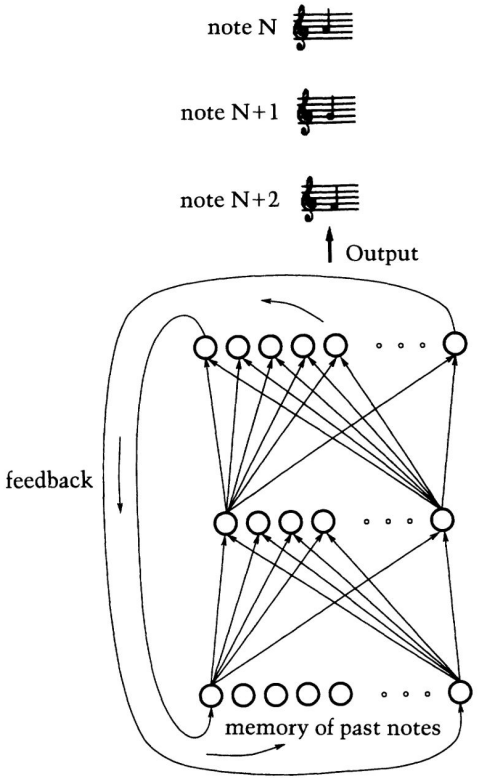


Peter M. Todd. "A connectionist approach to algorithmic composition," *Computer Music Journal*, 1989

Example: monophonic melody generation (Todd, 1989)

Network design (2)

- "*A sequential network design which can learn to produce a sequence of notes, using a memory of the notes already produced. This memory is provided by the feedback connections shown, which channel produced notes back into the network.*"
- Treat music as a sequential phenomenon, with notes being produced one after another in succession.

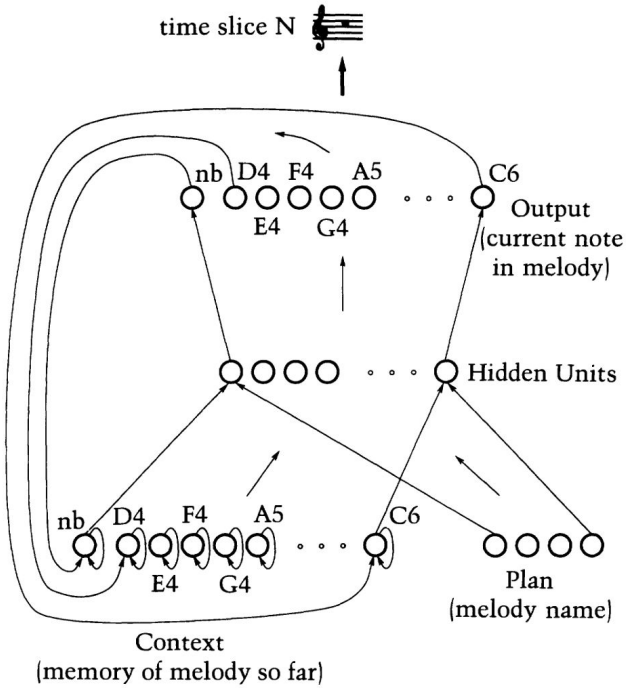


Peter M. Todd. "A connectionist approach to algorithmic composition," *Computer Music Journal*, 1989

Example: monophonic melody generation (Todd, 1989)

Network design (3)

- "*The sequential network design used for compositional purposes. The current musical representation requires note-begin (nb) and pitch (D4-C6) units. Each network output indicates the pitch at a certain time slice in the melody.*"
- **Plan Units**: sequence to be learned
- **Context Units**: maintain the memory

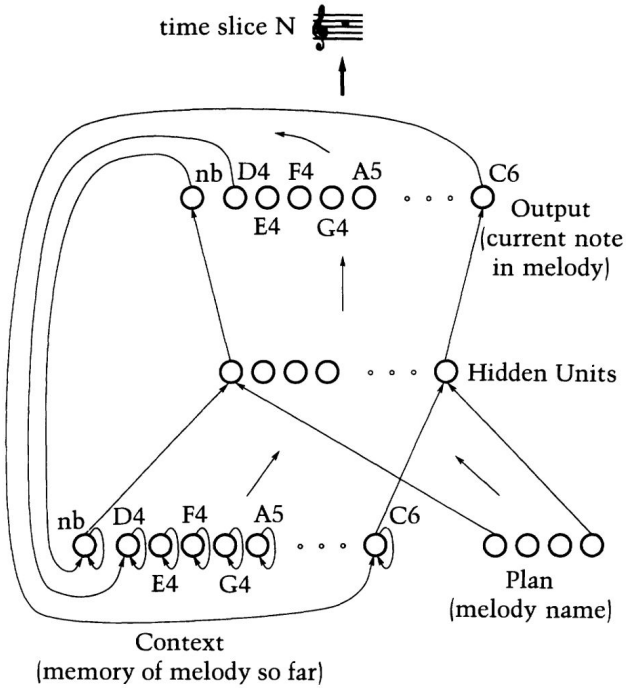


Peter M. Todd. "A connectionist approach to algorithmic composition," *Computer Music Journal*, 1989

Example: monophonic melody generation (Todd, 1989)

Melody representation

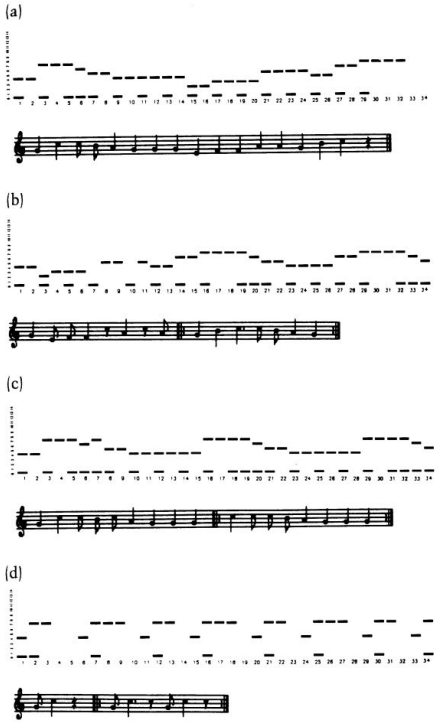
- only capture the **pitch** and **duration** of each note; ignore tempo and loudness
- **pitch**: actual value vs intervals
  - actual pitch: A, B, C
  - pitch intervals: A, +2, +1
- **intervals**
  - not restricted in the pitch range it can cover
  - melodies are encoded in a more-or-less key-independent fashion→let the network discover common structure between melodies
  - but, easy to make global errors (i.e., an interval error can change the sequence globally)



Peter M. Todd. "A connectionist approach to algorithmic composition," *Computer Music Journal*, 1989

Example: monophonic melody generation (Todd, 1989)

Example



Peter M. Todd. “A connectionist approach to algorithmic composition,” *Computer Music Journal*, 1989

Take-homes

- Plenty more in survey!
- Point so far is just to get a sense of the complexities of the problem
- Also note that the “new” way of approaching symbolic models (hint: LLMs) is not necessarily different from how some people were already approaching these problems (though the models are more powerful)

Symbolic music generation

3.2: Introduction to next-token prediction approaches

Music as a *stochastic process*

Slides based on

“Generative AI for Music and Audio Creation” (Michigan):

https://hermandong.com/teaching/pat498_598_fall2024/

and

https://scholarship.claremont.edu/cgi/viewcontent.cgi?article=1848&context=jhm

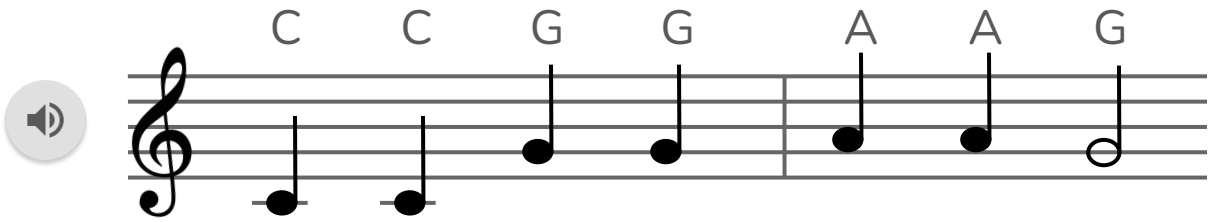
and slides from **“Intro to Computer Music”:**

https://www.cs.cmu.edu/~15322/

Music as a *stochastic process*

A *stochastic process* characterizes a sequence of random decisions

Music can be interpreted as the result of a stochastic process. Example:



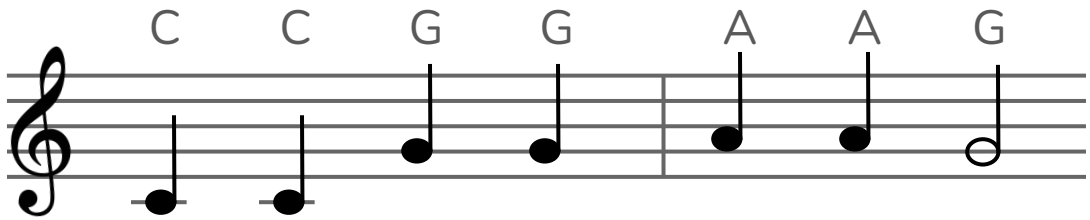
Roll a 🎲 seven times. Map outcomes {1, 2, 3, 4, 5, 6} to pitches {C, D, E, F, G, A}

Question: What is the *likelihood* that we get Twinkle Twinkle Little Star?

Answer: Exceedingly low. ⅙ · ⅙ · ⅙ · ⅙ · ⅙ · ⅙ · ⅙, or 1 in 279936

Music as a *stochastic process*

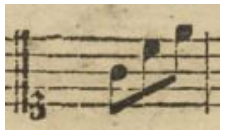
But! Maybe it shouldn't be quite so unlikely?



- Maybe repeated (C→C) or "nearby" (C→A) notes are more likely than other options?
- Maybe some larger leaps (e.g. C→G) are more likely than others?
- **So:** we need a **model** of the likelihood: the model needs to be expressive enough to understand rules and musical context, etc.

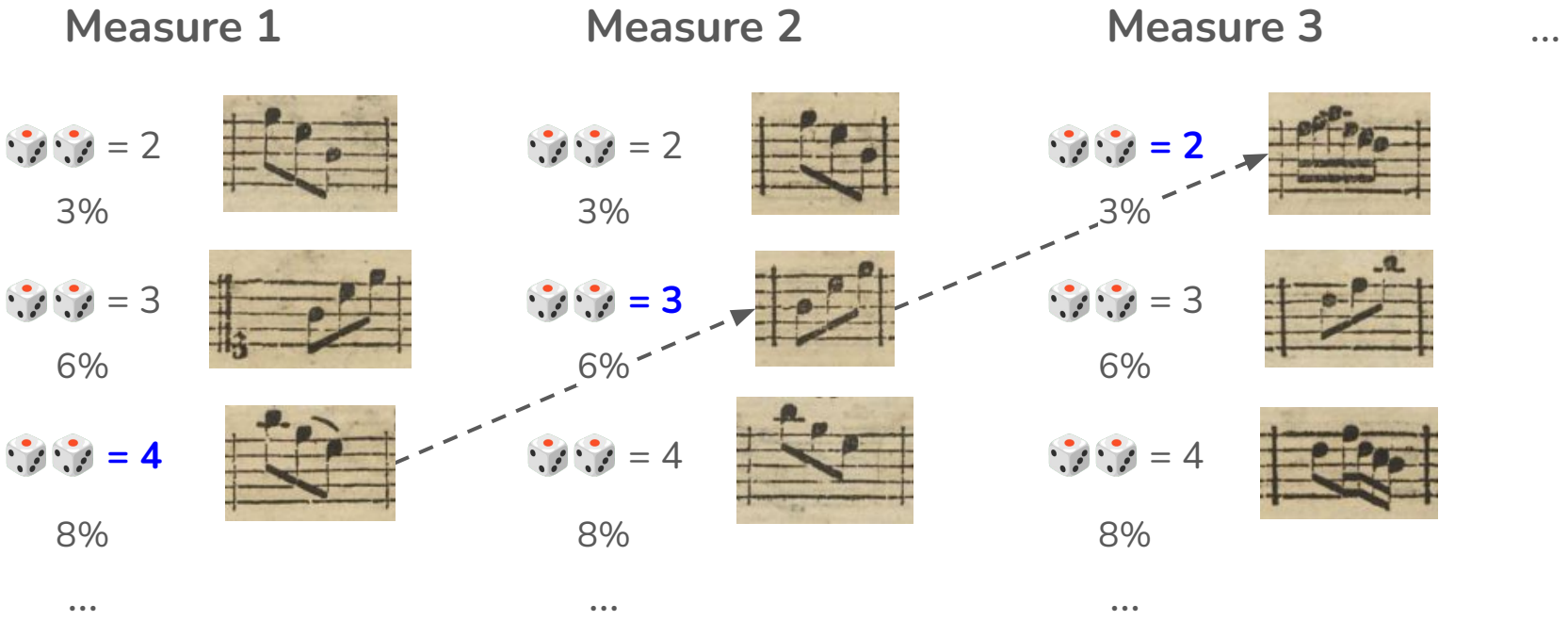
Stochasticity in music (classical)

Example: Mozart's *Musikalisches Würfelspiel* ("dice music", 1792)

Measure 1	Measure 2	Measure 3	...
🎲🎲 = 2 	🎲🎲 = 2 	🎲🎲 = 2 	
🎲🎲 = 3 	🎲🎲 = 3 	🎲🎲 = 3 	
🎲🎲 = 4 	🎲🎲 = 4 	🎲🎲 = 4 	
...	...	...	

Stochasticity in music (classical)

In stochastic processes, some outcomes can be more likely than others:



Stochasticity in music (classical)

Mozart probably (?) didn't consider the different probabilities of dice-roll outcomes when writing *Musikalisches Würfelspiel;* he just wanted to introduce randomness

But we'd like to model random processes in such a way that "good" musical choices are ***likely*** and "bad" musical choices are ***unlikely***

That is, we'd like to:

1. **Model** the likelihood that a particular set of events (e.g. notes) would occur in a "real" piece of music
2. **Generate** a piece of music that is likely under this model (i.e., "musical")

Markov chain approaches

We'll build our first model of "music likelihood" using **Markov chains**

But I already said these don't work well?!?

Markov chains:

- Are useful for teaching **next token** approaches to music generation, which form the basis of "state-of-the-art" models
- Easy to teach and implement
- Can work well (enough) once we look at more complex variants

Introduction to Markov chains

Given a sequence of events (among some discrete set S), a Markov Chain assumes that the probability of the next event given the event history can be written purely in terms of the previous event:

$$p(i^{(t+1)} = i \mid i^{(t)} \ldots i^{(1)}) = p(i^{(t+1)} = i \mid i^{(t)})$$

next event (e.g. note)

event history

previous event

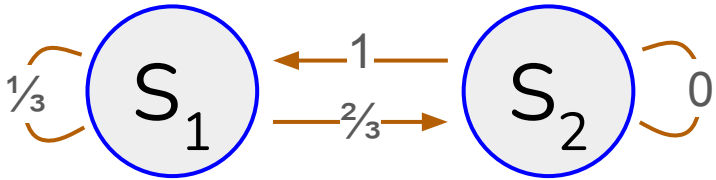
Introduction to Markov chains

In the context of musical sequences, we might use such a method to estimate what note or chord tends to follow from the previous one

(we'll discuss later why this assumption is quite limiting!)

Introduction to Markov chains

We can visualize the Markov chain as a collection of (1) *states*, e.g., S_1, S_2, and (2) *transition probabilities*: likelihood of moving from $S_i \rightarrow S_j$

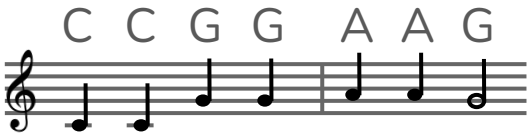


Markov chains are a stochastic process where output behavior depends on context

They are rudimentary language models!

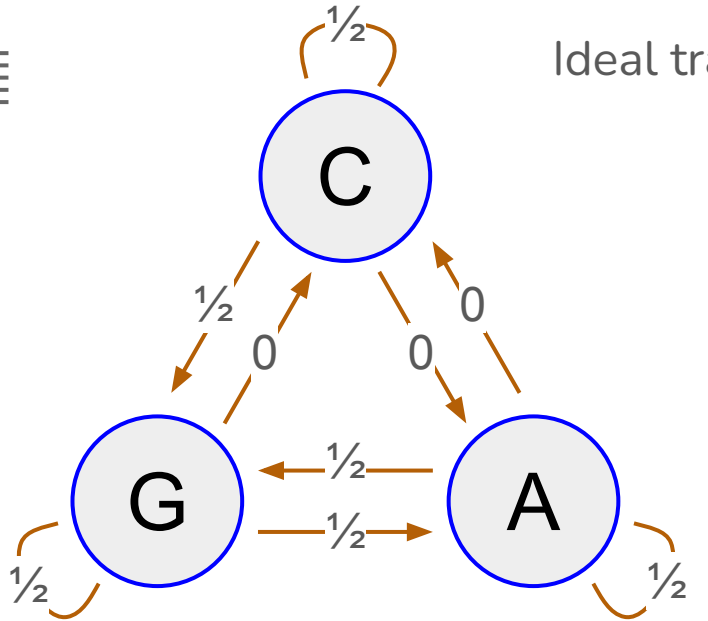
How to compute the *ideal* transition probabilities?

Define "*ideal*" as those which *maximize the likelihood* of the data; e.g.:



States:
{C, G, A}

Transitions:
C → C, C → G
G → G, G → A
A → A, A → G
G → ? (ignored)



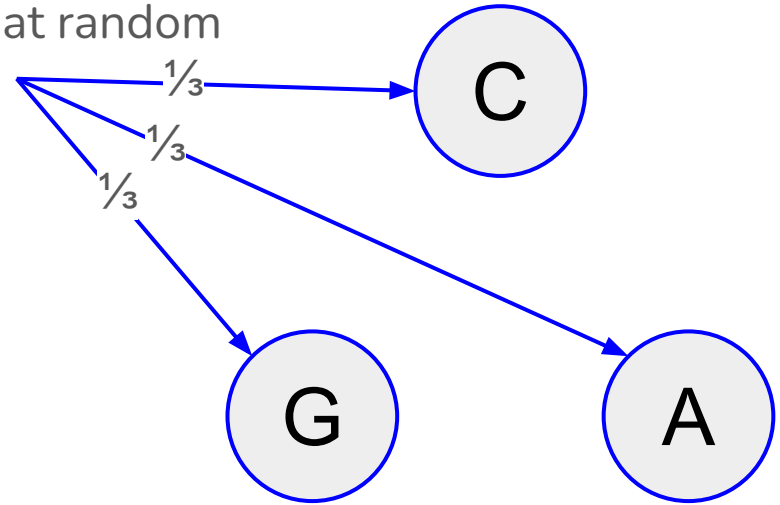
Ideal transition prob for $S_i \rightarrow S_j$:

$$\frac{\text{count}(S_i \rightarrow S_j)}{\text{count}(S_i \rightarrow \text{any})}$$

How do we generate from this process?

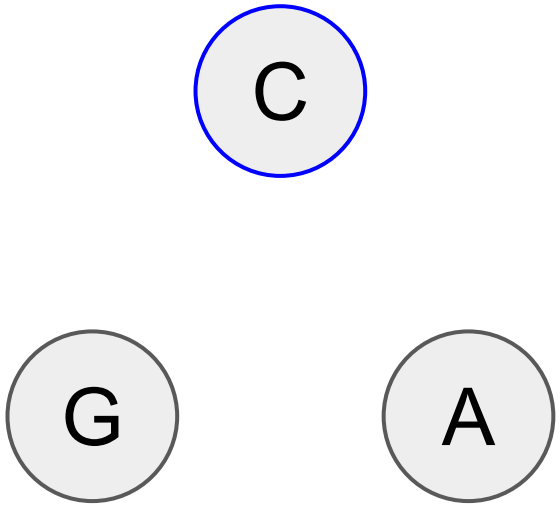
Running sequence:

Pick a starting state uniformly at random



How do we generate from this process?

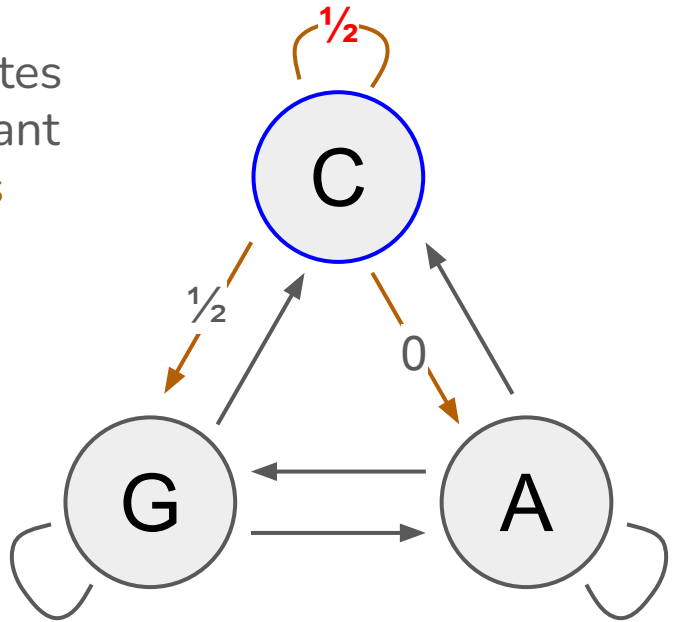
Running sequence: C (⅓)



How do we generate from this process?

Running sequence: C (⅓)

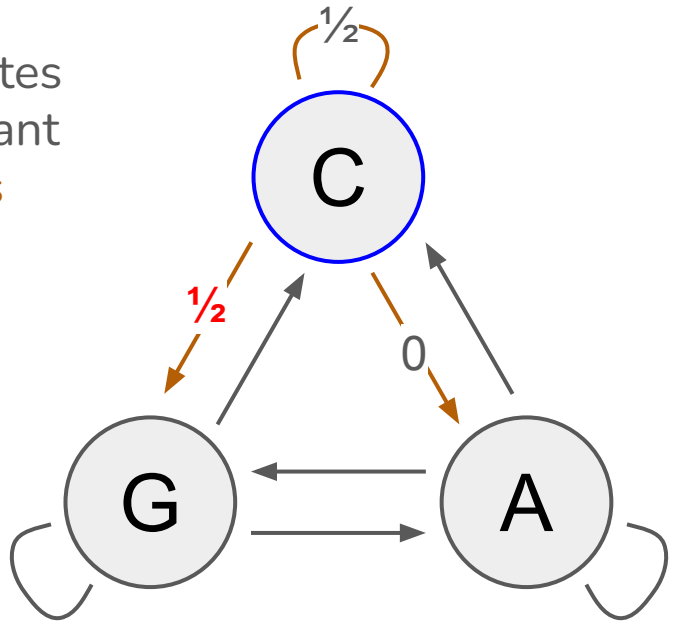
Select subsequent states at random using relevant transition probabilities



How do we generate from this process?

Running sequence: C (⅓), C (½)

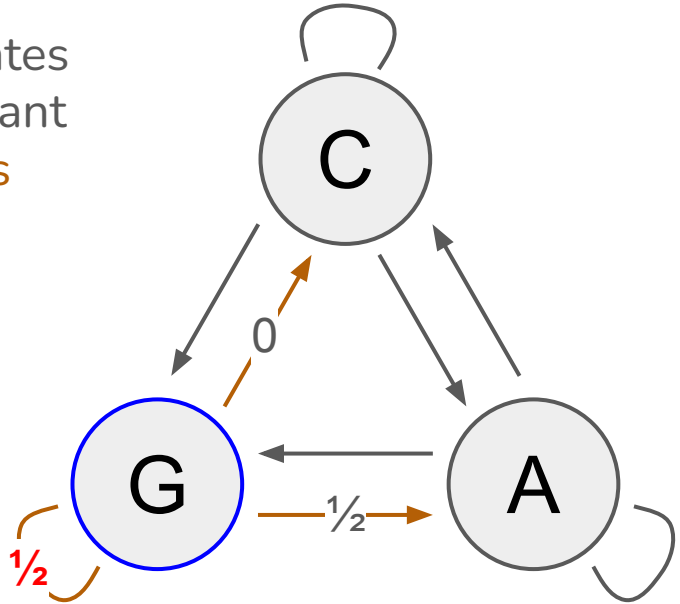
Select subsequent states at random using relevant transition probabilities



How do we generate from this process?

Running sequence: C (⅓), C (½), G (½)

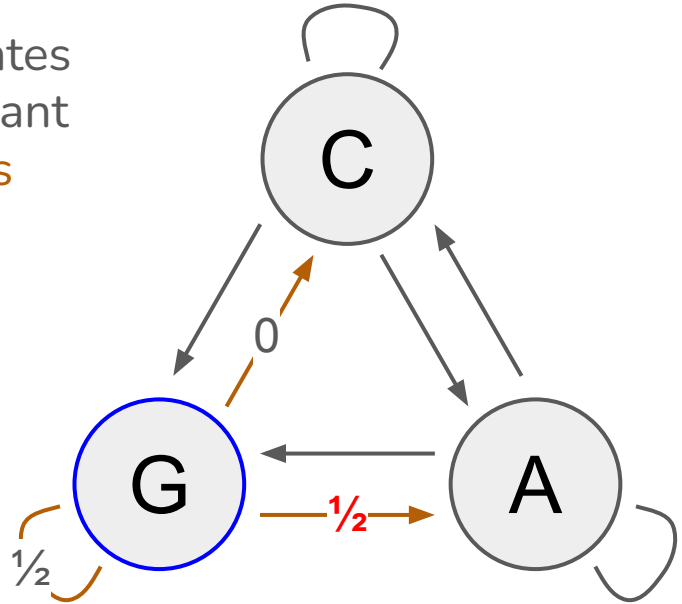
Select subsequent states at random using relevant transition probabilities



How do we generate from this process?

Running sequence: C (⅓), C (½), G (½), G (½)

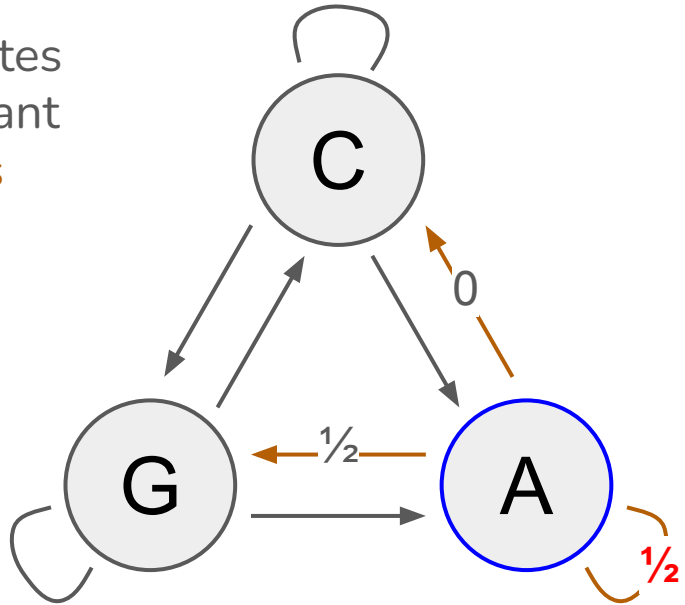
Select subsequent states at random using relevant transition probabilities



How do we generate from this process?

Running sequence: C (⅓), C (½), G (½), G (½), A (½)

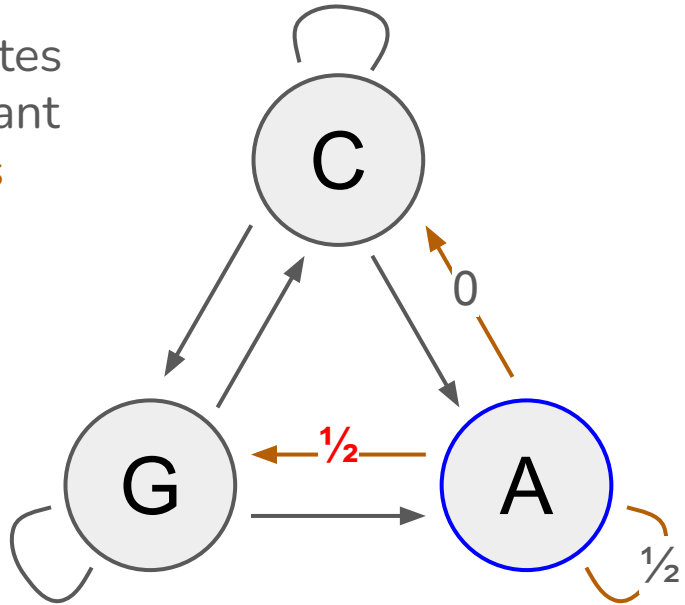
Select subsequent states at random using relevant transition probabilities



How do we generate from this process?

Running sequence: C (⅓), C (½), G (½), G (½), A (½), A (½)

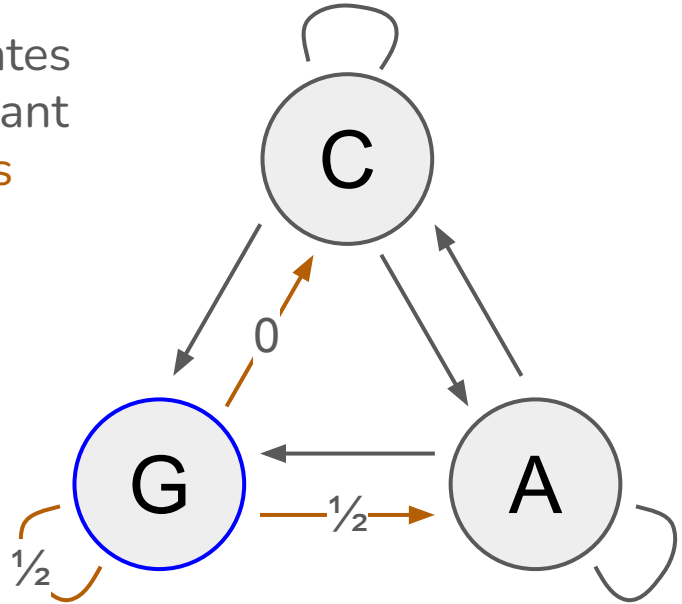
Select subsequent states at random using relevant transition probabilities



How do we generate from this process?

Running sequence: C (⅓), C (½), G (½), G (½), A (½), A (½), G (½)

Select subsequent states at random using relevant transition probabilities

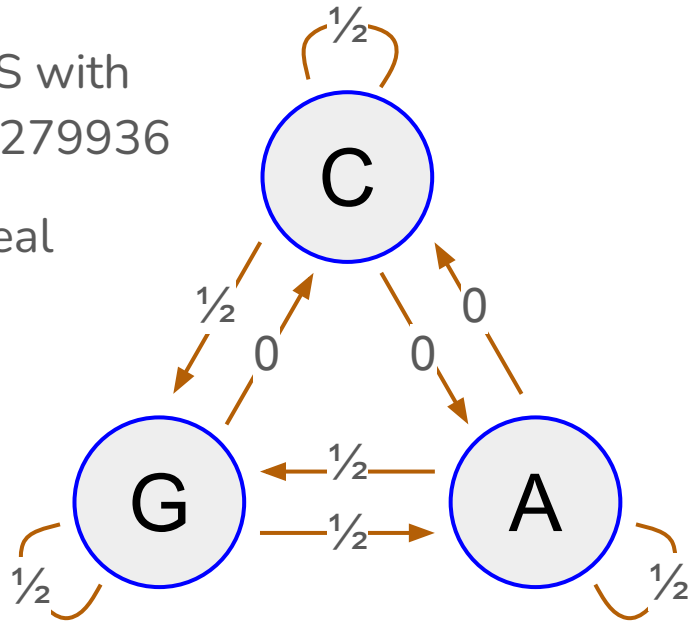


Context-free vs context-dependent

Running sequence: C (⅓), C (½), G (½), G (½), A (½), A (½), G (½)

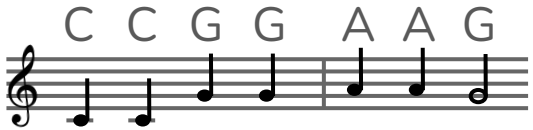
Recall: likelihood of TTLS with context-free model: 1 in 279936

Likelihood of TTLS w/ ideal Markov model: **1 in 192**



Generalizing Markov chains → n-grams

N-grams generalize Markov chains to longer contexts



unigrams
1-gram

C = 2/7
G = 3/7
A = 2/7

bigrams
2-gram (Markov chains)

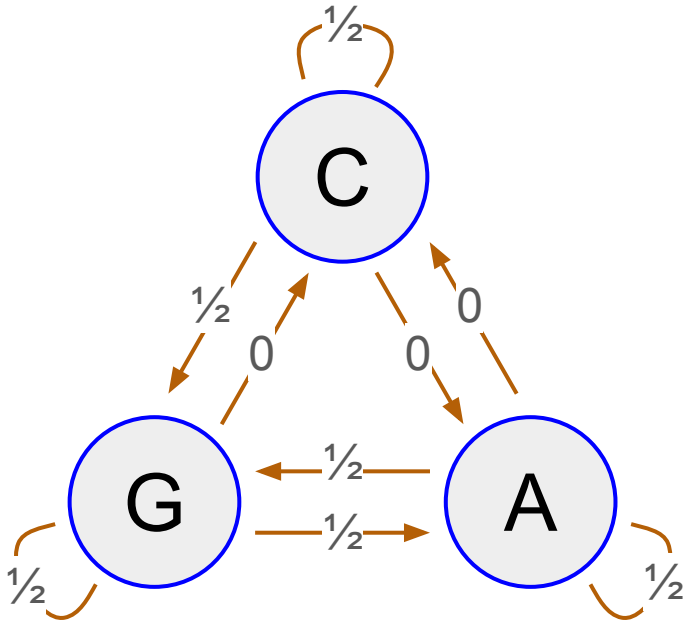
C→C = 1/2
C→G = 1/2
C→A = 0
G→C = 0
G→G = 1/2
G→A = 1/2
... (3 more)

trigrams
3-gram

{C,C}→C = 0
{C,C}→G = 1
{C,C}→A = 0
{C,G}→C = 0
{C,G}→G = 1
{C,G}→A = 0
... (21 more)

For N-gram, num transition probs = num states^N

Markov chains as matrices



Context \ Next	C	G	A
C	½	½	0
G	0	½	½
A	0	½	½

Worked example

(see workbook3.ipynb)

Markov chain approaches

Basic approach:

- Extract sequences of notes or chords from data
- Extract *bigrams* of notes or chords (or longer, if you want a higher-order Markov chain)
- Compute transition probabilities from these bigrams
- Sample a new song based on these probabilities

Markov chain approaches

I extracted chord progressions from 456 jazz standards from *The Jazzomat Research Project*

(see e.g. https://jazzomat.hfm-weimar.de/dbformat/synopsis/solo7.html)

This data is available in the dataset directory (chords.json)

Markov chain approaches

Parsed data (one row) looks like this:

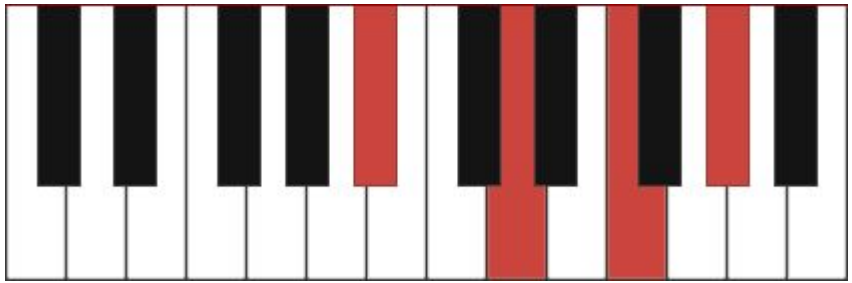

```
{'title': 'Benny Carter “I Got It Bad”', 'chords': [['Dbj7'],
['Bb-7'], ['Eb7'], ['Eb7'], ['Eb-7'], ['F7', 'Bb7', 'Eb7',
'Ab7'], ['Db6'], ['G7911#']], [['Gbj7'], ['F#j7'], ['Gb-6'],
['B7'], ['Dbj7', 'B7'], ['F-7', 'Bb7'], ['Eb-7'], ['Ab7']],
[['Dbj7'], ['Bb-7'], ['Eb7'], ['Eb7'], ['Eb-7'], ['F7', 'Bb7',
'Eb7', 'Ab7'], ['Db6'], ['Eb-7', 'Ab7']], [['Gbj7'], ['Gbj7'],
['Gb-6'], ['B7'], ['Dbj7', 'B7'], ['F-7', 'Bb7'], ['Eb-7'],
['Ab7']], [['Dbj7'], ['Bb-7'], ['Eb7'], ['Eb7'], ['Eb-7'], ['F7',
'Bb7', 'Eb7', 'Ab7'], ['Eb-7', 'Ab7'], ['Dbj7'], ['Dbj7'],
['Dbj7']]], 'signature': '4/4', 'key': 'Db-maj'}
```

Markov chain approaches

Reminder (?): chord names

A chord is just a set of notes that are played together; they describe the underlying harmony (not the melody) of the piece

So really we're building a system for *chord progressions*



Bb7 ("B flat 7")

Markov chain approaches

Next, extract and count bigrams:

```
[(85, ('Eb-7', 'Ab7')),
 (89, ('Bb-7', 'Eb7')),
 (92, ('Bb7', 'Ebj7')),
 (97, ('A-7', 'D7')),
 (97, ('G7', 'C-7')),
 (104, ('D-7', 'G7')),
 (108, ('F7', 'Bbj7')),
 (146, ('F-7', 'Bb7')),
 (162, ('G-7', 'C7')),
 (202, ('C-7', 'F7'))]
```

Markov chain approaches

For this to work, we'll have to limit our vocabulary to a set of chords we "like"; here we'll just pick the chords that appear in one song:

```
dictionary = set(flatDataset[3])
```

```
{'Ab7','Abj7','Ao7','Bb-7','Bb7','C7','Eb6','Eb7','F7'}
```

Q: Why do we need to do this?

Q (bonus): What key is this piece in?

Markov chain approaches

- Limit our bigrams to this vocabulary, and convert to probabilities (see code)
- And sample a new piece!

```
> sample(10)

['Bb7', 'Eb7', 'Eb7', 'Abj7', 'Ab7', 'Eb7', 'Ab7', 'Eb7',
'Eb7', 'Ab7']
```

Markov chain approaches

Some limitations:

- We had to build a limited dictionary for this to work; requires some music theory to explain, but without this, the piece would wander between keys
- Could use a higher-order Markov chain, though (probably) wouldn't help much

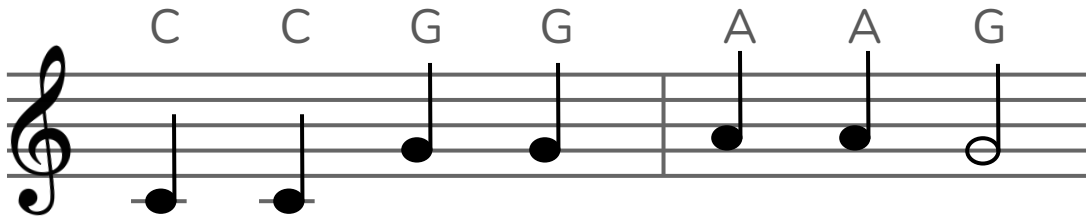
Markov chain approaches

So why talk about models like this?

- It can (sort of be made to) work okay!
- Simplest possible demonstration of the basic setup of symbolic generation: for fancier models (including e.g. transformers), the essential idea is basically the same
- Practically speaking: I wanted to teach a model that everyone can run on their laptop; you can try out similar but more complex models for Assignment 2...

Evaluation: Perplexity

How do we evaluate whether a music generation model is “good”? Earlier we said that a good model of music should be expressive enough to understand rules and musical context, etc.



- Maybe repeated (C→C) or “nearby” (C→A) notes are more likely than other options?
- A Markov chain can capture the above rules just fine!

Perplexity

What we mean when we say it “understands the rules” is the following:

It will be more likely to generate twinkle twinkle little star (or songs similar to what it was trained on) compared to “random” music

We can also reverse this problem: rather than generating, we can ask the Markov chain what the probability of generating a certain song is:

- How likely is twinkle twinkle little star to be generated by the model?
- How likely is a random sequence of notes?

Perplexity

Put differently, we can ask the Markov chain (or any model) whether it is “surprised” to see a sequence of notes, or how well it can guess what comes next; i.e., does it think that sequence is likely or not?

Formally, this can be measured by the ***perplexity:***

$$exp(-\frac{1}{N}\sum_{i=1}^{N} log(p(w_i)))$$

Perplexity

Or for our Markov model:

$$exp(-\frac{1}{N}\sum_{i=1}^{N} log(p(w_i|w_{i-1})))$$

The exact definition may vary a bit, but the point is that we're just measuring **the (log) probability that the model assigns to each token**

A "good" model should assign a high probability to (held-out) examples of music

(note that high probability = *low* perplexity)

Perplexity

Why is this relevant/important?

- This is the main approach used to evaluate token-generation approaches (especially language models): a “better” model is one that assigns lower perplexity to held-out data
- This same type of equation is what’s used to *train* models (including the more complex models we’ll see later): i.e., *what model parameters will lead to the lowest perplexity?*
- (for our Markov chain, our “parameters” are just measurements of pairwise frequencies, but we could e.g. have fit them using gradient ascent if we wanted)

Chorale harmonization

Let's finish this section by looking at ways to extend Markov models: we'll look at a ***conditional*** music generation task:

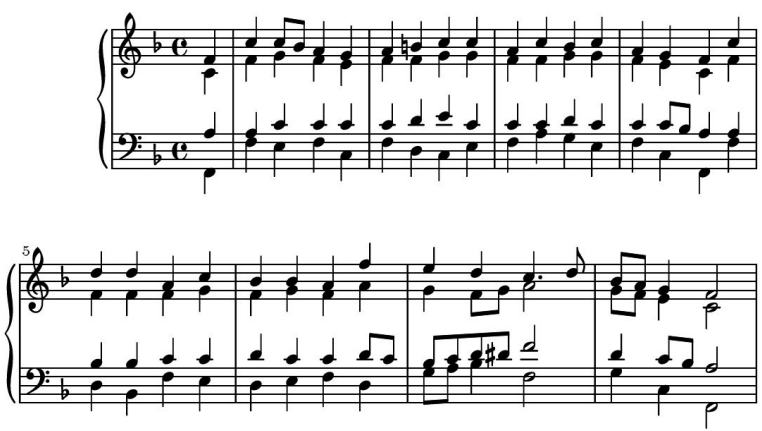
- Given a **melody (condition)**, estimate the underlying **chords**
- (this task is called "harmonization")
- see: **Harmonising Chorales by Probabilistic Inference** (https://papers.nips.cc/paper_files/paper/2004/hash/b628386c9b92481fab68fbf284bd6a64-Abstract.html) which uses a dataset of chorales composed by Johann Sebastian Bach to train (Hidden) Markov Models

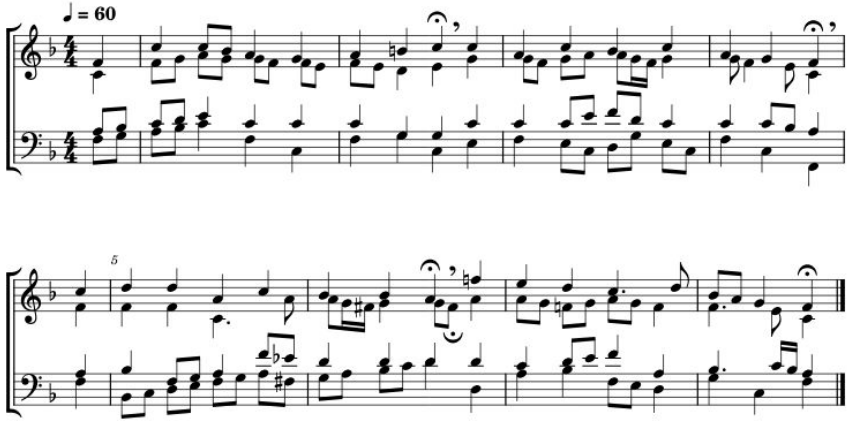
Chorale harmonization

- A traditional part of the theoretical education of Western classical musicians
- **Task definition:** Given a melody, create three further lines of music which will sound pleasant when played simultaneously with the original melody
- “Pleasant?”: lots of “rules” or suggestions
  - These pieces are for singers, so each part should its own melodic line (which mostly means they should have notes that are close to each other)
  - The melodies should use notes based on the implied underlying chord (requires musical knowledge!)
  - Different singers (e.g. tenor and baritone) should not cross each other between notes
  - Avoid parallel 4ths, 5ths, and octaves (why?)
  - Avoid the “tritone” (why?)
- Following these rules is enough to produce something that sounds okay

Chorale harmonization

An example harmonization result for BWV 438 “Wo Gott zum Haus nich gibt sein Gunst”, and Bach’s actual version (from musescore)





which is which?!? (**hint:** which one sounds better?)

Hidden Markov Models (HMMs)

We'll solve this using a ***Hidden* Markov Model (HMM)**

This is an extension of a Markov model in which the model has an internal or "hidden" state

The next token now depends on this hidden state (rather than just the last token), and the hidden state may also change

Hidden Markov Models (HMMs)

What does “hidden” mean? (an example from CSE 250A)

We **“observe”** some states of a puppy:

- $O_i \in$ {sleeping; eating; barking; waiting by door; etc}

What are its actual (**“hidden”**) needs?

- $S_i \in$ {playful; hungry; tired; ready to burst}

A HMM is a statistical model where observed states (e.g. waiting by door) depend on hidden states (e.g. playful), allowing for the prediction of a sequence of unknown variables S_i from a set of observed variables O_i:

- $P(S_i | O_1 , O_2 , ..., O_i)$

Hidden Markov Models (HMMs)

What about in a musical context? Hidden states might mean:

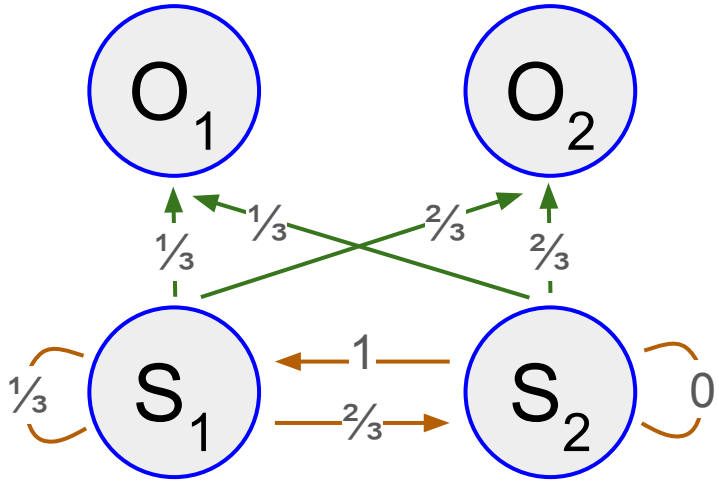
- What key is the piece currently in (i.e., what set of notes might work?)
- What is the underlying chord the harmony is based on, or its chord “function” (sorry, music theory!)
- Are we in the “A” or the “B” section?
- Or more complex things: will the singer run out of breath?

Note: much like the puppy example, these mightn’t exactly be “hidden” to someone who knows music theory; we call them hidden states to indicate that they are **not the states that the model outputs**

Hidden Markov Models (HMMs)

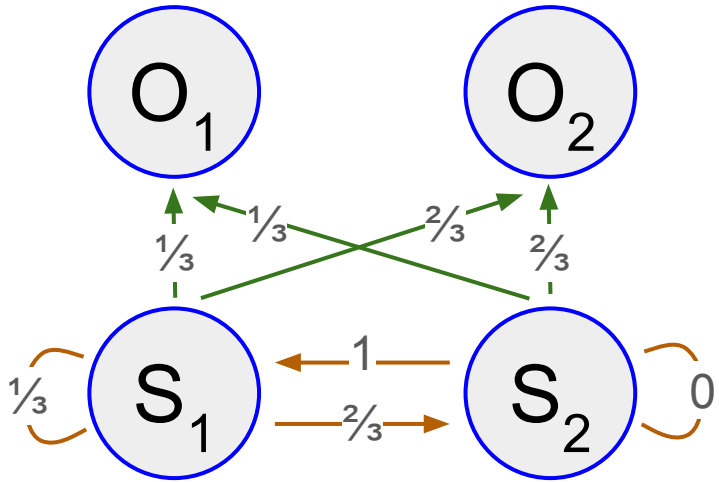
The Hidden markov model is a Markov model in which the **observations O** are dependent on a **hidden Markov process S**

- **transition probabilities**: likelihood of moving from $S_i \rightarrow S_j$;
- **emission probabilities**: likelihood of moving from (generating) $S_i \rightarrow O_i$;
- initial state S_0



HMMs for chorale harmonization

- **Model**
  - melody line: a sequence of **observed** events O_i
  - harmonisation (chords): a sequence of **hidden** events S_i



HMMs for chorale harmonization

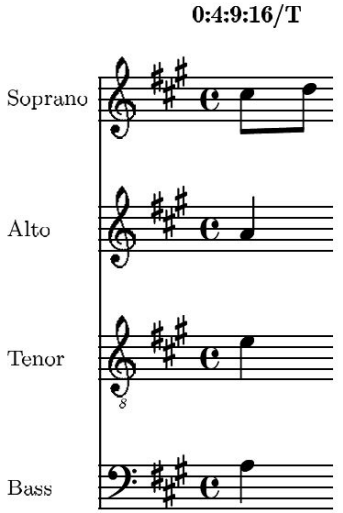
How does it work?

- A chorale has an underlying **harmonic structure**, and the individual notes of the melody line are chosen to be compatible with this harmonic state at each time step
- The HMM models how a visible melody line is emitted by a hidden sequence of harmonies
- Separate models for chorales in **major and minor keys**, since these groups have different harmonic structures
- **Assumptions**:
  - the harmonic state (i.e. notes) does not change during a beat
  - first-order Markov assumptions on transition / emission probabilities

HMMs for chorale harmonization

How to represent music data?

- **Observed states**: absolute pitch of the note
- **Hidden states**: a list of pitch intervals + harmonic labels
  - “By using intervals we represent the relationship between the added notes and the melody at a given time step, without reference to the absolute pitch of the melody note.”
  - “Adding harmonic labels means that our hidden symbols not only identify a particular chord, but also the harmonic function that the chord is serving.”
- **Example**: the alto, tenor and bass notes (an A major chord) are respectively 4, 9, and 16 semitones below the soprano melody; the harmonic label is ‘T’, labelling this as functionally a ‘tonic’ chord
- (don’t worry too much about details; basically the hidden states are *roughly* the underlying chords)



HMMs for chorale harmonization

- **Learning**
  - transition and emission probabilities are learned (or really just “measured”) directly from observations in the training data of harmonisations (i.e., hidden states are measurable in the training data)
- **Inference**
  - given a new melody line, the Viterbi algorithm can be used to find the most likely hidden state sequence (i.e. harmonisation) (see: CSE 250A)
  - harmonisation system will “plan” over an entire harmonisation rather than simply making immediate choices based on the local context

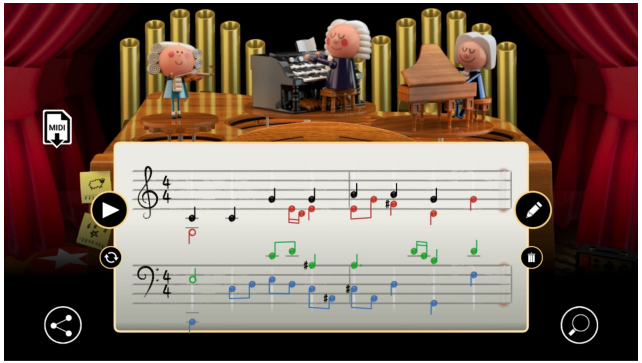
Hidden Markov Models

Once again, why mention these?

- Still pretty simple, but for once we have something that actually works pretty well (and it's from 2004!)
- The idea of a "hidden state" is a component of more complex models (RNNs, LSTMs, or other sequence models)

Example: Coconet & Bach Doodle (Huang et al., 2017)

- A recent chorale harmonization model using convolution networks (not HMM, but the underlying idea is the same)
- Coconet: https://magenta.tensorflow.org/coconet
- **Play it! https://doodles.google/doodle/celebrating-johann-sebastian-bach/**
- (we'll cover this one later in the module)



Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, "Counterpoint by convolution", *ISMIR,* 2017
"Coconet: the ML model behind today's Bach Doodle", *Magenta Blog*, March 20, 2019

Take-homes

- Get a sense of how music synthesis can be viewed as a “next token prediction” problem
- Understand the ideas of (a) modeling the probability of a sequence; (b) choosing sequences that have high probability; and (c) sampling from that distribution
- Understand why the above approach should generate “good” music (or at least music that is similar to a training dataset), if a model is sufficiently powerful
- Understand why Markov chains, as described, are mostly *not* capable of handling the complexities of music synthesis, and think about what components they are still missing

Symbolic music generation

3.3: Music tokenization

Representing music as language

There are two fundamental issues we need to deal with:

1. How should music be ***tokenized?***
2. What sorts of models can be used to ***generate*** token sequences

These two questions are fairly “entangled”: neither of these problems is quite “solved” for music representation, and lots of papers develop new tokenization schemes and architectures simultaneously (so it’s hard to talk about the two problems separately!)

I’ll talk about tokenization first (but could just as easily put this the other way around)

Representing music as language

How should *language* be tokenized?

~7-8 years ago:

the→quick→brown→fox→jumped→over→the … (word level) **vs**

t→h→e→_→q→u→i→c→k→_→b→r→o→w→ … (character level)

Both have their merits! (what are they?)

Representing music as language

More recently, the community has settled on encoding schemes that are “between” characters and words:

(from https://research.google/blog/a-fast-wordpiece-tokenization-system/)

Representing music as language

The idea that language should be tokenized in this way is reasonably “settled”

With music the situation is much more complex! Some issues:

- Events occur simultaneously and overlap
- Events have significant amounts of associated metadata (instrumentation, dynamics, phrasing, etc.)
- Events are associated with timing information (which can also be stored in various ways)
- For humans, reading music is *really, really hard!* (why?)
- others?
- (and these are just problems of *tokenization:* there are also problems in trying to model these data!)

Representing music as language

We'll sort of discuss representations in order from ones that are more "character-like" to ones that are more "word-like"

(though the analogy to "characters" and "words" maybe isn't quite accurate: think more just more "fine-grained" to more "coarse-grained")

Representing music as language

Several slides based on

“Generative AI for Music and Audio Creation” (Michigan):

https://hermandong.com/teaching/pat498_598_fall2024/

and some from

“Deep Learning for Music Analysis and Generation”:

https://github.com/affige/DeepMIR

Designing a machine-readable music language

How can we “represent” music in a way that machines understand?

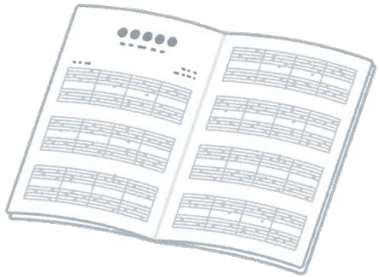
- Musical representation is a key component of a music generation system

Why not using sheet music “images” directly?

- Machines still have a hard time reading sheet music
- A challenging task known as “optical music recognition” (OMR)

Examples:

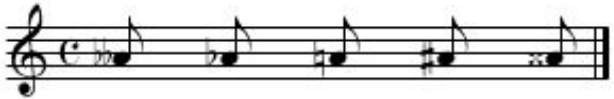
- ABC Notation
- MIDI



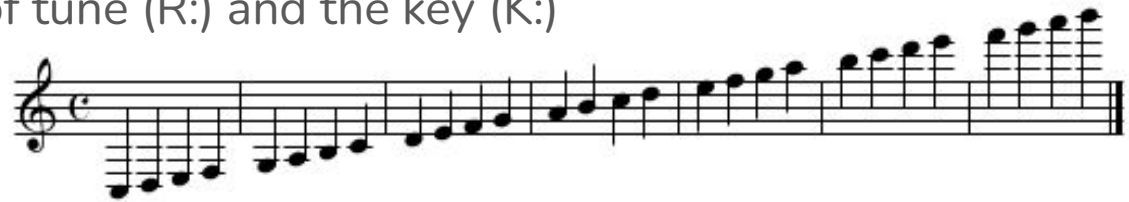
ABC Notation (already mentioned in Module 1)

ABC Notation (1997): https://abcnotation.com/

- A simple text-based notation
- Use letters to denote pitches
  - Lower octave (A–G), higher octave (a–g)
- Use prefix to denote accidentals
  - Sharp (^), flat (_), natural (=)
- a file (X:), the title (T:), the time signature (M:), the default note length (L:), the type of tune (R:) and the key (K:)



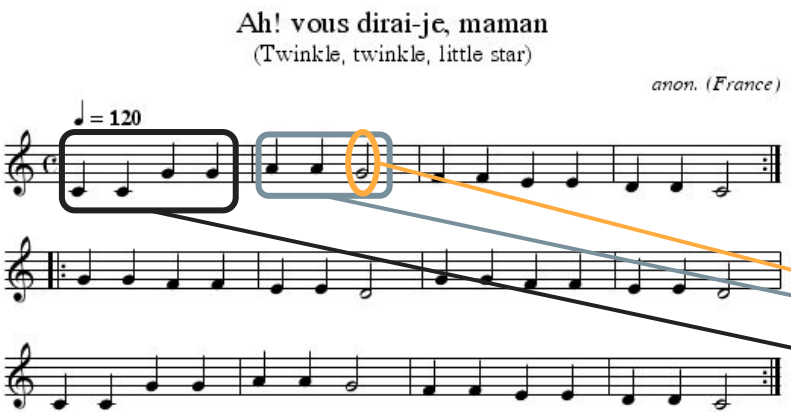
__A _A =A ^A ^^A



C, D, E, F, | G, A, B, C | D E F G | A B c d | e f g a | b c' d' e' | f' g' a' b'

abcnotation.com/examples

An example of ABC notation



Metadata

```
X:571
T:Ah! vous dirai-je, maman
T:(Twinkle, twinkle, little
star)
C:anon.
O:France
R:Nursery song
M:C  Meter
L:1/4  Unit note length (temporal resolution)
Q:120  Tempo
K:C  Key
CCGG AAG2|FFEE|DDC2:|
|:GGFF|EED2|GGFF|EED2|
CCGG|AAG2|FFEE|DDC2:|
```

abcnotation.com/tunePage?a=trillian.mit.edu/~jc/music/abc/mirror/musicaviva.com/france/ah-vous-c/ah-vous-c-1/0000

ChatGPT can also Compose Songs in ABC Notation!

You
Write a cheerful folk song using ABC notation

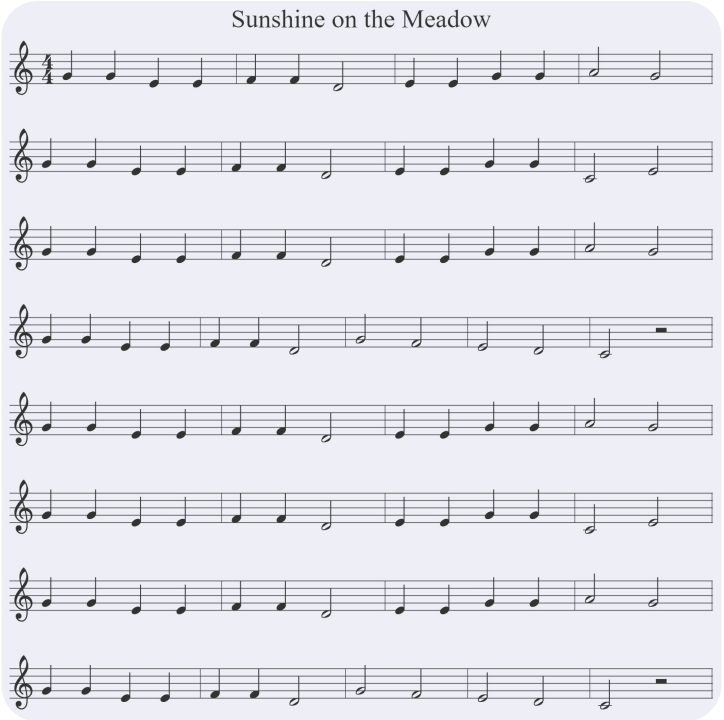
ChatGPT
Certainly! Here's a cheerful folk song in ABC notation:

vbnet — Copy code

```
X:1
T:Sunshine on the Meadow
M:4/4
K:C
G2 G2 E2 E2 | F2 F2 D4 | E2 E2 G2 G2 | A4 G4 |
G2 G2 E2 E2 | F2 F2 D4 | E2 E2 G2 G2 | C4 E4 |
G2 G2 E2 E2 | F2 F2 D4 | E2 E2 G2 G2 | A4 G4 |
G2 G2 E2 E2 | F2 F2 D4 | G4 F4 | E4 D4 | C4 z4 |
G2 G2 E2 E2 | F2 F2 D4 | E2 E2 G2 G2 | A4 G4 |
G2 G2 E2 E2 | F2 F2 D4 | E2 E2 G2 G2 | C4 E4 |
G2 G2 E2 E2 | F2 F2 D4 | E2 E2 G2 G2 | A4 G4 |
G2 G2 E2 E2 | F2 F2 D4 | G4 F4 | E4 D4 | C4 z4 |
```

Feel free to play around with the tempo and instrumentation to give it your own unique style!

chat.openai.com



sheet music rendered with abc.rectanglered.com

Limitations of ABC notation

“General-purpose” LMs can generate (decent enough) music in ABC notation, presumably given their exposure to training data and its relative simplicity

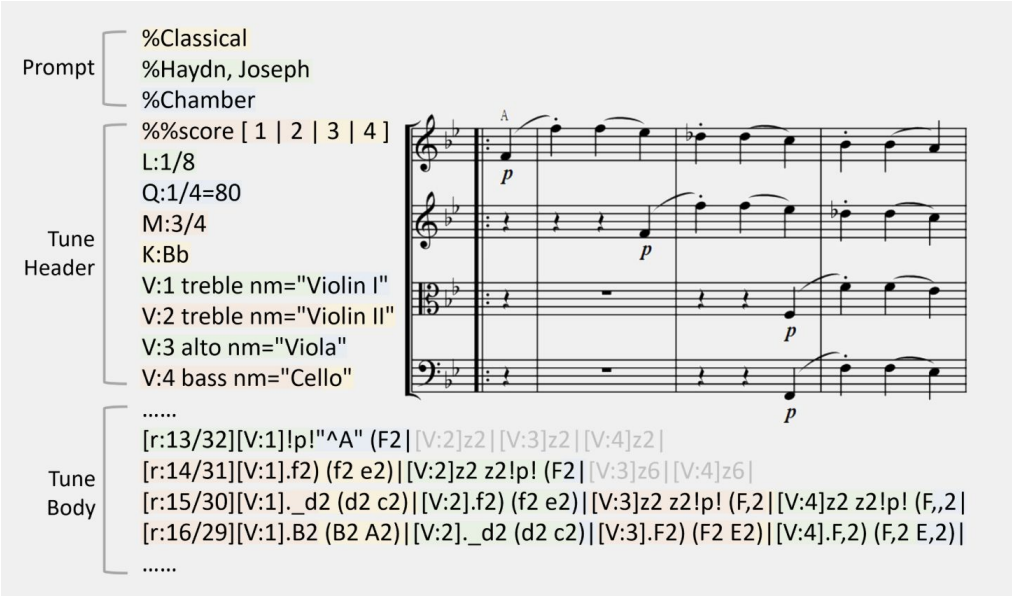
However it has some limitations:

- Limited expressiveness
- Monophonic tunes only

Idea: *how can ABC notation be adapted to be more general while maintaining its “comprehensibility”?*

Example: NotaGen (Wang et al., 2025)

- **Interleaved ABC notation**
  - represent multi-track music in ABC notation
  - different voices of the same bar are rearranged into a single line and differentiated using voice indicators "[V:]".
- Demo page: https://electricalexis.github.io/notagen-demo/
- **Note:** state-of-the-art generation results (at the time of writing), though the main insight is (probably) about tokenization!

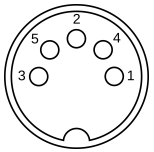


Yashan Wang, Shangda Wu, Jianhuai Hu, Xingjian Du, Yueqi Peng, Yongxin Huang, Shuai Fan, Xiaobing Li, Feng Yu and Maosong Sun. "NotaGen: Advancing Musicality in Symbolic Music Generation with Large Language Model Training Paradigms." *CoRR, vol. abs/2502.18008*, 2025

Shangda Wu, Dingyao Yu, Xu Tan, and Maosong Sun. "Clamp: Contrastive language-music pretraining for cross-modal symbolic music information retrieval." *ISMIR*, 2023

MIDI (Musical Instrument Digital Interface)

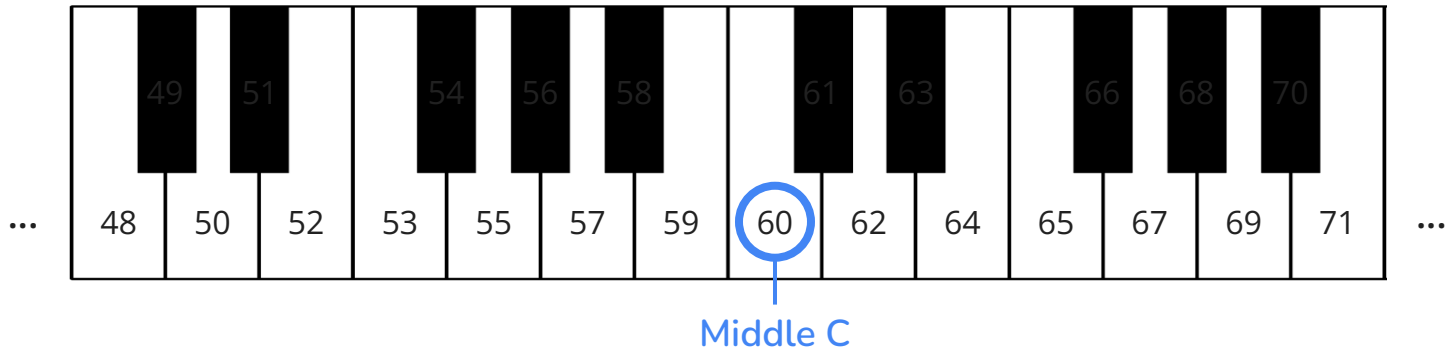
- A communication protocol between devices
- MIDI Messages
  - Note on
  - Note off
  - Delta time
  - Program change
  - Control change
  - Pitch bend change



MIDI I/O

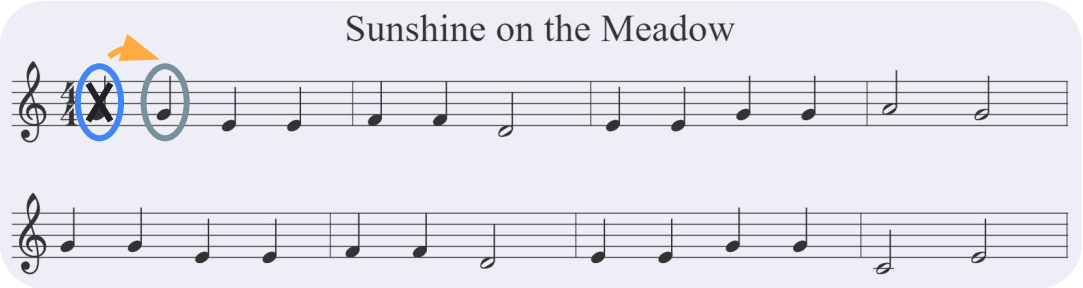
MIDI note numbers

- Ranging from 0 to 127
  - Middle C is 60
  - Wider than standard piano's pitch range
- Widely used in software, keyboards, controllers, and algorithms



Representing music using MIDI messages

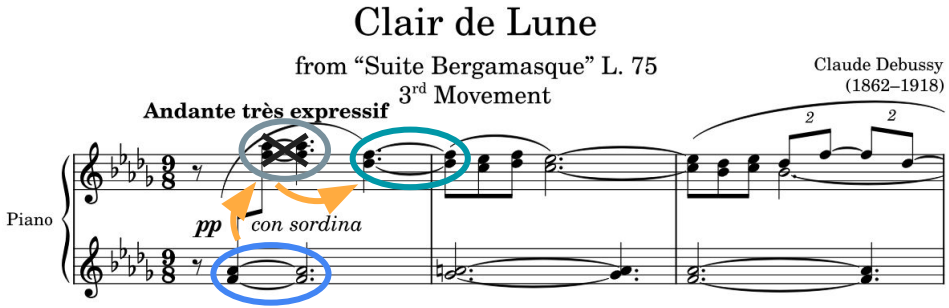
- Represent a music piece as a token sequence
- Three main MIDI messages
  - Note on
  - Note off
  - Time Shift




```
Note_on_67, Time_shift_quarter_note, Note_off_67,
Note_on_67, Time_shift_quarter_note, Note_off_67,
Note_on_64, Time_shift_quarter_note, Note_off_64,
Note_on_64, Time_shift_quarter_note, Note_off_64,
...
```

Representing polyphonic music

- Can natively handle music with multiple pitches at the same time
  - In the literature, “polyphonic” & “multi-pitch” are often used interchangeably

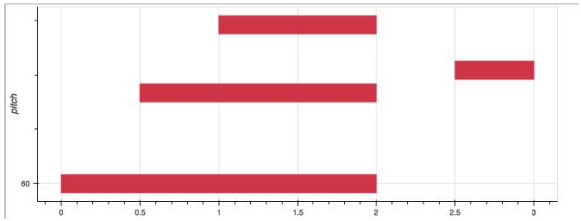



```
Note_on_65, Note_on_68, Time_shift_eighth_note, Note_on_77, Note_on_80,
Time_shift_half_note, Note_off_77, Note_off_80 Note_on_73, Note_on_77,
Time_shift_dotted_quarter_note, Note_off_65, Note_off_68, ...
```

Example: Performance RNN (Simon & Oore, 2017)

- Data
  - Yamaha e-Piano Competition dataset (MAESTRO)
- **MIDI-like Representation**
  - 128 **Note-On** events + 128 **Note-Off** events: start or release a MIDI note
  - 100 **Time-Shift** events (10ms–1s)
  - 32 **Set-Velocity** events
- Demo page: https://magenta.tensorflow.org/performance-rnn

dynamics




```
SET_VELOCITY<80>, NOTE_ON<60>
TIME_SHIFT<500>, NOTE_ON<64>
TIME_SHIFT<500>, NOTE_ON<67>
TIME_SHIFT<1000>, NOTE_OFF<60>, NOTE_OFF<64>,
NOTE_OFF<67>
TIME_SHIFT<500>, SET_VELOCITY<100>, NOTE_ON<65>
TIME_SHIFT<500>, NOTE_OFF<65>
```


Ian Simon and Sageev Oore, "Performance RNN: Generating Music with Expressive Timing and Dynamics," *Magenta Blog*, June 29, 2017

Sageev Oore, Ian Simon, Sander Dieleman, Douglas Eck, and Karen Simonyan, "This Time with Feeling: Learning Expressive Musical Performance", *Neural Computing and Applications*, 32, 2020

Limitations of MIDI-like representations

- High-level (*semantic*) information, such as **downbeat, tempo, and chords**, are not included in these MIDI-like representations*
- Humans tend to organize regularly recurring patterns and accents over a metrical structure defined in terms of **sub-beats, beats, and bars**; in MIDI-like, this information needs to be learned by the model implicitly

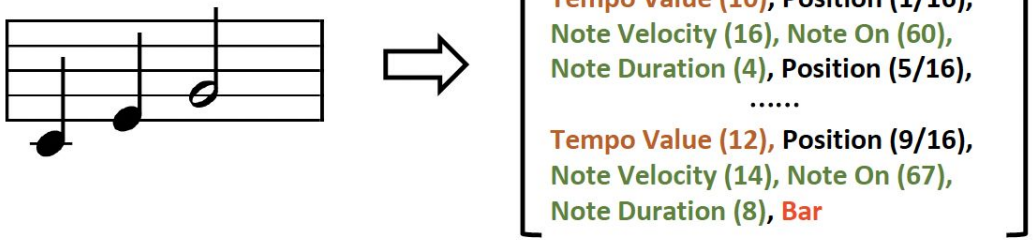
Idea: *how can we include these types of information during tokenization?*

*note that ABC-like does include bar information, but is less expressive in other ways!

Yu-Siang Huang, Yi-Hsuan Yang. "Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions", *ACM Multimedia,* 2020

Example: REMI (Huang & Yang, 2020)

- Represent MIDI data following the way humans read them
  - Note-Off —> **Note Duration**
  - Time-Shift —> **Bar & Position**
    - **Bar**: mark the bar lines; **Position**: indicate the position of a note within a bar
    - Provide structured timing information explicitly, in a beat-based manner
  - **Tempo & Chord**: supportive musical tokens to capture higher-level information

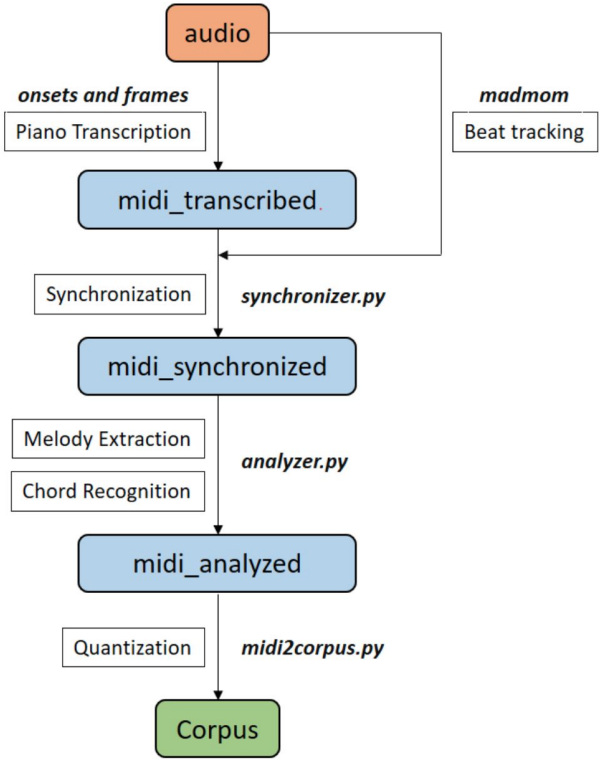


Yu-Siang Huang, Yi-Hsuan Yang. "Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions", *ACM Multimedia*, 2020

Example: REMI (Huang & Yang, 2020)

How to get bar/beat information?

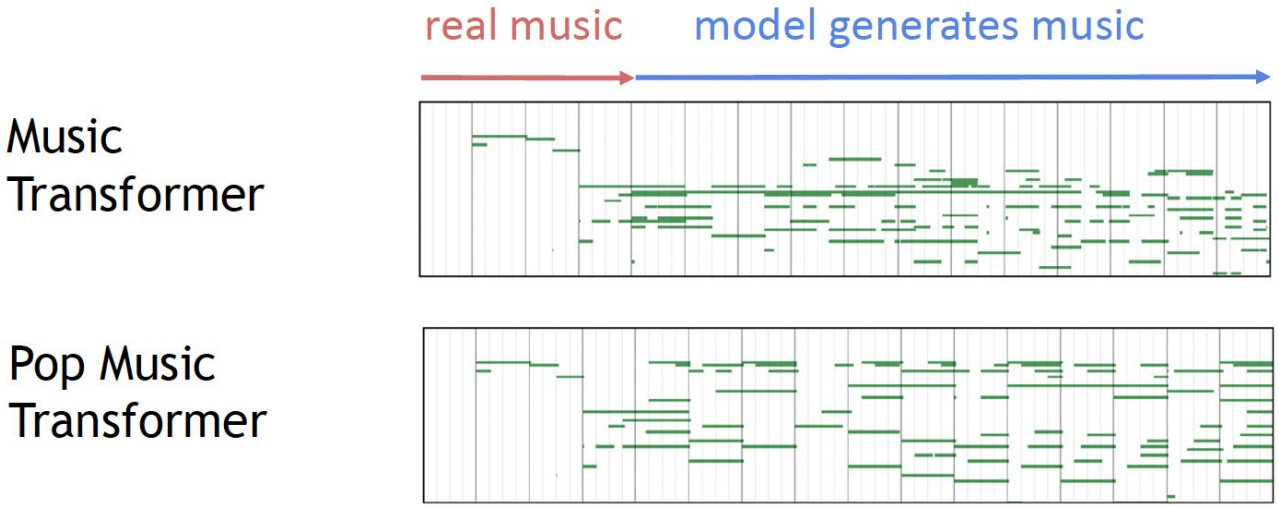
- MIDI files
  - already there
- Transcribed performances
  - beat/downbeat tracking



https://github.com/YatingMusic/compound-word-transformer/blob/main/dataset/Dataset.md

Example: REMI (Huang & Yang, 2020)

Outperforms MIDI-like representation for generating pop piano performances



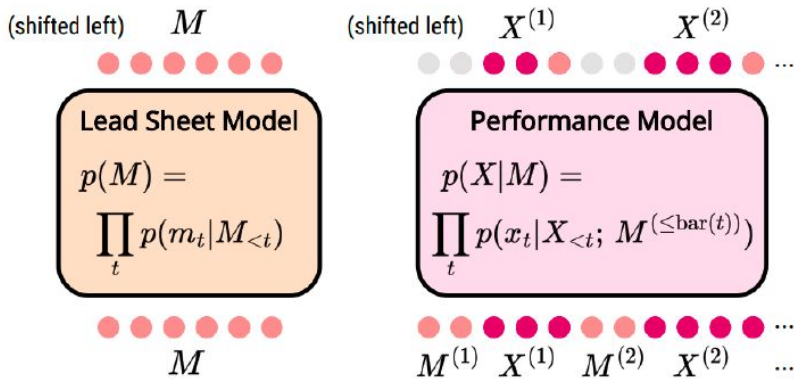
audio demo: https://drive.google.com/open?id=1LzPBjHPip4S0CBOLquk5CNapvXSfys54

Yu-Siang Huang, Yi-Hsuan Yang. "Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions", *ACM Multimedia*, 2020

Example: Conditional music generation

- Generate piano performances for lead sheets in a single model
  - lead sheet = melody & chord
- **How?** an interleaved sequence of lead sheets M and piano performances X

[bar], [track=lead sheet], (content of the lead sheet for a bar), [track=piano], (content of the piano for the same bar), [bar], ...



Wen-Yi Hsiao, Jen-Yu Liu, Yin-Cheng Yeh, Yi-Hsuan Yang. "Compound Word Transformer: Learning to Compose Full-Song Music over Dynamic Directed Hypergraphs", *AAAI* 2021
Shih-Lun Wu, Yi-Hsuan Yang. "Compose & Embellish: Well-Structured Piano Performance Generation via A Two-Stage Approach", *ICASSP* 2023

More limitations...

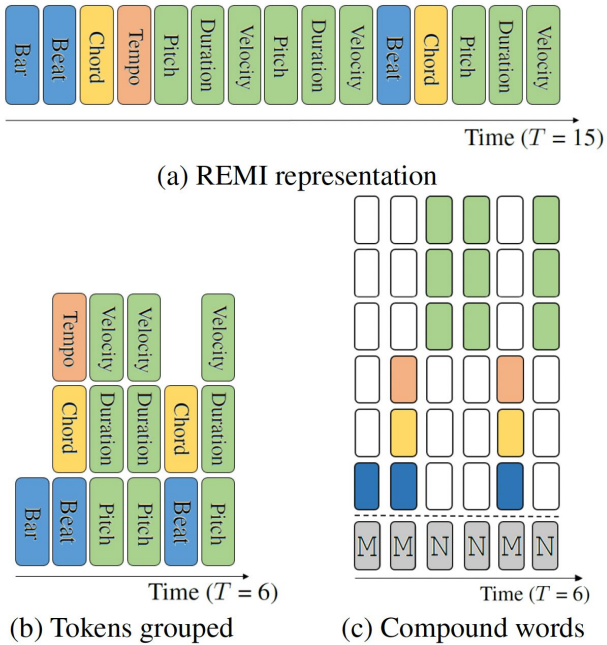
- Both MIDI-like and REMI use multiple tokens to represent a musical note
  - A piano cover of a pop song can contain up to 5k tokens
- As such, it is hard for the model to learn **long-term patterns**
- (and long term patterns are very important in music!)

Idea: *how can we develop "denser" (fewer token) representations, i,e., by representing more information per-token?*

Wen-Yi Hsiao, Jen-Yu Liu, Yin-Cheng Yeh, Yi-Hsuan Yang. "Compound Word Transformer: Learning to Compose Full-Song Music over Dynamic Directed Hypergraphs", *AAAI* 2021

Example: Compound Word (Hsiao et al., 2021)

- Group tokens into "**Compound Words**"
  - the vocabulary of music involves tokens of various token types (e.g., note-related, metric-related)
  - e.g. pitch + duration + velocity
- Predict multiple tokens of various types at once in a single time step
  - Shorter training time & inference time
- Demo page: https://ailabs.tw/human-interaction/compound-word-transformer-generate-pop-piano-music-of-full-song-length/



Representing multiple instruments

- Using **MIDI program change** messages
  - Program numbers: 1–128 (or 0–127)
  - 128 instruments in 16 families

Prog#	INSTRUMENT
	1-8 PIANO
1	Acoustic Grand
2	Bright Acoustic
3	Electric Grand
4	Honky-Tonk
5	Electric Piano 1
6	Electric Piano 2
7	Harpsichord
8	Clav

Prog#	INSTRUMENT	Prog#	INSTRUMENT
	1-8 PIANO		**9-16 CHROMATIC PERCUSSION**
1	Acoustic Grand	9	Celesta
2	Bright Acoustic	10	Glockenspiel
3	Electric Grand	11	Music Box
4	Honky-Tonk	12	Vibraphone
5	Electric Piano 1	13	Marimba
6	Electric Piano 2	14	Xylophone
7	Harpsichord	15	Tubular Bells
8	Clav	16	Dulcimer
	17-24 ORGAN		**25-32 GUITAR**
17	Drawbar Organ	25	Acoustic Guitar(nylon)
18	Percussive Organ	26	Acoustic Guitar(steel)
19	Rock Organ	27	Electric Guitar(jazz)
20	Church Organ	28	Electric Guitar(clean)
21	Reed Organ	29	Electric Guitar(muted)
22	Accoridan	30	Overdriven Guitar
23	Harmonica	31	Distortion Guitar
24	Tango Accordian	32	Guitar Harmonics
	33-40 BASS		**41-48 STRINGS**
33	Acoustic Bass	41	Violin
34	Electric Bass(finger)	42	Viola
35	Electric Bass(pick)	43	Cello
36	Fretless Bass	44	Contrabass
37	Slap Bass 1	45	Tremolo Strings
38	Slap Bass 2	46	Pizzicato Strings
39	Synth Bass 1	47	Orchestral Strings
40	Synth Bass 2	48	Timpani
	49-56 ENSEMBLE		**57-64 BRASS**
49	String Ensemble 1	57	Trumpet
50	String Ensemble 2	58	Trombone
51	SynthStrings 1	59	Tuba
52	SynthStrings 2	60	Muted Trumpet
53	Choir Aahs	61	French Horn
54	Voice Oohs	62	Brass Section
55	Synth Voice	63	SynthBrass 1
56	Orchestra Hit	64	SynthBrass 2

	65-72 REED		**73-80 PIPE**
65	Soprano Sax	73	Piccolo
66	Alto Sax	74	Flute
67	Tenor Sax	75	Recorder
68	Baritone Sax	76	Pan Flute
69	Oboe	77	Blown Bottle
70	English Horn	78	Shakuhachi
71	Bassoon	79	Whistle
72	Clarinet	80	Ocarina
	81-88 SYNTH LEAD		**89-96 SYNTH PAD**
81	Lead 1 (square)	89	Pad 1 (new age)
82	Lead 2 (sawtooth)	90	Pad 2 (warm)
83	Lead 3 (calliope)	91	Pad 3 (polysynth)
84	Lead 4 (chiff)	92	Pad 4 (choir)
85	Lead 5 (charang)	93	Pad 5 (bowed)
86	Lead 6 (voice)	94	Pad 6 (metallic)
87	Lead 7 (fifths)	95	Pad 7 (halo)
88	Lead 8 (bass+lead)	96	Pad 8 (sweep)
	97-104 SYNTH EFFECTS		**105-112 ETHNIC**
97	FX 1 (rain)	105	Sitar
98	FX 2 (soundtrack)	106	Banjo
99	FX 3 (crystal)	107	Shamisen
100	FX 4 (atmosphere)	108	Koto
101	FX 5 (brightness)	109	Kalimba
102	FX 6 (goblins)	110	Bagpipe
103	FX 7 (echoes)	111	Fiddle
104	FX 8 (sci-fi)	112	Shanai
	113-120 PERCUSSIVE		**121-128 SOUND EFFECTS**
113	Tinkle Bell	121	Guitar Fret Noise
114	Agogo	122	Breath Noise
115	Steel Drums	123	Seashore
116	Woodblock	124	Bird Tweet
117	Taiko Drum	125	Telephone Ring
118	Melodic Tom	126	Helicopter
119	Synth Drum	127	Applause
120	Reverse Cymbal	128	Gunshot

(source: Roger Dannenberg)

www.cs.cmu.edu/~music/cmsip/readings/GMSpecs_Patches.htm

Example: Multitrack Music Machine (Ens & Pasquier, 2020)

MULTI-TRACK

- A sequence of tracks beginning with PIECE_START
- All tracks sound simultaneously

TRACK

- TRACK_START, <TRACK>, TRACK_END
- <TRACK>: a sequence of bars
- **INST**: specify the MIDI program
- **DENSITY**: note density, for controllability

BAR

- BAR_START, <BAR>, BAR_END
- <BAR>: MIDI-like representation

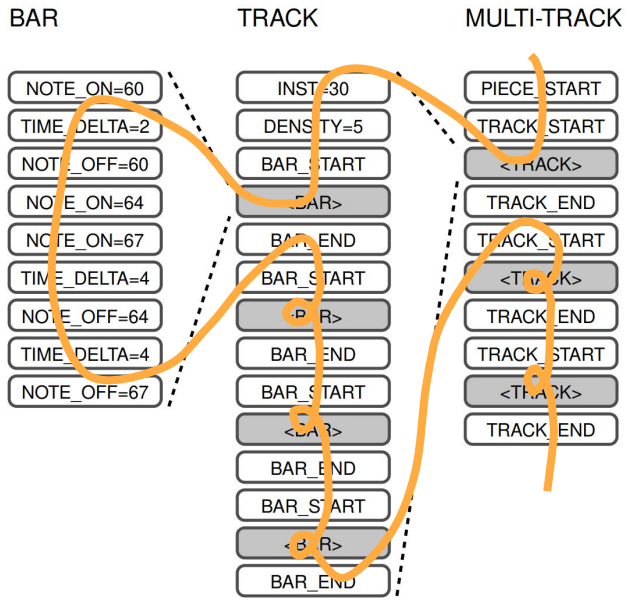


Fig. 1. The MultiTrack and BarFill representations are shown. The `<bar>` tokens correspond to complete bars, and the `<track>` tokens correspond to complete tracks.

(Ens & Pasquier, 2020)

Jeff Ens and Philippe Pasquier, "MMM : Exploring Conditional Multi-Track Music Generation with the Transformer," *arXiv preprint arXiv:2008:06048*, 2020

Example: Multitrack Music Machine (Ens & Pasquier, 2020)

BAR-FILL

–> support bar-level inpainting

Inpainting?

- filling in missing regions of a piece of music

How?

- Replace each bar to be predicted with a placeholder token “FILL_IN”
- Insert contents of these bars at the end of the piece

(Ens & Pasquier, 2020)

Interactive demo

youtu.be/NdeMZ3y-84Q

Jeff Ens and Philippe Pasquier, “MMM : Exploring Conditional Multi-Track Music Generation with the Transformer,” *arXiv preprint arXiv:2008:06048*, 2020

Example: Multitrack Music Transformer (Dong et al., 2023)

Representation based on Compound Word

- "(beat, position, pitch, duration, instrument)"
- support instrument-informed generation
- Demo page: https://hermandong.com/mmt/

(0, 0, 0, 0, 0, 0)	Start of song
(1, 0, 0, 0, 0, 15)	Instrument: accordion
(1, 0, 0, 0, 0, 36)	Instrument: trombone
(1, 0, 0, 0, 0, 39)	Instrument: brasses
(2, 0, 0, 0, 0, 0)	Start of notes
(3, 1, 1, 41, 15, 36)	Note: beat=1, position=1, pitch=E2, duration=48, instrument=trombone
(3, 1, 1, 65, 4, 39)	Note: beat=1, position=1, pitch=E4, duration=12, instrument=brasses
(3, 1, 1, 65, 17, 15)	Note: beat=1, position=1, pitch=E4, duration=72, instrument=accordion
(3, 1, 1, 68, 4, 39)	Note: beat=1, position=1, pitch=G4, duration=12, instrument=brasses
(3, 1, 1, 68, 17, 15)	Note: beat=1, position=1, pitch=G4, duration=72, instrument=accordion
(3, 1, 1, 73, 17, 15)	Note: beat=1, position=1, pitch=C5, duration=72, instrument=accordion
(3, 1, 13, 68, 4, 39)	Note: beat=1, position=13, pitch=G4, duration=12, instrument=brasses
(3, 1, 13, 73, 4, 39)	Note: beat=1, position=13, pitch=C5, duration=12, instrument=brasses
(3, 2, 1, 73, 12, 39)	Note: beat=2, position=1, pitch=C5, duration=36, instrument=brasses
(3, 2, 1, 77, 12, 39)	Note: beat=2, position=1, pitch=E5, duration=36, instrument=brasses
...	...
(4, 0, 0, 0, 0, 0)	End of song

(source: Dong et al., 2023)

Example of generated music

(**ensemble**: contrabass, harp, english-horn, flute)

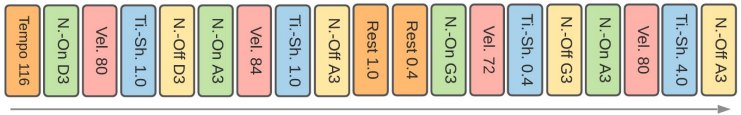
Hao-Wen Dong, Ke Chen, Shlomo Dubnov, Julian McAuley, and Taylor Berg-Kirkpatrick, "Multitrack Music Transformer," *ICASSP*, 2023

Library: MidiTok

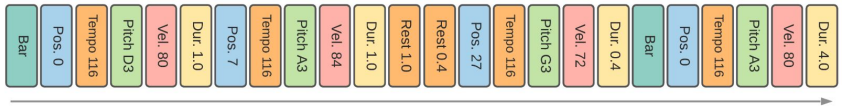
Supports most of tokenization methods we discussed above, and more variants

- REMI
- REMI+
- MIDI-Like
- TSD
- Structured
- CPWord
- Octuple
- MuMIDI
- MMM
- PerTok

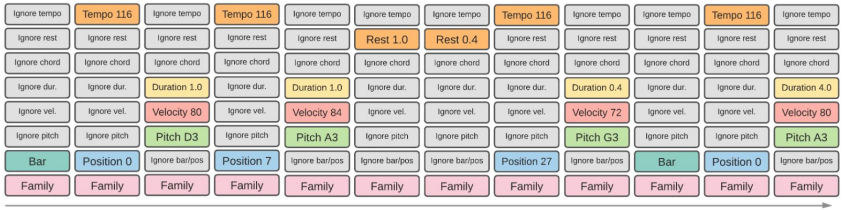
MIDI-Like



REMI



CPWord



MIDITok

github.com/Natooz/MidiTok

https://github.com/Natooz/MidiTok
https://miditok.readthedocs.io/en/latest/index.html

Library: MidiTok

Load required libraries

```
from miditok import REMI, TokenizerConfig
from symusic import Score
```

Train your MIDI tokenizer

```
config = TokenizerConfig(num_velocities= 1, use_chords=False, use_programs=True)
tokenizer = REMI(config)
tokenizer.train(vocab_size= 1000, files_paths=train_files)
tokenizer.save("tokenizer.json")
```

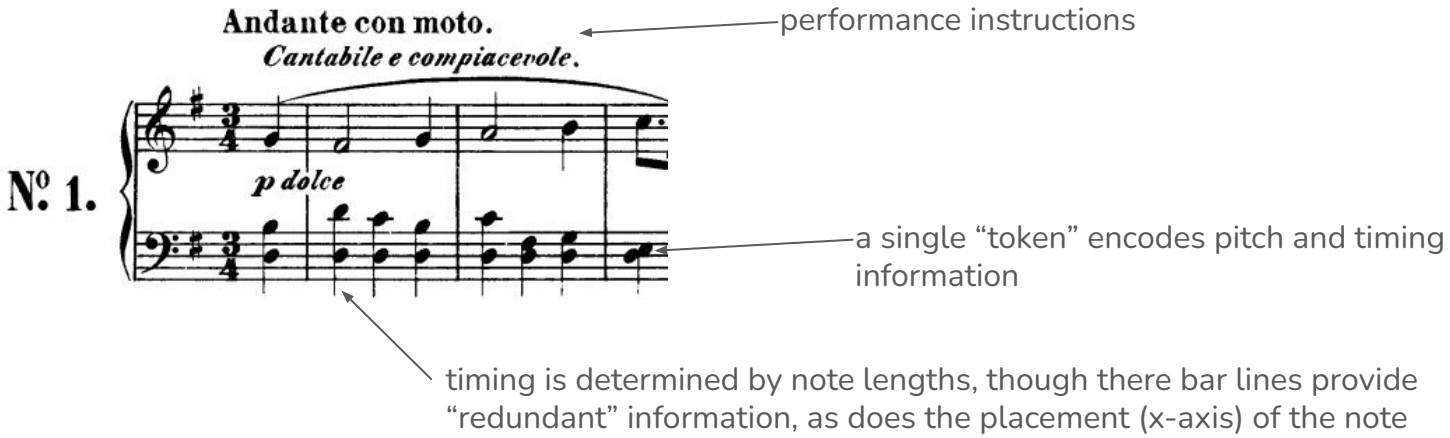
(see workbook3.ipynb)

Construct a PyTorch Dataset

```
class MIDIDataset(Dataset):
    def __init__(self, file_paths: List[str], tokenizer):
        self.tokenizer = tokenizer
        self.file_paths = file_paths
    def __len__(self):
        return len(self.file_paths)
    def __getitem__(self, idx):
        midi = Score(self.file_paths[idx])
        tokens = self.tokenizer(midi)
        return np.array(tokens)
```

Final thoughts

- Even though music is sequential, tokenization schemes don’t borrow all that much from language models
- Also draws surprisingly little inspiration from how humans read music, which is much more “2d” (though we’ll come back to piano rolls later)



Take-homes

- Understand how language tokenization and music tokenization are similar, but different
- Understand why tokenization is quite so difficult to get right for music
- We are probably still in the “Cambrian explosion” of ideas for how to tokenize music, and identifying the “best” approach is by no means a settled problem

Symbolic music generation

3.4: Introduction to language modeling

Language models for symbolic music generation

We've seen some options for tokenization in the last section

We've also seen a very simple approach for next-token prediction (Markov models)

- In this section we'll give a *high level* view of how "large" language models do tokenization
- (and in the next section, we'll see some specific adaptations to music)

Language models for symbolic music generation

Several slides based on

“Connecting Music Audio and Natural Language”:
https://mulab-mir.github.io/music-language-tutorial/intro.html#

and others from

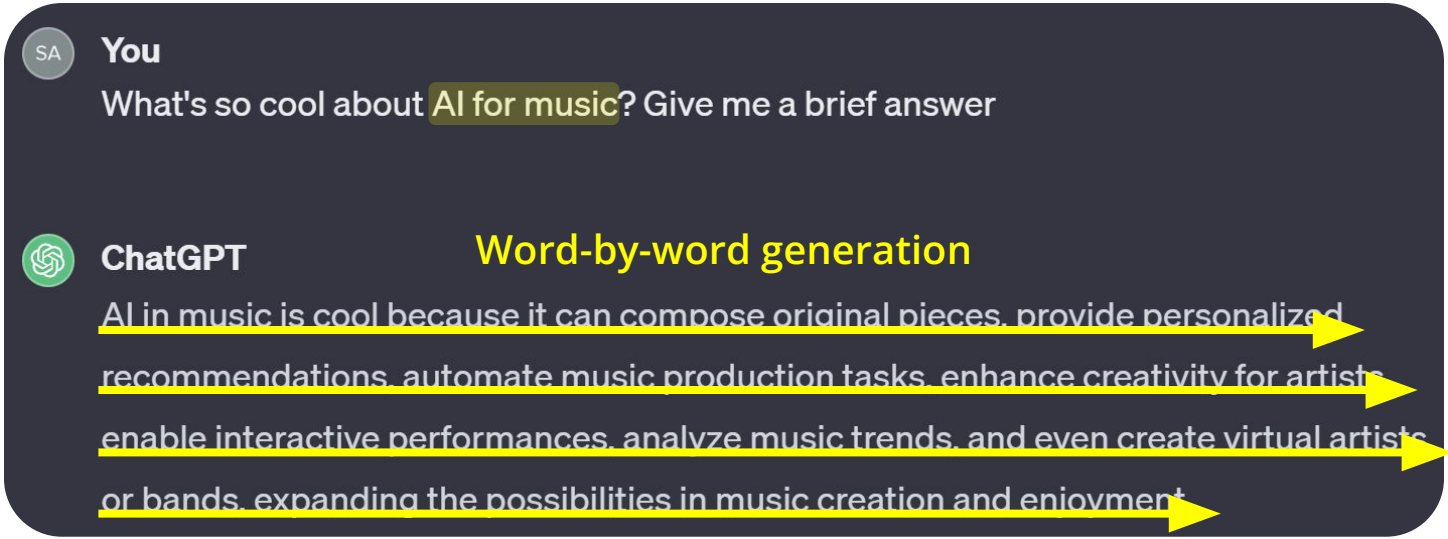
“Generative AI for Music and Audio Creation” (Michigan):
https://hermandong.com/teaching/pat498_598_fall2024/

“Deep Learning for Music Analysis and Generation”:
https://github.com/affige/DeepMIR

“**Understanding LSTM Networks**”:
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

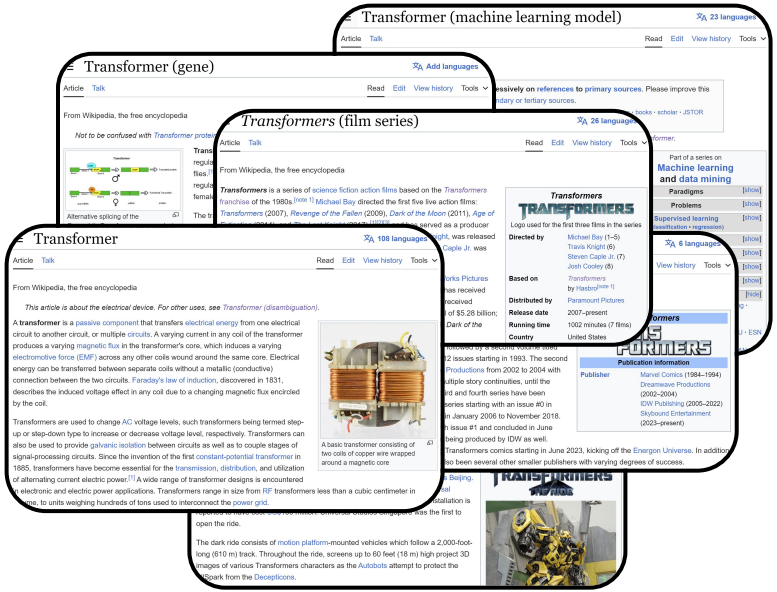
Large Language Models (LLMs)

- The models behind ChatGPT (or whatever people use these days)!

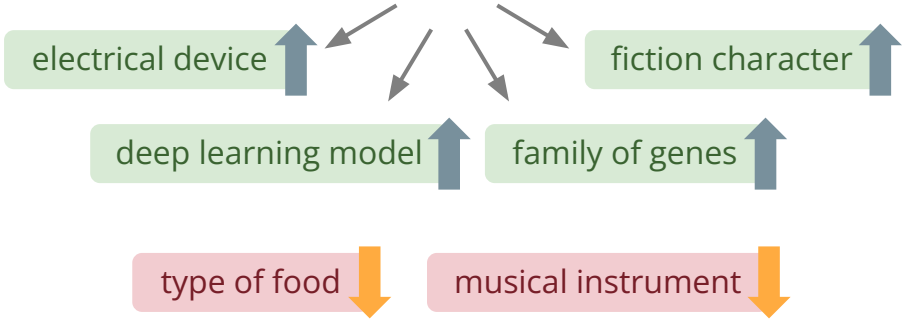


Language models

- Predicting the next word given the **past sequence of words**
- **Recall:** a (first-order) Markov model only depends on **one previous word**

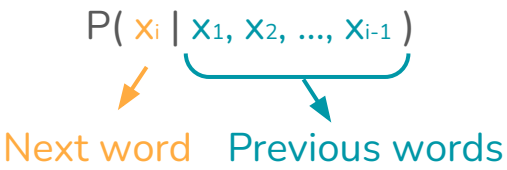


A transformer is a _____



Language models (Mathematically)

- A class of machine learning models that learn the next word probability



P (electrical | A transformer is a)

P (character | A transformer is a)

P (gene | A transformer is a)

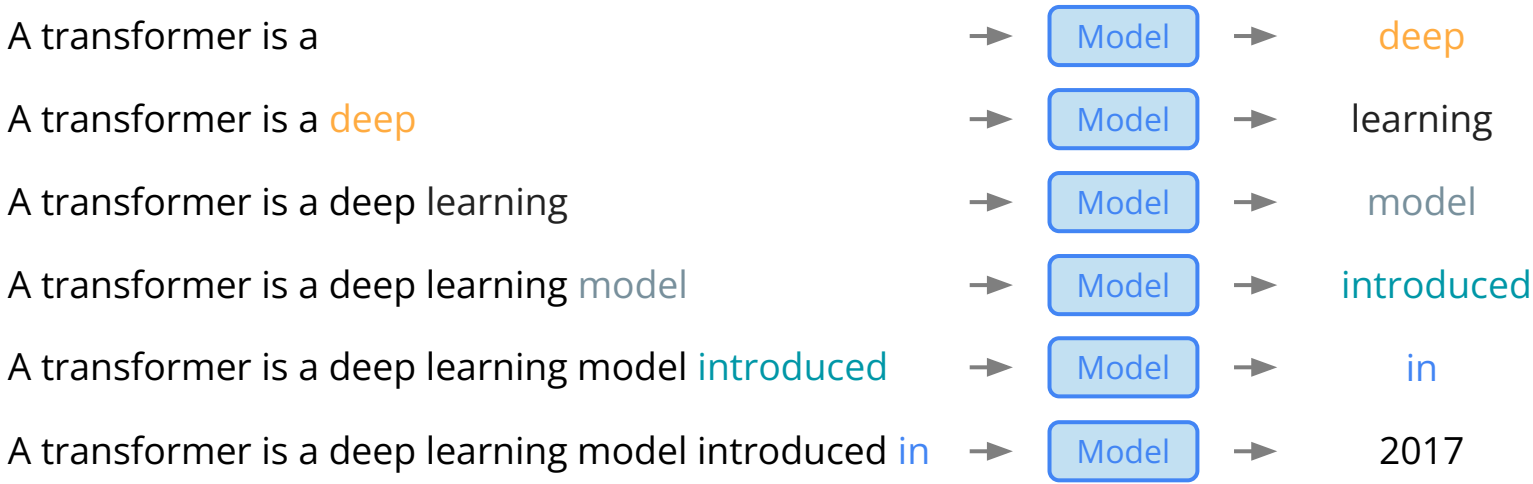
P (model | A transformer is a)

P (food | A transformer is a)

P (musical | A transformer is a)

Language models: generation

- How do we generate a new sentence using a trained language model?



Language models

Probability distribution defined over natural languages:

$$P(\text{some text})$$

from: https://mulab-mir.github.io/music-language-tutorial/intro.html# (Seung Heon Doh, Ilaria Manco, Zachary Novack, Jong Wook Kim, Ke Chen)

Language models

Probability distribution defined over natural languages:

$$P(\text{some text} \mid \text{condition})$$

Language models

Probability distribution defined over natural languages:

P (some text | condition)

“output” “input”

from Jong Wook Kim

Language models

Probability distribution defined over natural languages:

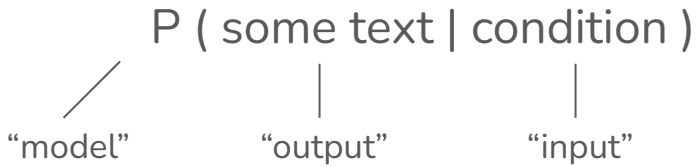
$$P(\text{some text} \mid \text{condition})$$

“output”	“input”	
answer	question	sequence-to-sequence models (e.g. T5)
substring	surroundings	masked language models (e.g. BERT)
continuation	prefix	autoregressive language models (e.g. GPT)
chat response	chat history	conversational AI (e.g. ChatGPT)
caption	music	music captioning model

from Jong Wook Kim

Language models

Probability distribution defined over natural languages:



from Jong Wook Kim

Language models

Probability distribution defined over natural languages:

P (some text | condition)

“model”

n-grams

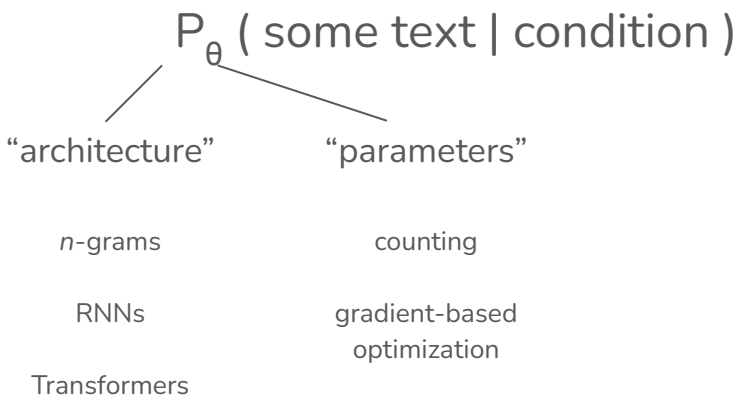
RNNs

Transformers

from Jong Wook Kim

Language models

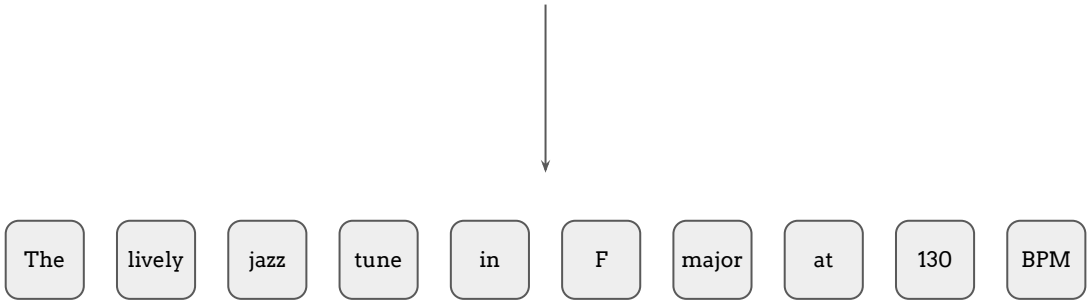
Probability distribution defined over natural languages:



from Jong Wook Kim

Representation: text as a sequence of tokens

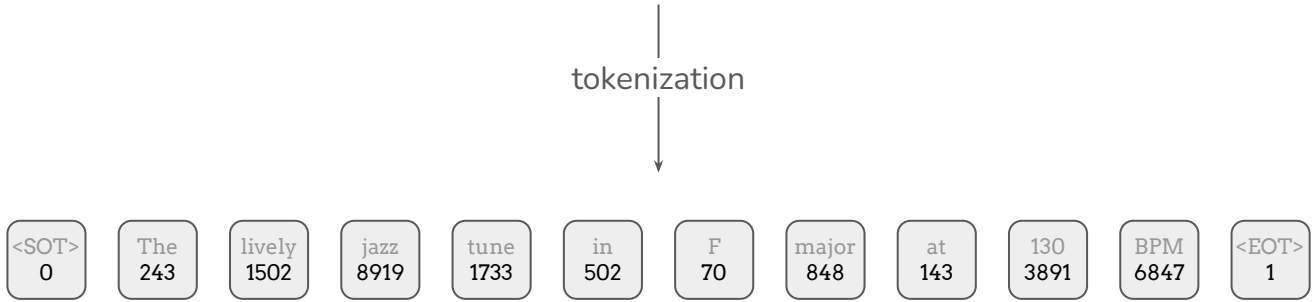
P_θ (output text | condition)



from Jong Wook Kim

Representation: text as a sequence of tokens

P_θ (output text | condition)



→ a sequence of tokens, really a list of token IDs

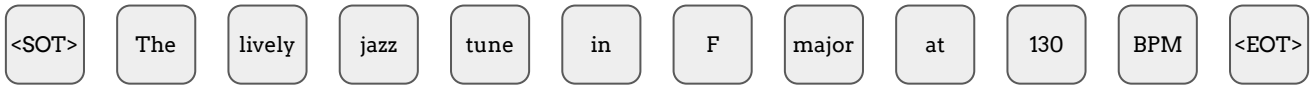
→ break into subword tokens to get fixed vocabulary size (e.g. “unforgettable”)

→ special tokens to mark text boundaries

from Jong Wook Kim

Implementing language models

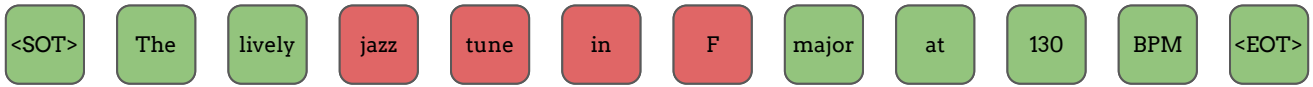
$$P_\theta\ (\ \text{output text}\ |\ \text{condition}\ )$$



from Jong Wook Kim

Implementation: Masked Language Models

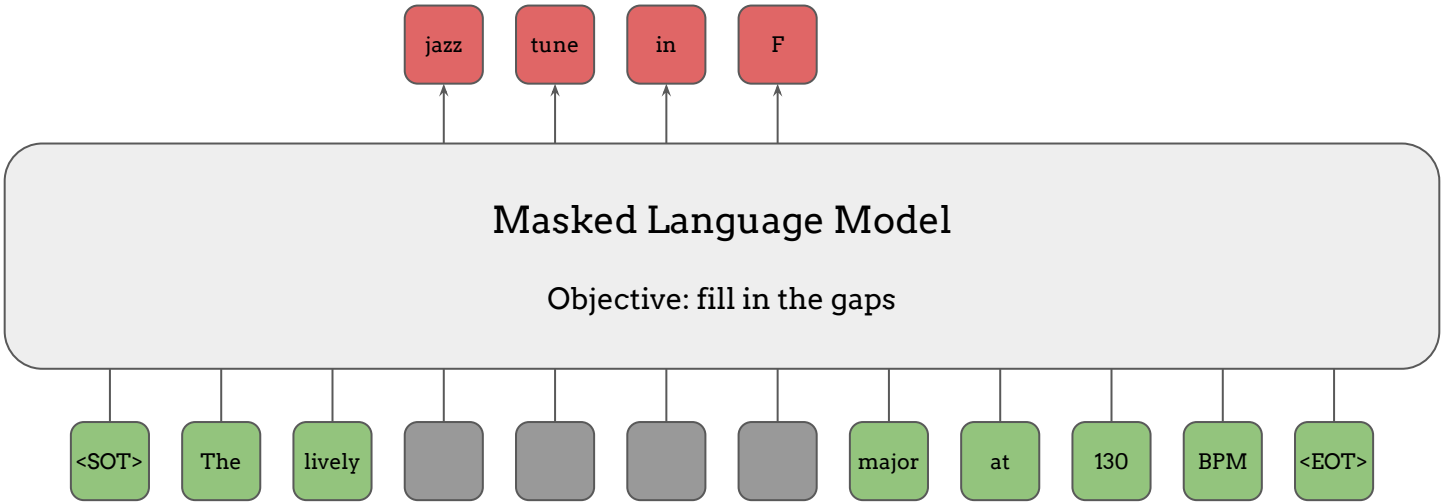
$$P_\theta \left(\text{output text} \mid \text{condition} \right)$$



from Jong Wook Kim

Implementation: Masked Language Models

P_θ (output text | condition)

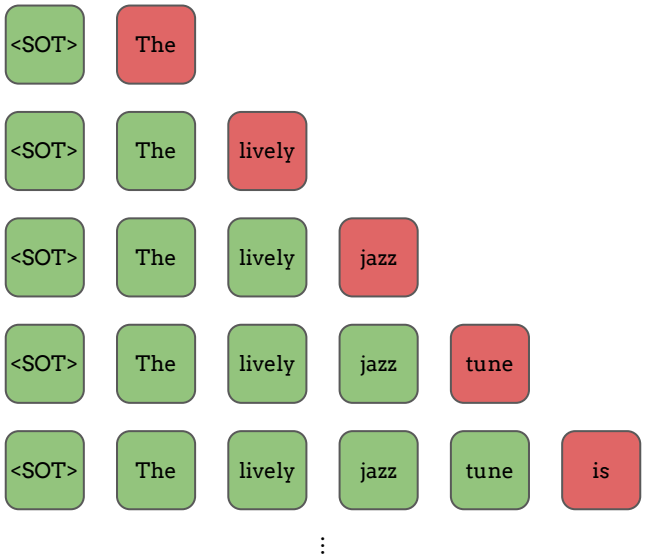


from Jong Wook Kim

Devlin, J. et al. "BERT: Pre-training of deep bidirectional transformers for language understanding". *arXiv preprint arXiv:1810.04805*, 2018
Liu, Y. et al. "RoBERTa: A robustly optimized bert pretraining approach". *arXiv preprint arXiv:1907.11692*, *364*, 2019

Implementation: Autoregressive Language Models

P_θ (output text | condition)

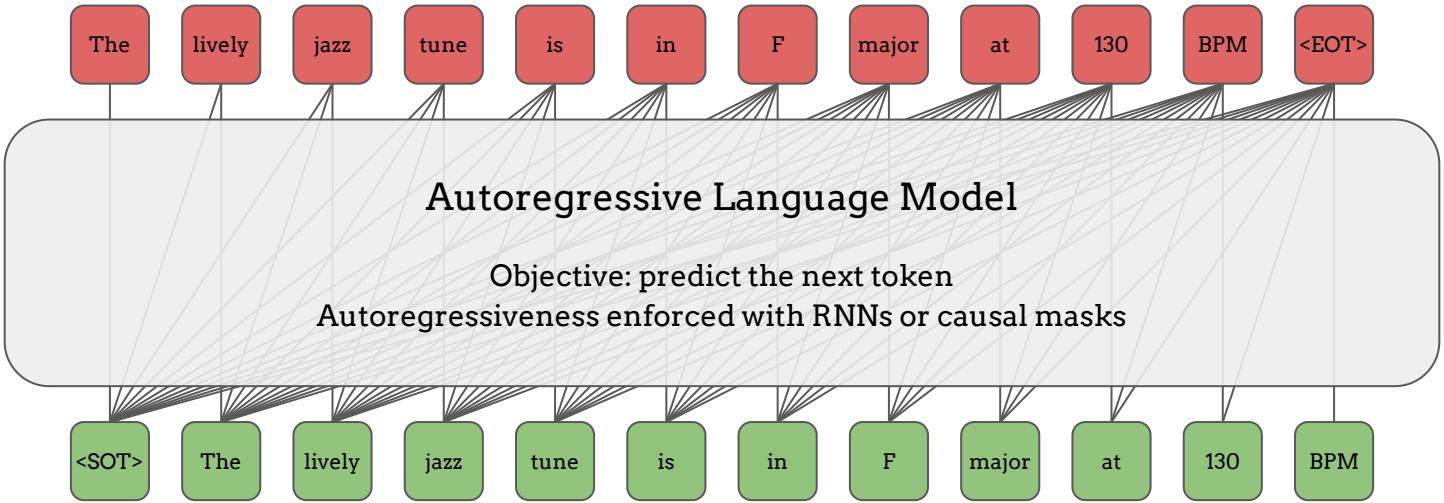


from Jong Wook Kim

Howard, J., & Ruder, S. "Universal language model fine-tuning for text classification". *arXiv preprint arXiv:1801.06146*, 2018
Radford, A. et al. "Improving language understanding by generative pre-training", 2018

Implementation: Autoregressive Language Models

P_θ (output text | condition)



from Jong Wook Kim

Howard, J., & Ruder, S. "Universal language model fine-tuning for text classification". *arXiv preprint arXiv:1801.06146*, 2018
Radford, A. et al. "Improving language understanding by generative pre-training", 2018

Implementation: Conditioning

P_θ (output text | condition)

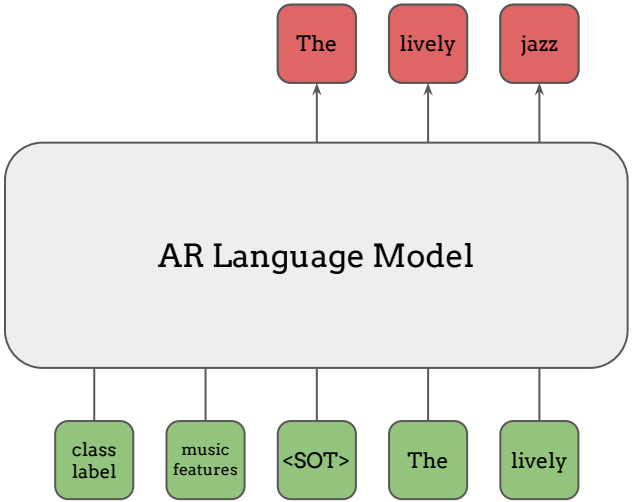
Doesn’t have to be a discrete sequence of tokens!

Doesn’t have to make a probability distribution!

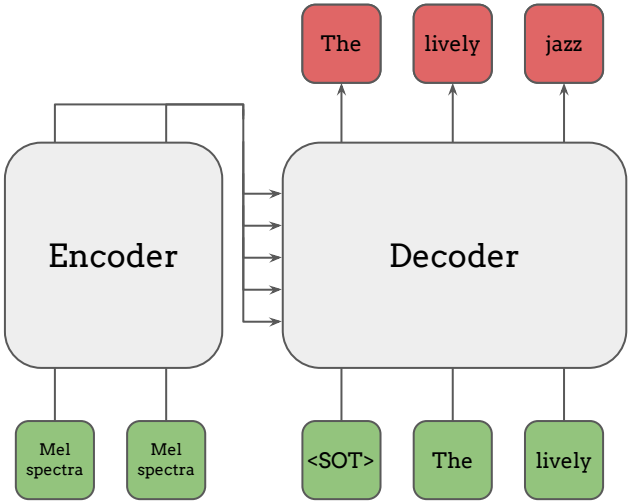
from Jong Wook Kim

Implementation: Conditioning

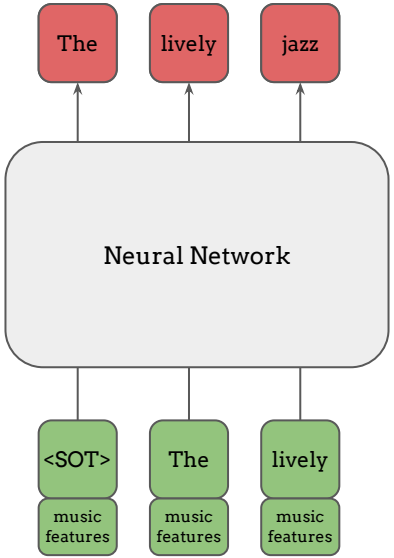
Prefix Conditioning



Encoder-Decoder Attention
(a.k.a. Cross Attention)



Channel Concatenation



from Jong Wook Kim

Language models as a framework

Language models provide a simple, general-purpose framework ML problems whose outputs can be represented as a sequence of discrete tokens

$$P_\theta \; (\text{outputs} \mid \text{condition})$$

Framework: optimize a model using data formatted as inputs (condition) and outputs

from Jong Wook Kim

Language models as a framework

Why discuss language models?

- Can use LMs to generate music symbolically:
  - As models that generate sequences of musical “tokens”
  - As models that generate structured (text-based) representations of music
- We’ll mostly use these as “black boxes” in this class; we’ll get into a few specific architectures in the next section, but for more see an NLP class!
- Later (Module 5?) we’ll look at other tasks involving music + LMs

Take-homes

- Mostly just try to get a handle on the *very general* p(sequence | condition) view of token generation
- The specific architectures don't matter that much (and this *probably* isn't the right class to "go deep" on how Transformers etc. work); as usual mostly treat these as "black box" tools for learning sequence generation models

Symbolic music generation

3.5: Language models for symbolic music generation

Language models for symbolic music generation

Based on

“Generative AI for Music and Audio Creation” (Michigan):
https://hermandong.com/teaching/pat498_598_fall2024/

Language models for symbolic music generation

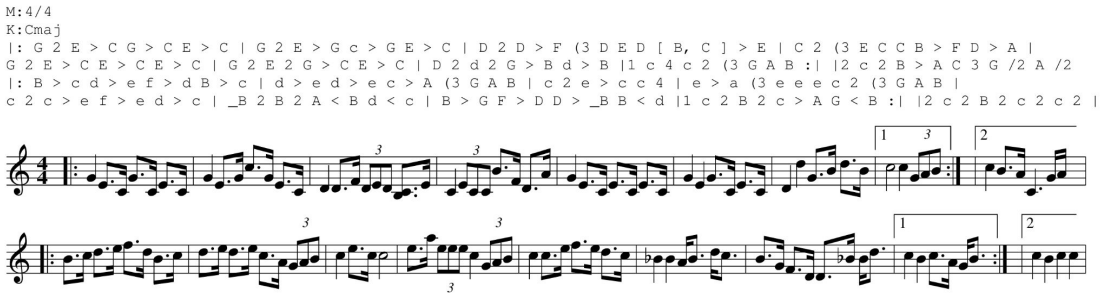
We've explored

1. Music tokenization schemes (without thinking too much about models)
2. And we've talked about language models (without thinking too much about music)

Finally, let's talk about both together and see what models and tokenization schemes some specific systems use

Example: Folk RNN (Sturm et al., 2015)

- Data
  - Collections of **folk tunes**
- Representation
  - **ABC notation** without metadata
- Model
  - **FolkRNN** uses a **character** level **LSTM** (long short-term mem

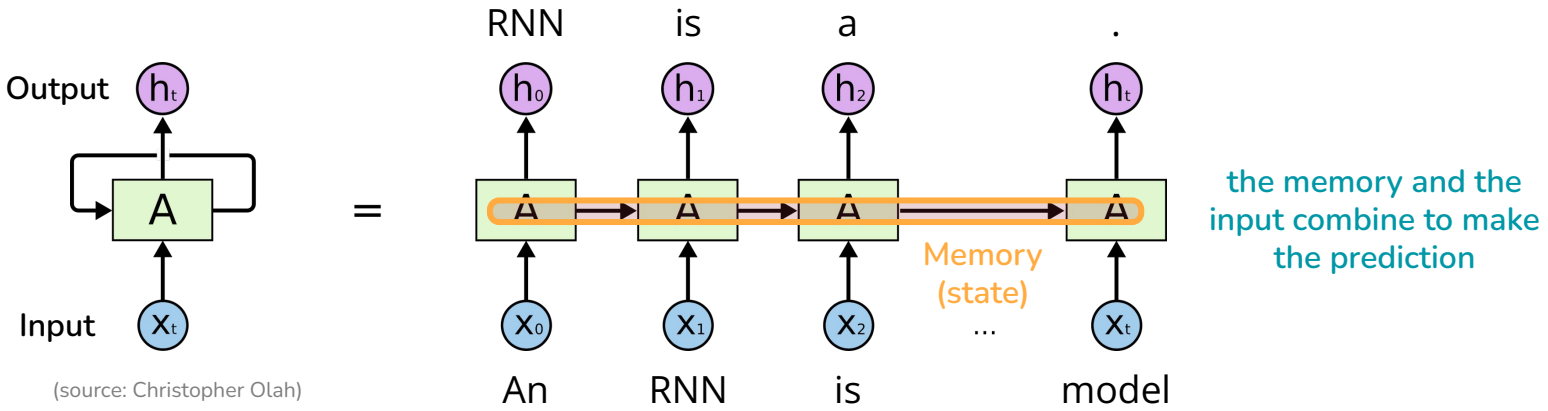


folkrnn.org

Bob L. Sturm, Joao Felipe Santos, and Iryna Korshunova, "Folk Music Style Modelling by Recurrent Neural Networks with Long Short Term Memory Units," *ISMIR-LBD*, 2015

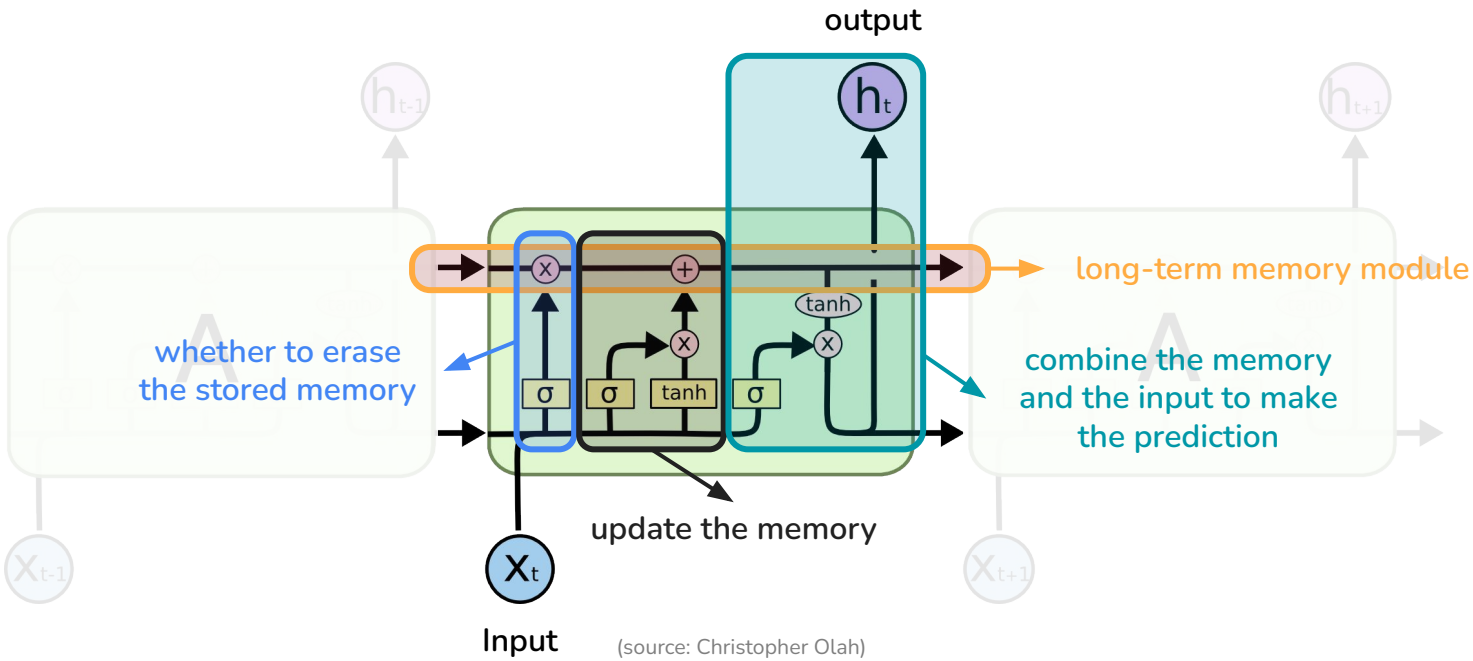
What is an RNN (Recurrent Neural Network)?

- A type of neural networks that has **loops**
- (Formerly) widely used for **modeling sequences** (e.g., in natural language processing)



colah.github.io/posts/2015-08-Understanding-LSTMs/

What is an LSTM?

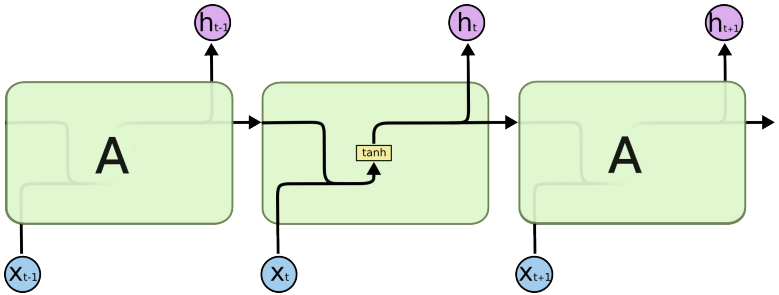


colah.github.io/posts/2015-08-Understanding-LSTMs/

Vanilla RNNs vs LSTMs

Vanilla RNNs

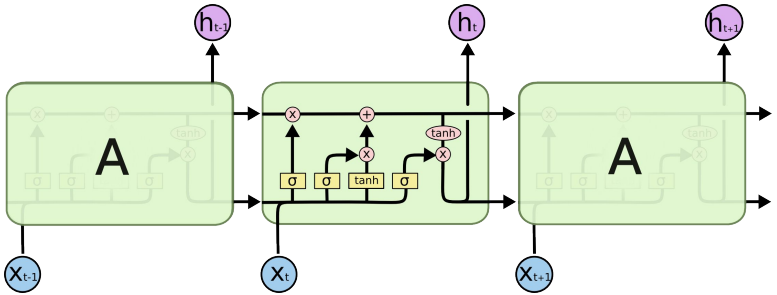
- Simplest form of RNNs
- Limited long-term memory



(source: Christopher Olah)

LSTMs

- Improved memory module
- Better long-term memory



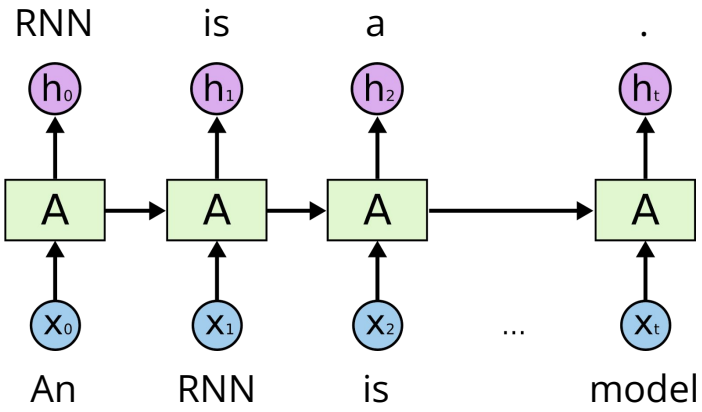
(source: Christopher Olah)

colah.github.io/posts/2015-08-Understanding-LSTMs/

What is a character-level RNN?

Word-level RNNs

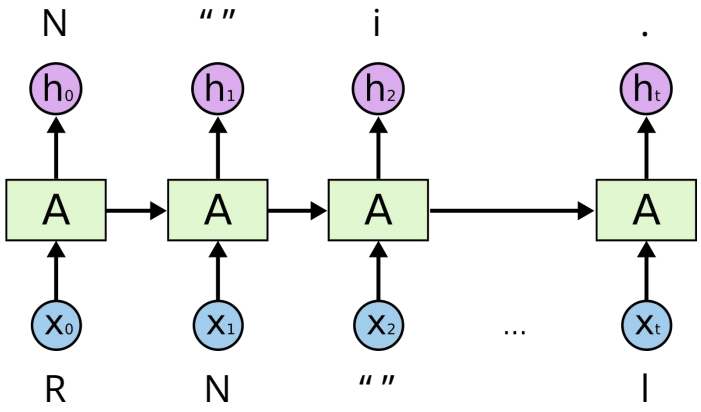
- Predicting word by word
- Most common



(source: Christopher Olah)

Character-level RNNs

- Predicting character by character
- Useful when there is no natural “spaces”



(source: Christopher Olah)

colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN-based symbolic music generation

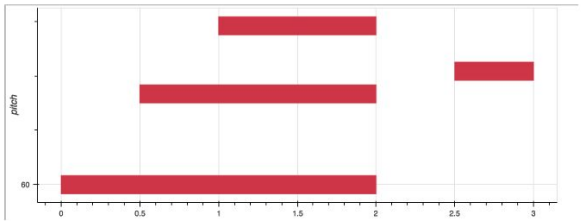
- Used to be the state-of-the-art
- Good for **not-very-long sequences** (e.g. 16 bars)
- Examples
  - Folk RNN (Sturm et al., 2015)
  - Melody RNN (Abolafia, 2016)
  - Performance RNN (Simon & Oore, 2017)
  - DeepBach (Hadjeres et al., 2017)

Bob L. Sturm, Joao Felipe Santos, and Iryna Korshunova, "Folk Music Style Modelling by Recurrent Neural Networks with Long Short Term Memory Units," *ISMIR-LBD,* 2015
Dan Abolafia, "A Recurrent Neural Network Music Generation Tutorial," *Magenta Blog*, June 10, 2016
Ian Simon and Sageev Oore, "Performance RNN: Generating Music with Expressive Timing and Dynamics," *Magenta Blog*, June 29, 2017
Gaëtan Hadjeres, François Pachet, Frank Nielsen, "DeepBach: a Steerable Model for Bach Chorales Generation," *ICML*, 2017

Example: Music Transformer (Huang et al., 2019)

- Data
  - Yamaha e-Piano Competition dataset (MAESTRO)
- Representation
  - **MIDI-like** representation
- Model
  - **Transformer**
- Demo page: https://magenta.tensorflow.org/music-transformer

Examples of generated music



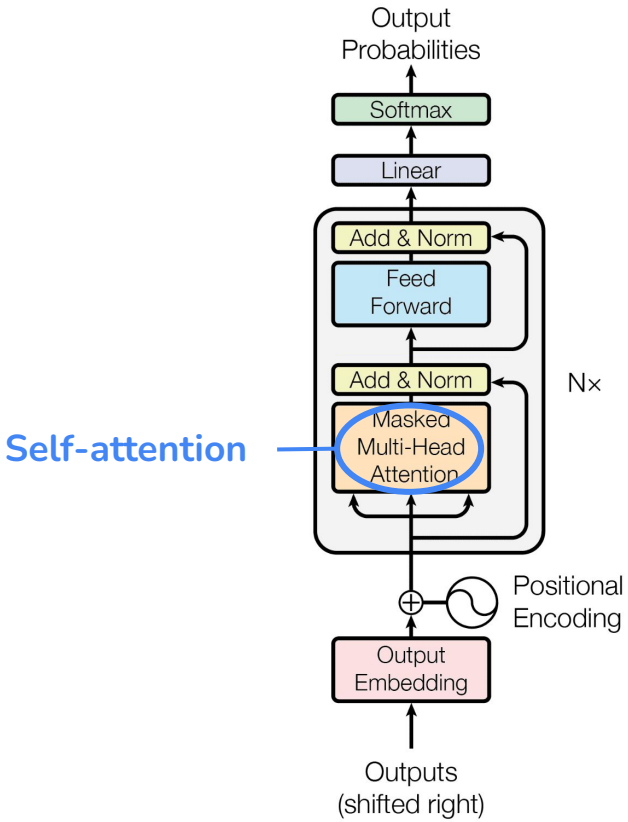

```
SET_VELOCITY<80>, NOTE_ON<60>
TIME_SHIFT<500>, NOTE_ON<64>
TIME_SHIFT<500>, NOTE_ON<67>
TIME_SHIFT<1000>, NOTE_OFF<60>, NOTE_OFF<64>,
NOTE_OFF<67>
TIME_SHIFT<500>, SET_VELOCITY<100>, NOTE_ON<65>
TIME_SHIFT<500>, NOTE_OFF<65>
```


Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck, "Music Transformer: Generating Music with Long-Term Structure," *ICLR*, 2019

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck, "Music Transformer: Generating Music with Long-Term Structure," *Magenta Blog*, December 13, 2018

What is a Transformer?

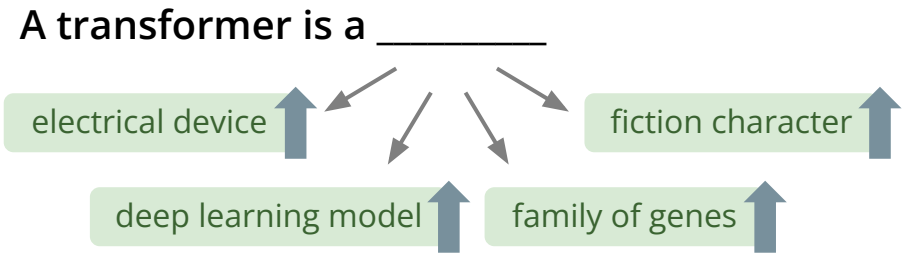
- a type of neural network that uses the **self-attention mechanism**
- we discuss **decoder-only** architecture here, usually used for unconditional, or prompt-based generation
- refer to the original paper for encoder/decoder architecture

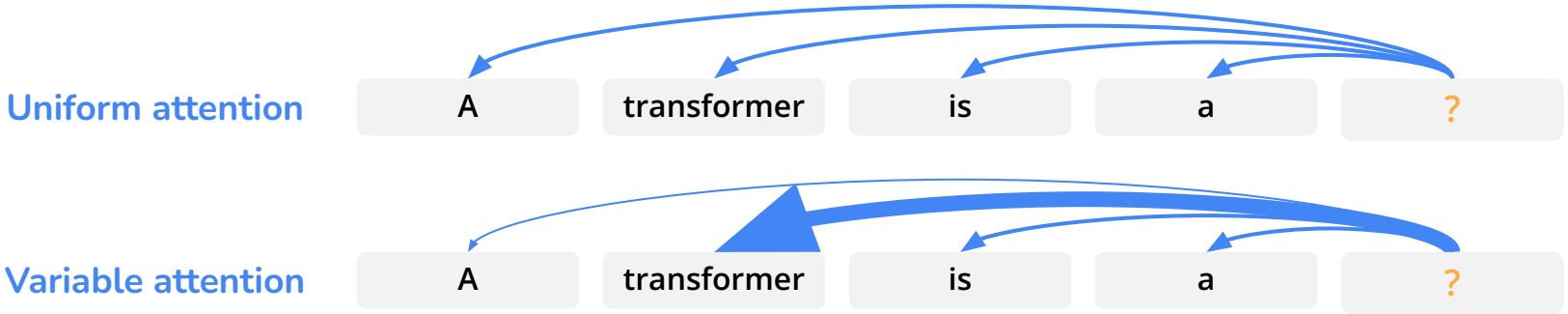


(source: Vaswani et al., 2017; adapted)

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, "Attention Is All You Need," *NeurIPS*, 2017

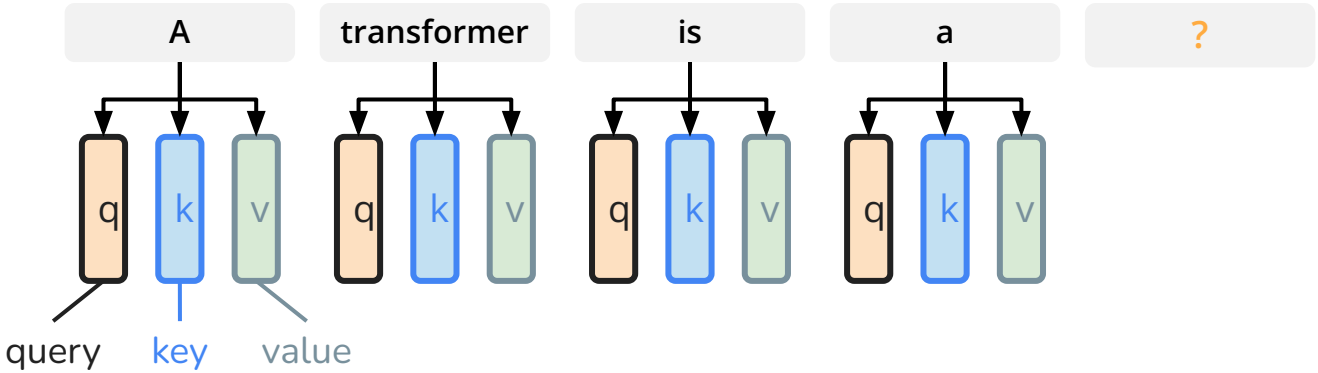
Self-attention mechanism



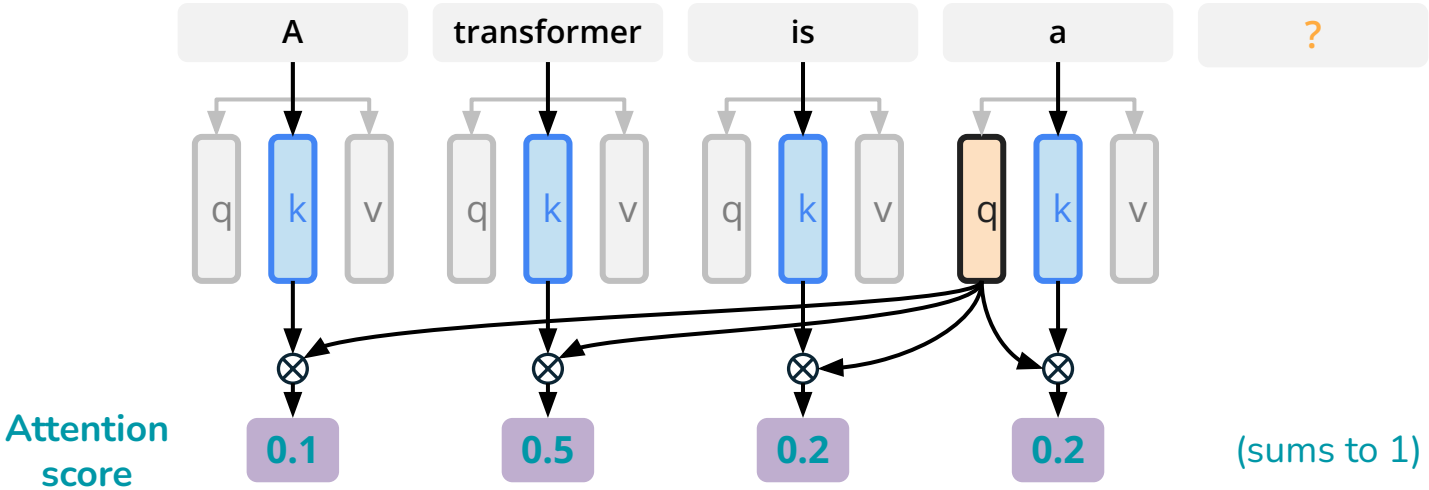


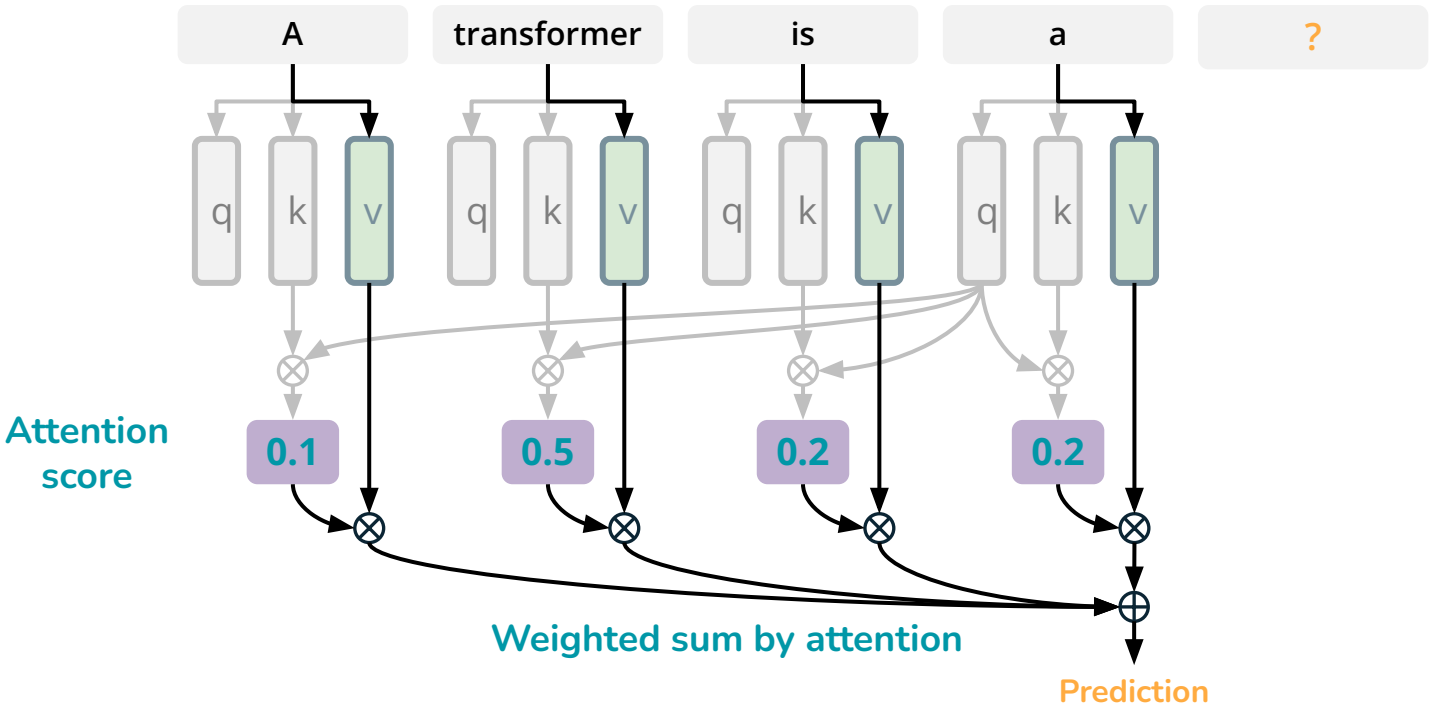
Transformers learn what to attend to from big data!

Self-attention mechanism

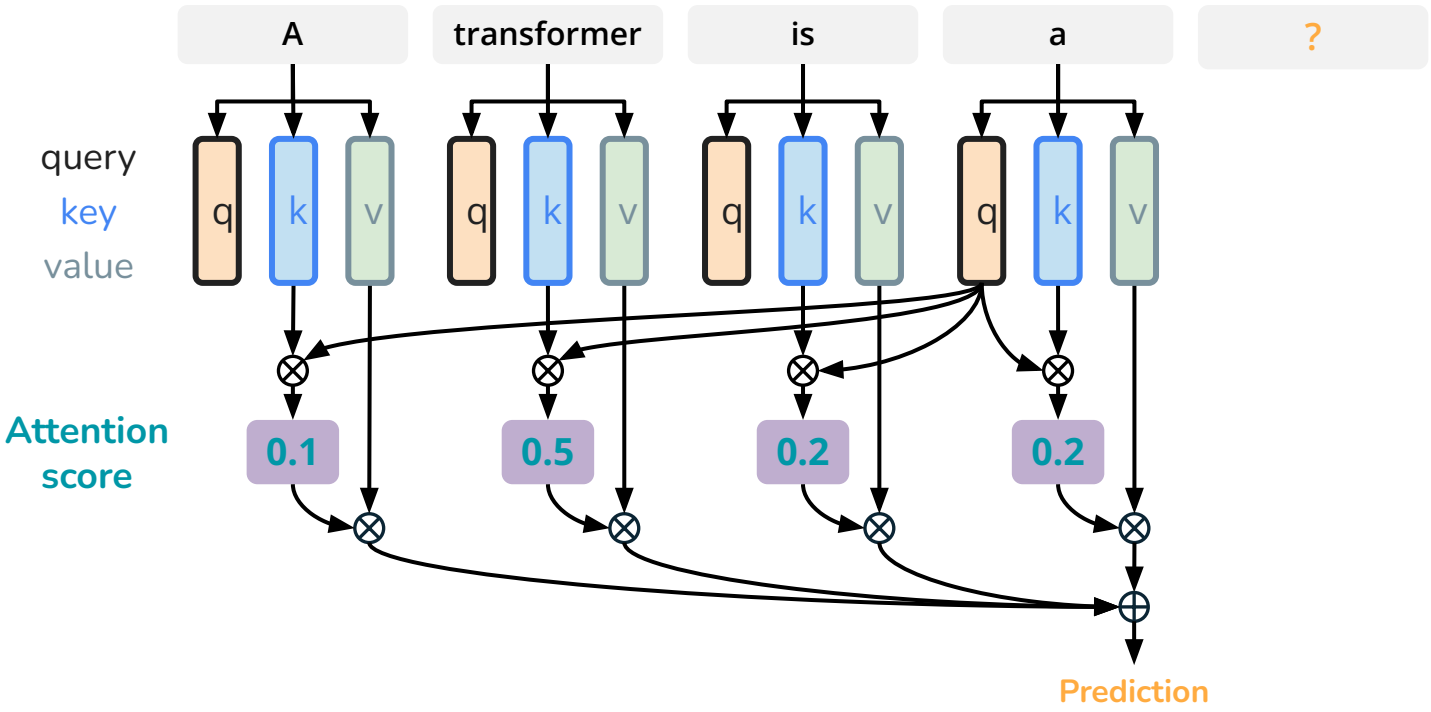


Self-attention mechanism




A
transformer
is
a
?
q
k
v
Attention score
0.1
0.5
0.2
0.2
Weighted sum by attention
Prediction

Self-attention mechanism



Visualizing musical self-attention

(each color represents an attention head)

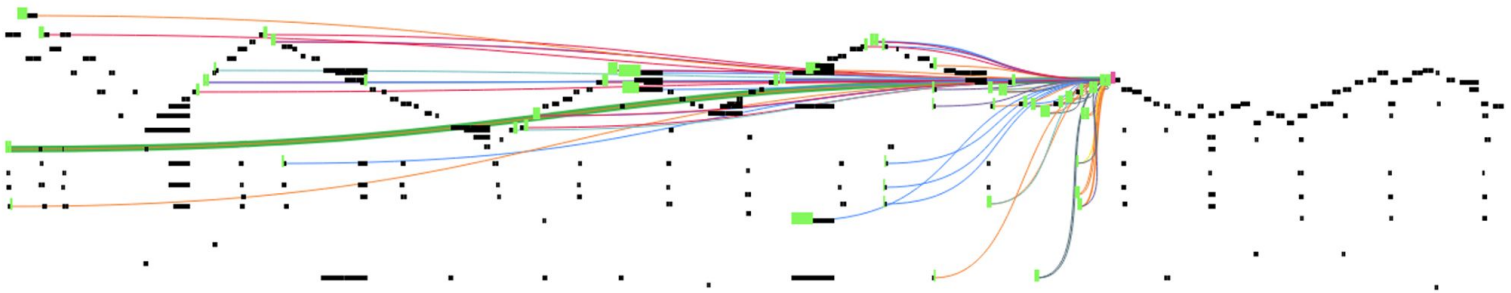


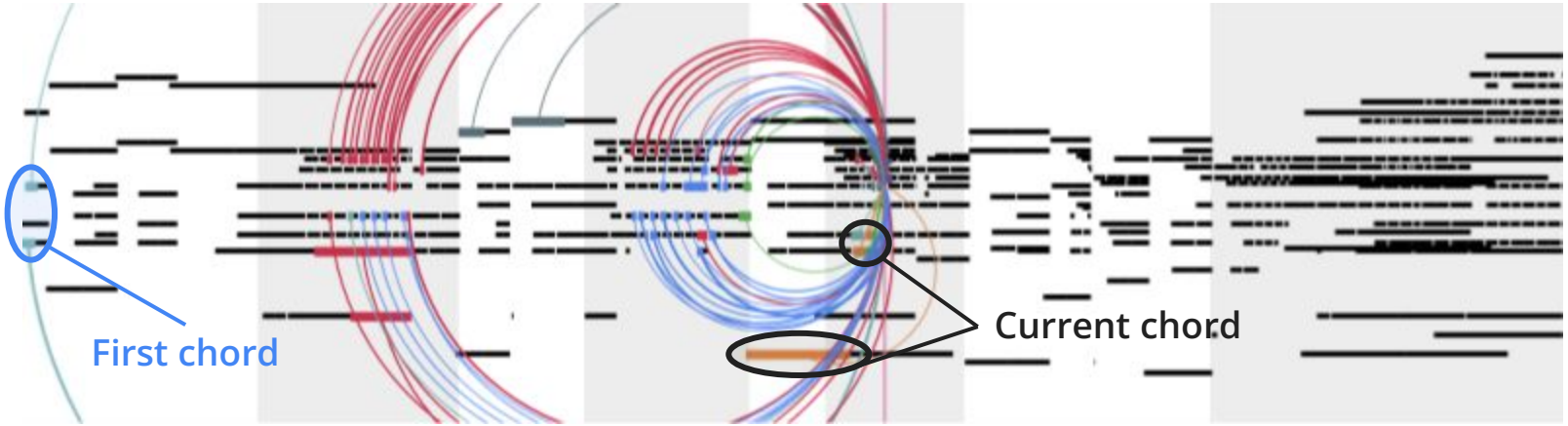
Figure 8: This piece has a recurring triangular contour. The query is at one of the latter peaks and it attends to all of the previous high notes on the peak, all the way to beginning of the piece.

(source: Huang et al., 2018)

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck, "Music Transformer: Generating Music with Long-Term Structure," *ICLR*, 2019

Visualizing musical self-attention

(each color represents an attention head)



(source: Huang et al., 2018)

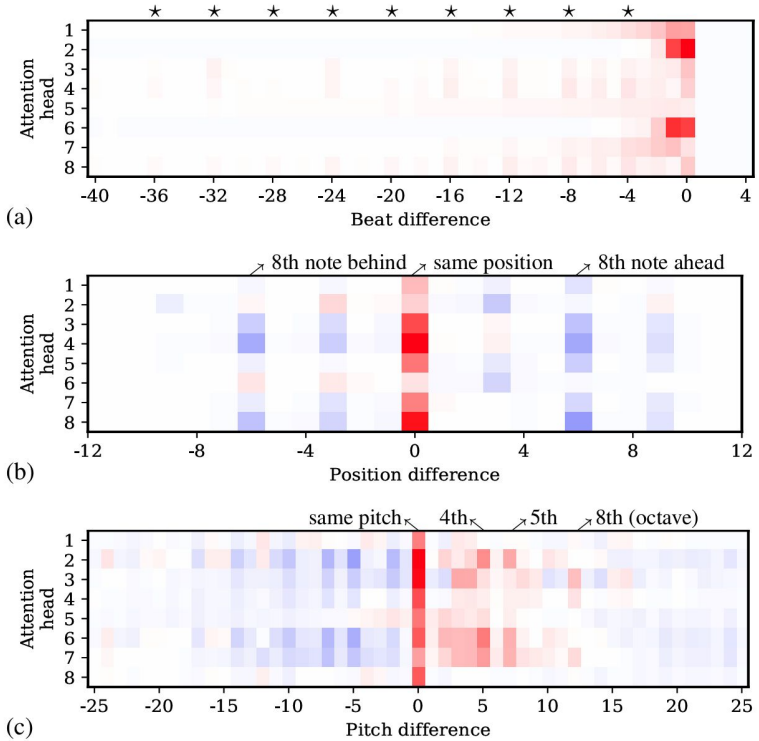
Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck, "Music Transformer: Generating Music with Long-Term Structure," *ICLR*, 2019

Visualizing musical self-attention

(source: Huang et al., 2018)

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck, "Music Transformer: Generating Music with Long-Term Structure," *ICLR*, 2019

Visualizing musical self-attention



- Red and blue colors indicate positive and negative values, respectively
- The learned self-attention is arguably related to music theory principles

(*figure b: notes “on beat” attend more to notes on beat, and notes “off beat” attend more to notes off beat)

Hao-Wen Dong, Ke Chen, Shlomo Dubnov, Julian McAuley, and Taylor Berg-Kirkpatrick, “Multitrack Music Transformer,” *ICASSP*, 2023

Transformer-based symbolic music generation

- Work better for **longer** sequences
- At the expense of computational power
- Represent the **SOTA** approach for sequence modeling
- Examples
  - Music Transformer (Huang et al., 2019)
  - LakhNES (Donahue et al., 2019)
  - Pop Music Transformer (Huang & Yang, 2020)
  - Multitrack Music Transformer (Dong et al., 2023)

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck, "Music Transformer: Generating Music with Long-Term Structure," *ICLR*, 2019

Chris Donahue, Huanru Henry Mao, Yiting Ethan Li, Garrison W. Cottrell, Julian McAuley, "LakhNES: Improving multi-instrumental music generation with cross-domain pre-training," *ISMIR* 2019

Yu-Siang Huang, Yi-Hsuan Yang. "Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions", *ACM Multimedia,* 2020.

Hao-Wen Dong, Ke Chen, Shlomo Dubnov, Julian McAuley, and Taylor Berg-Kirkpatrick, "Multitrack Music Transformer," *ICASSP*, 2023

Code example

workbook3.ipynb:

Simple RNN-based model; won't work very well (the training dataset and model are tiny, for the sake of getting it running!) but can use this as the basis of a more complex solution if you'd like to try this for Assignment 2!

Symbolic music generation

3.6: 2D symbolic representations & GANs

2D symbolic representations

Several slides based on

“Generative AI for Music and Audio Creation” (Michigan):

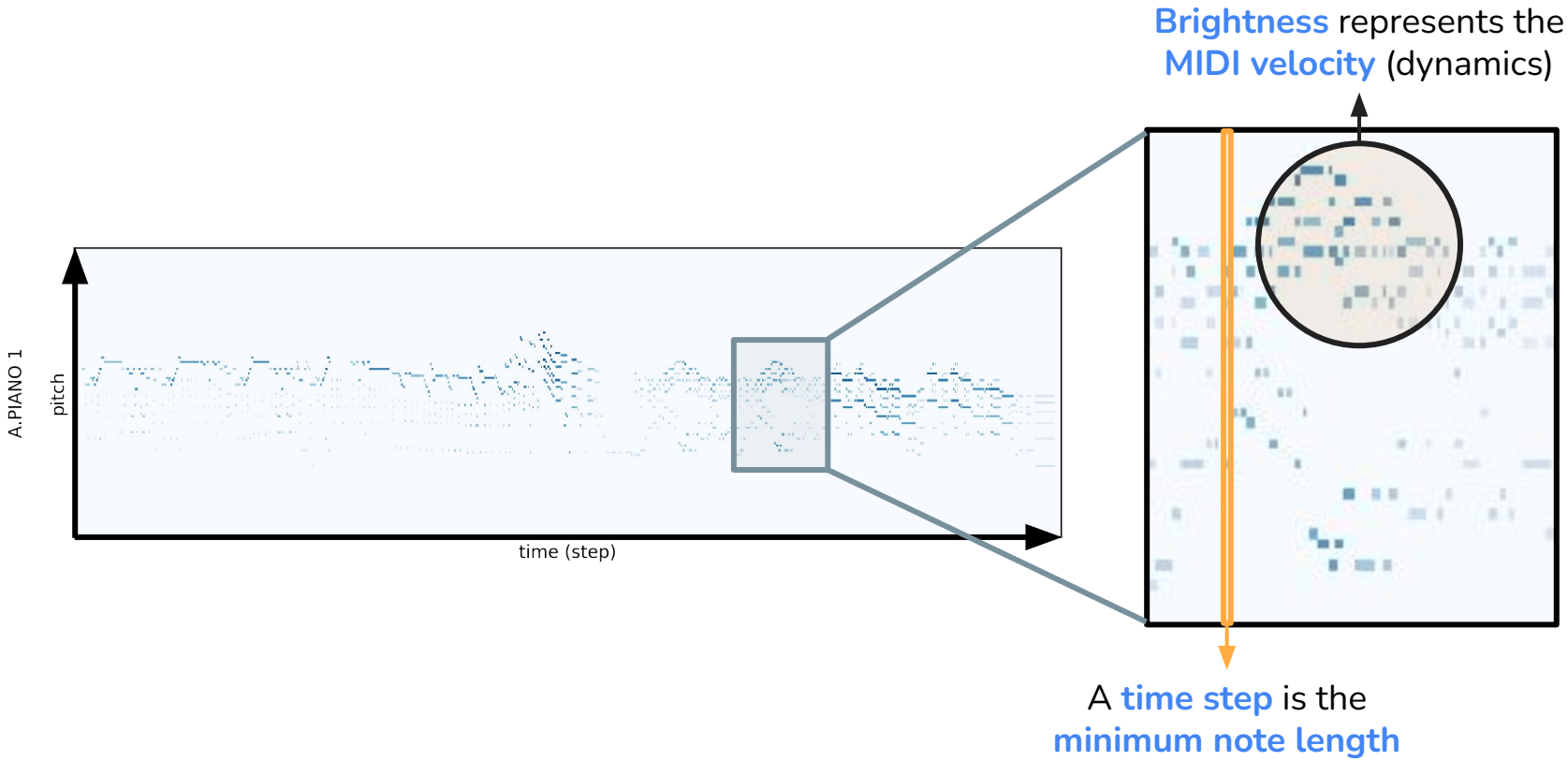
https://hermandong.com/teaching/pat498_598_fall2024/

and some from

“Deep Learning for Music Analysis and Generation”:

https://github.com/affige/DeepMIR

(Recap) Piano rolls

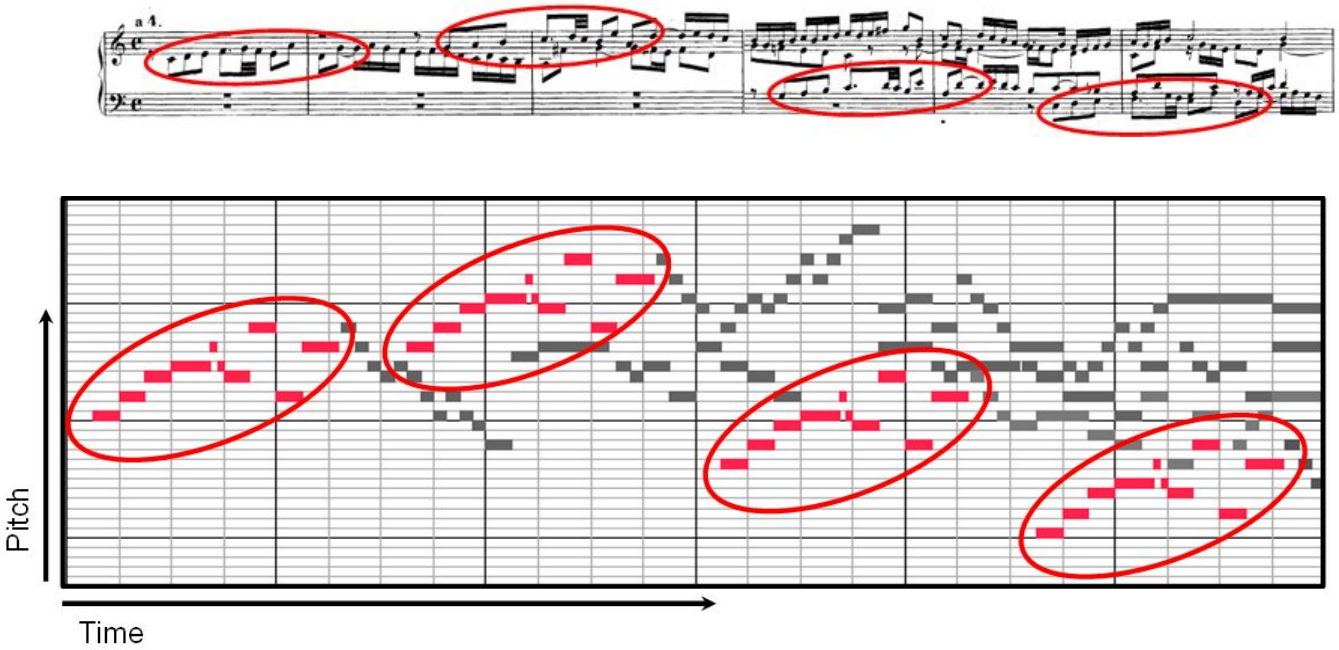


(Recap) Piano rolls

Why is this representation useful?

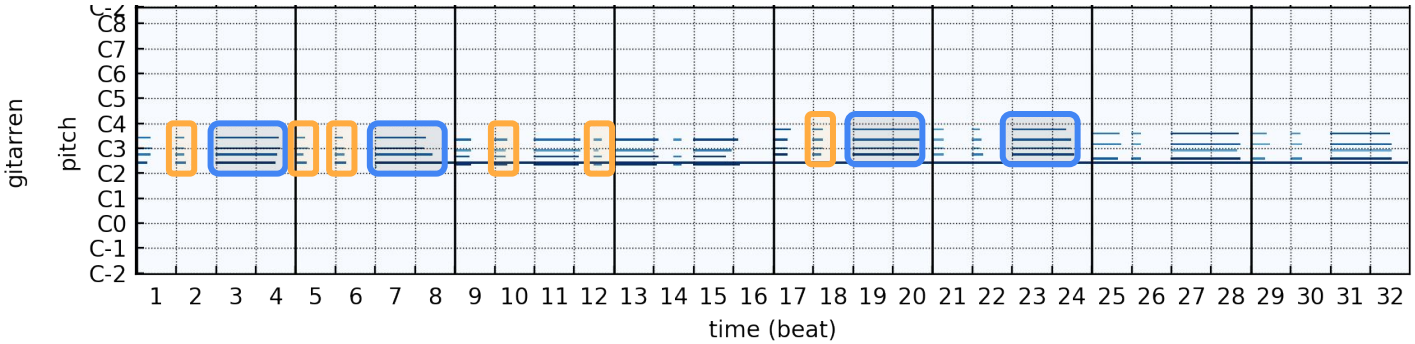
- Encodes time, pitch, and dynamics in a single object (a matrix)
- Straightforward to handle simultaneous (compared to e.g. language-ish representations before)
- Possibly will get the benefits we get from image models (CNNs): can learn invariances across pitch and time

Why piano rolls?



easy (for humans and models) to “see” repeated patterns!

Why piano rolls?



Many musical patterns like melodies, chords, scales and arpeggios are translation invariant in the temporal and pitch axes

Image-based symbolic music generation

We discuss two main approaches:

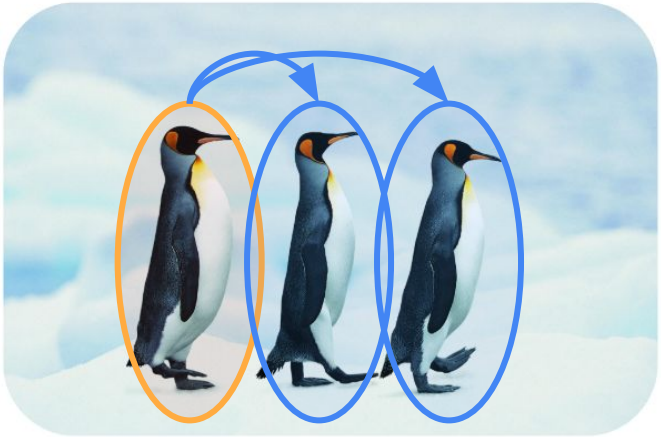
- Non-GAN
  - Coconet (Huang et al., 2017)
- Convolutional GAN
  - MidiNet (Yang et al., 2017)
  - MuseGAN (Dong et al., 2018)
  - LeadsheetGAN (Liu & Yang, 2018)
  - etc...

Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, “Counterpoint by convolution”, *ISMIR,* 2017
Li-Chia Yang, Szu-Yu Chou, and Yi-Hsuan Yang, “MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation,” *ISMIR*, 2017
Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang, “MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment,” *AAAI*, 2018
Hao-Min Liu, Yi-Hsuan Yang, "Lead sheet generation and arrangement by conditional generative adversarial network", *ICMLA*, 2018

(Recap) CNNs: Reusable pattern detectors

Intuition: Want to learn a **reusable local pattern detector**

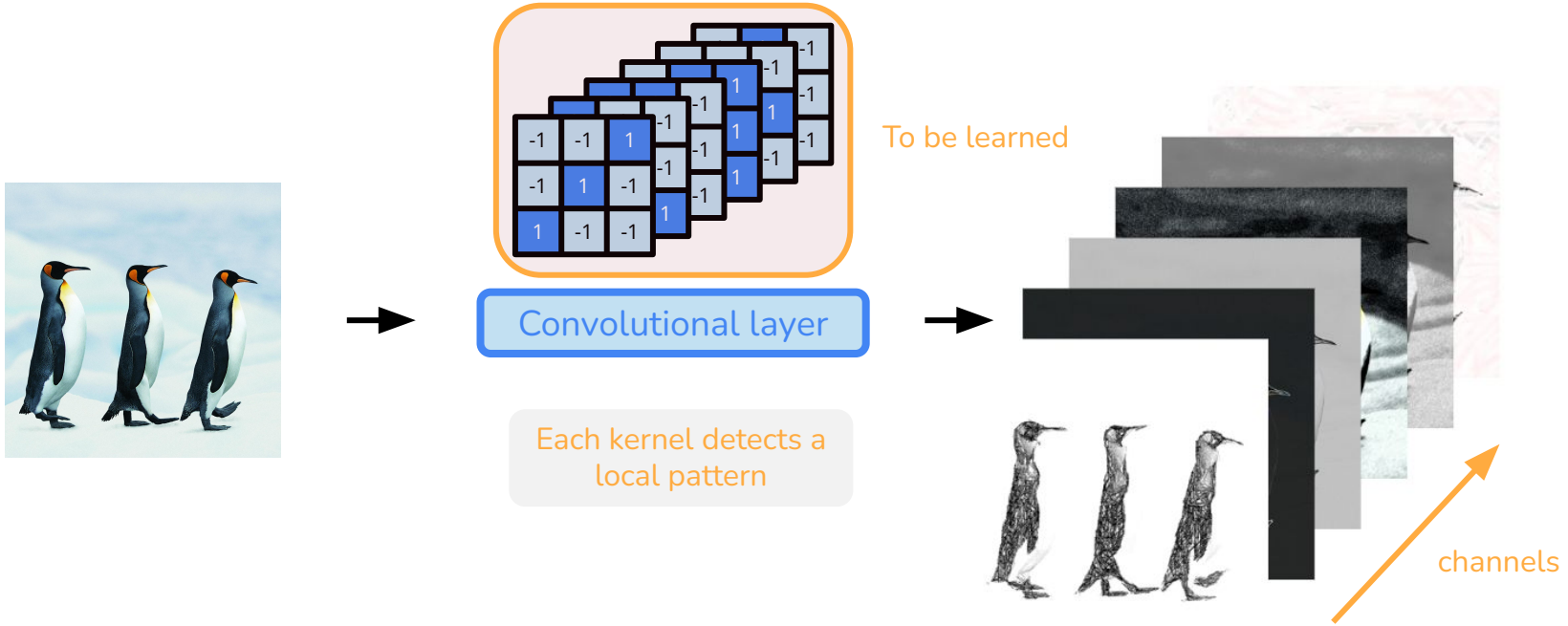
- Widely used in computer vision
- Also used for music and audio
  - Representing music as piano rolls
  - Representing audio as spectrograms



Objects have similar **local patterns** (coloring, beak shape, etc.); the patterns learned from one penguin can be reused to recognize similar features in the others.

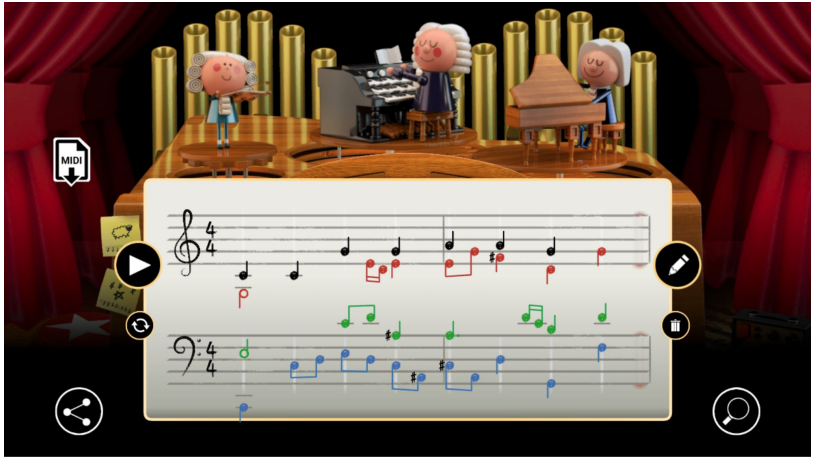
(Recap) 2D Convolution

A convolutional **layer** in a CNN consists of many **learnable kernels** (channels)



Example: Coconet & Bach Doodle (Huang et al., 2017)

- A **convolutional neural network** to complete **partial** musical scores
- How does it work? take a piece from Bach, randomly erase some notes, and ask the model to guess the missing notes from context

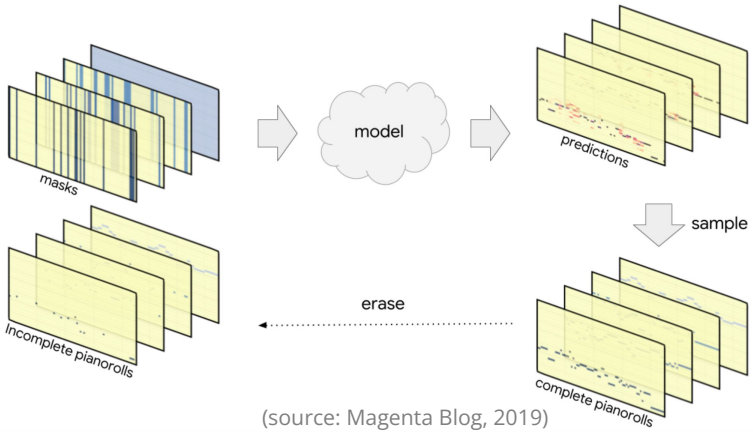


celebrating-johann-sebastian-bach

Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, "Counterpoint by convolution", *ISMIR,* 2017
"Coconet: the ML model behind today's Bach Doodle", *Magenta Blog*, March 20, 2019

Example: Coconet & Bach Doodle (Huang et al., 2017)

- Here, “musical scores” are three-dimensional convolutional featuremaps
  - 3D: time, pitch and voice (i.e. soprano, alto, tenor and bass)
- With a stack of piano rolls containing incomplete scores as input, the model outputs another stack of piano rolls containing probability distributions over the pitches of the erased notes



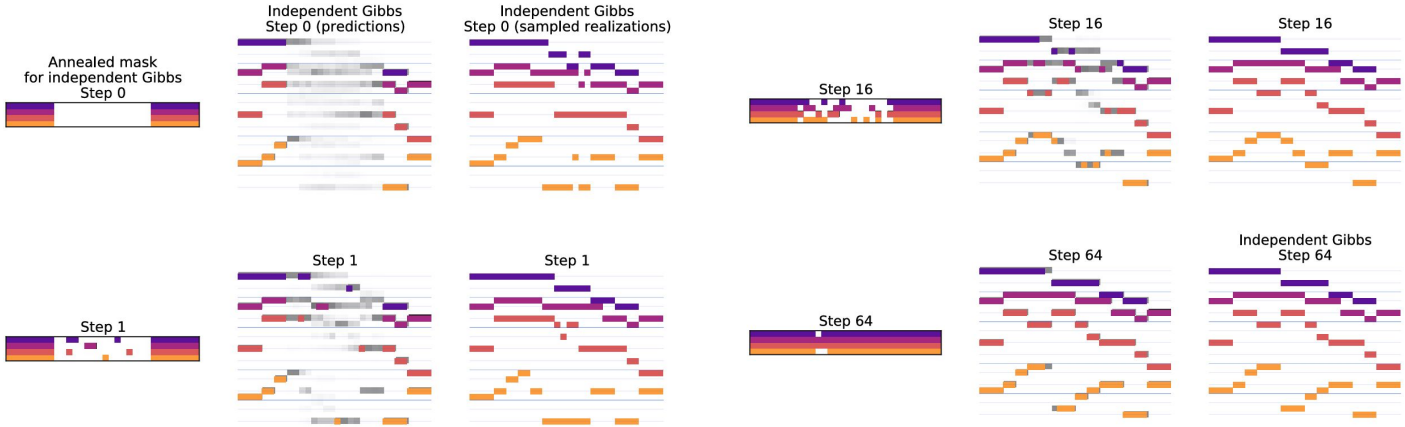
(source: Magenta Blog, 2019)

Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, “Counterpoint by convolution”, *ISMIR,* 2017
“Coconet: the ML model behind today’s Bach Doodle”, *Magenta Blog*, March 20, 2019

Example: Coconet & Bach Doodle (Huang et al., 2017)

How to sample? An analogue to rewriting!

- At each step, a random subset of notes is removed, and the model is asked to sample their values simultaneously from the probability distribution, obtaining a complete (but typically nonsensical) score; the process is repeated until a coherent result is found

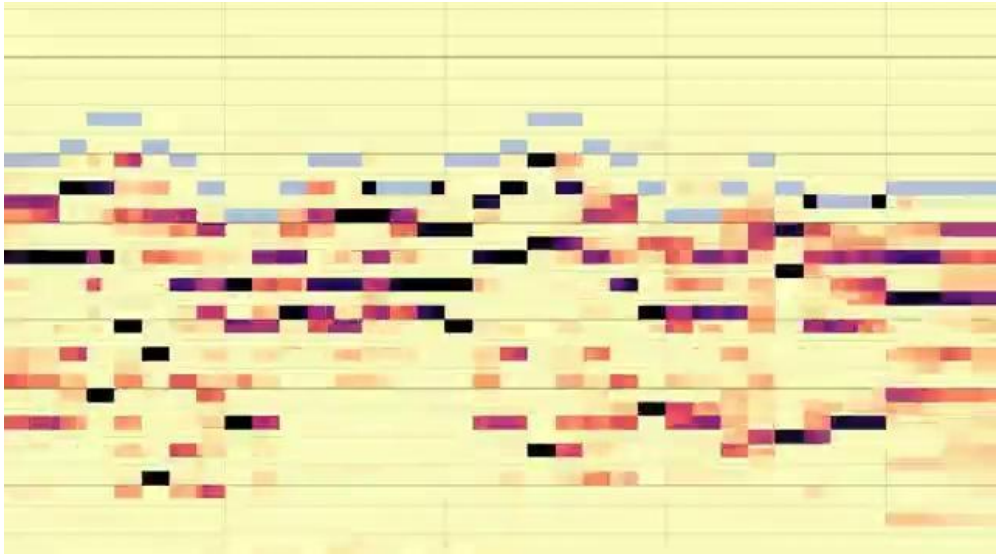


(source: Huang et al., 2017)

Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, "Counterpoint by convolution", *ISMIR,* 2017
"Coconet: the ML model behind today's Bach Doodle", *Magenta Blog*, March 20, 2019

Example: Coconet & Bach Doodle (Huang et al., 2017)

A visualization of this rewriting process



(source: Magenta Blog, 2019)

Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, “Counterpoint by convolution”, *ISMIR,* 2017
“Coconet: the ML model behind today’s Bach Doodle”, *Magenta Blog*, March 20, 2019

Example: Coconet & Bach Doodle (Huang et al., 2017)

- Whereas traditional models generate notes **in chronological order** from beginning to end (e.g. some text-based methods), Coconet can start anywhere and develop the material **in any order**
- It starts with rough ideas, and then goes back and forth to work out details and tweak the material into a coherent whole.
- Examples:
  - left: melody, right: harmonization

Example 1


Example 2


Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, "Counterpoint by convolution", *ISMIR,* 2017
"Coconet: the ML model behind today's Bach Doodle", *Magenta Blog*, March 20, 2019

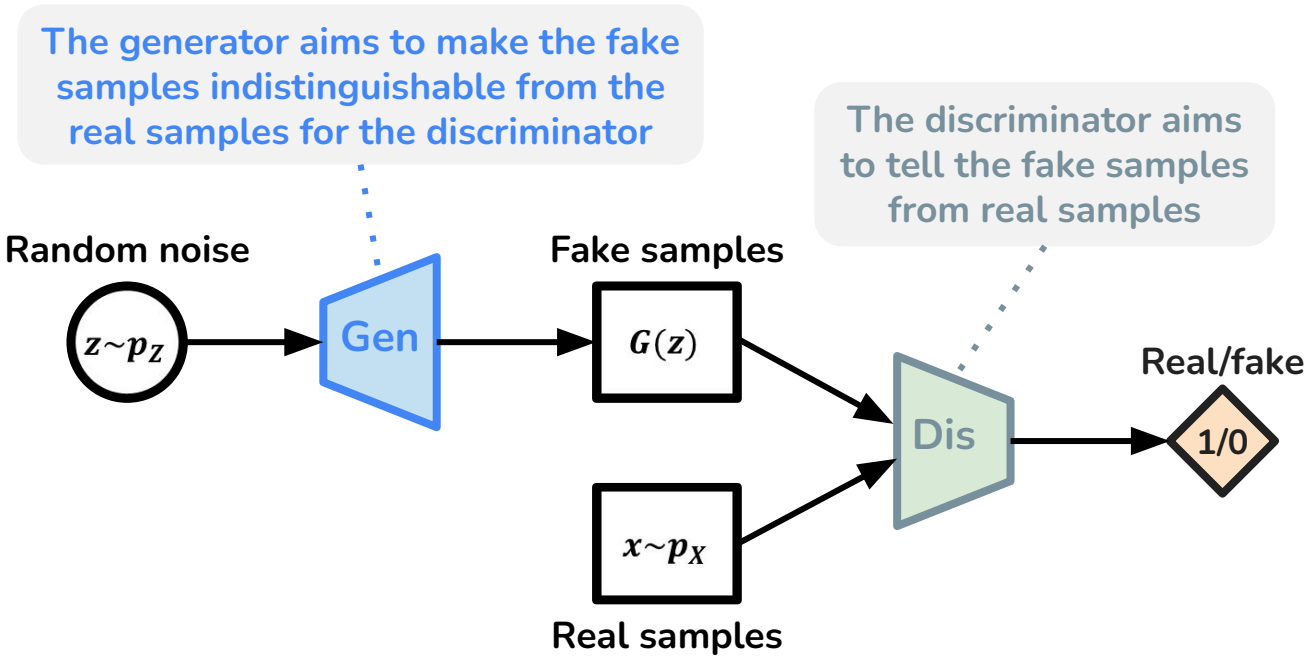
Image-based symbolic music generation

We discuss two main approaches:

- Non-GAN CNN-based Approach
  - Coconet (Huang et al., 2017)
- **Convolutional GAN**
  - MidiNet (Yang et al., 2017)
  - MuseGAN (Dong et al., 2018)
  - LeadsheetGAN (Liu & Yang, 2018)
  - etc...

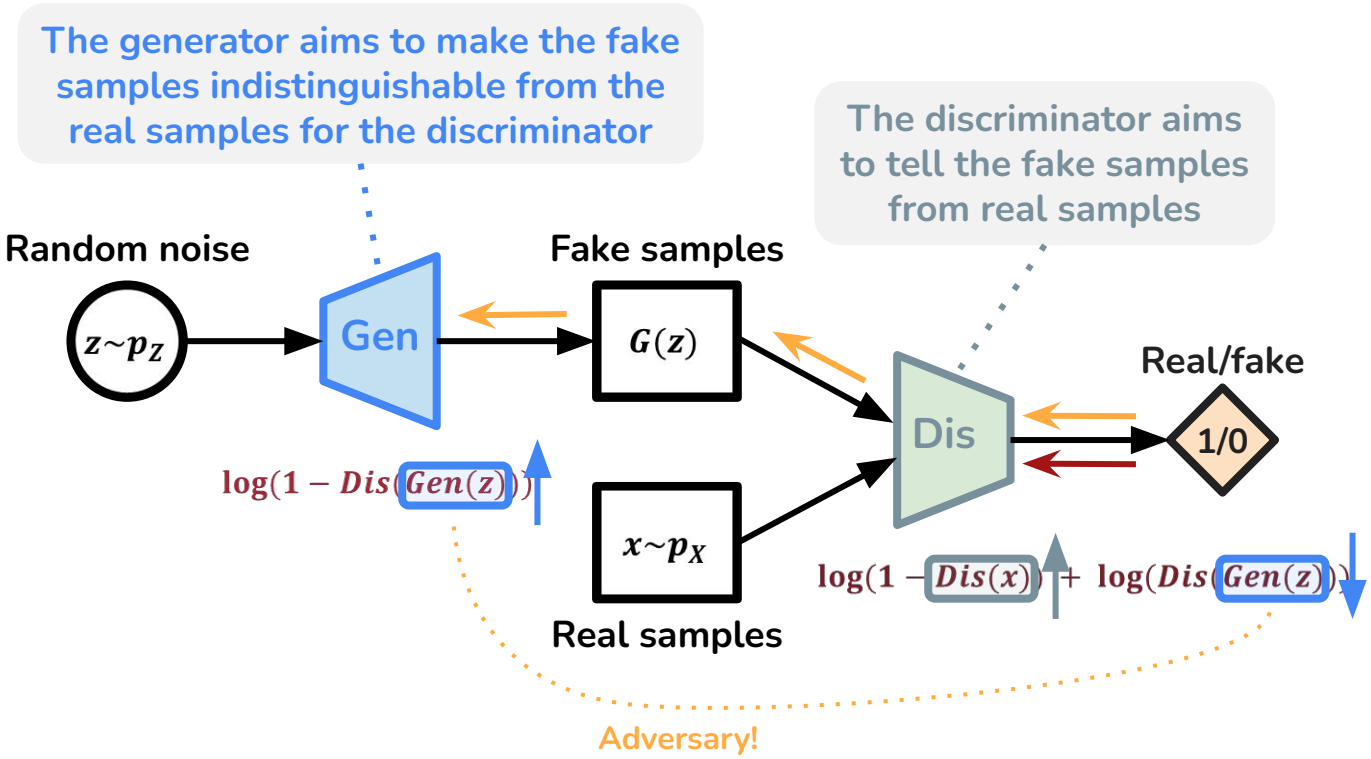
Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, Douglas Eck, “Counterpoint by convolution”, *ISMIR,* 2017
Li-Chia Yang, Szu-Yu Chou, and Yi-Hsuan Yang, “MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation,” *ISMIR*, 2017
Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang, “MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment,” *AAAI*, 2018
Hao-Min Liu, Yi-Hsuan Yang, "Lead sheet generation and arrangement by conditional generative adversarial network", *ICMLA*, 2018

Generative Adversarial Nets (GANs)



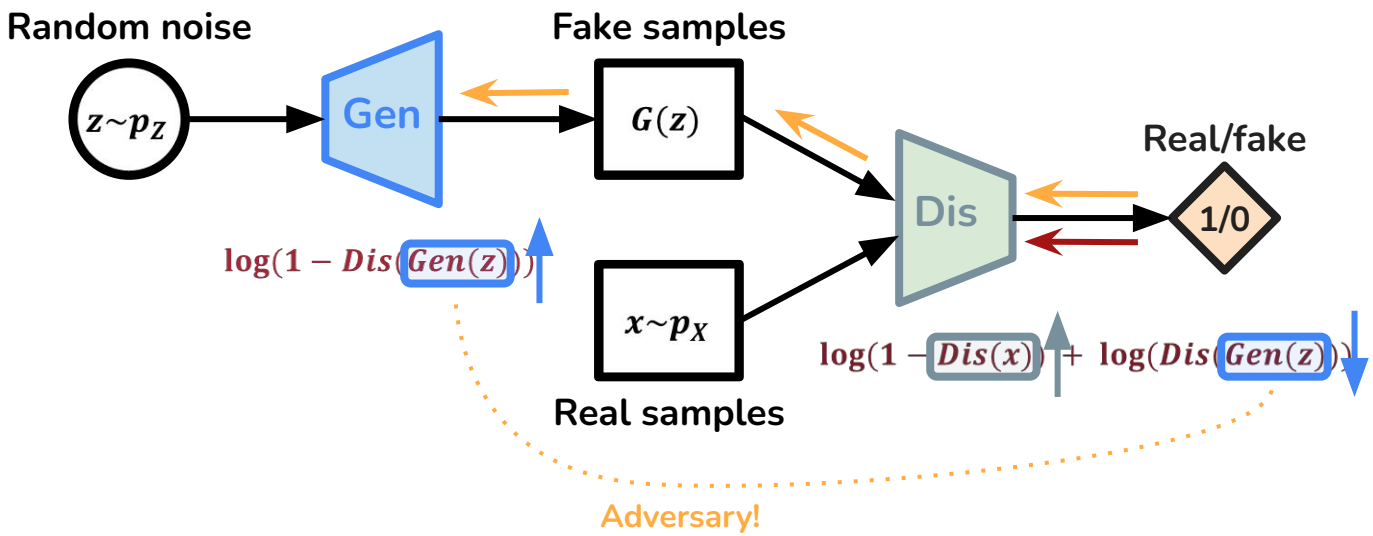
Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, "Generative Adversarial Networks," *NeurIPS*, 2014

Generative Adversarial Nets (GANs): Training



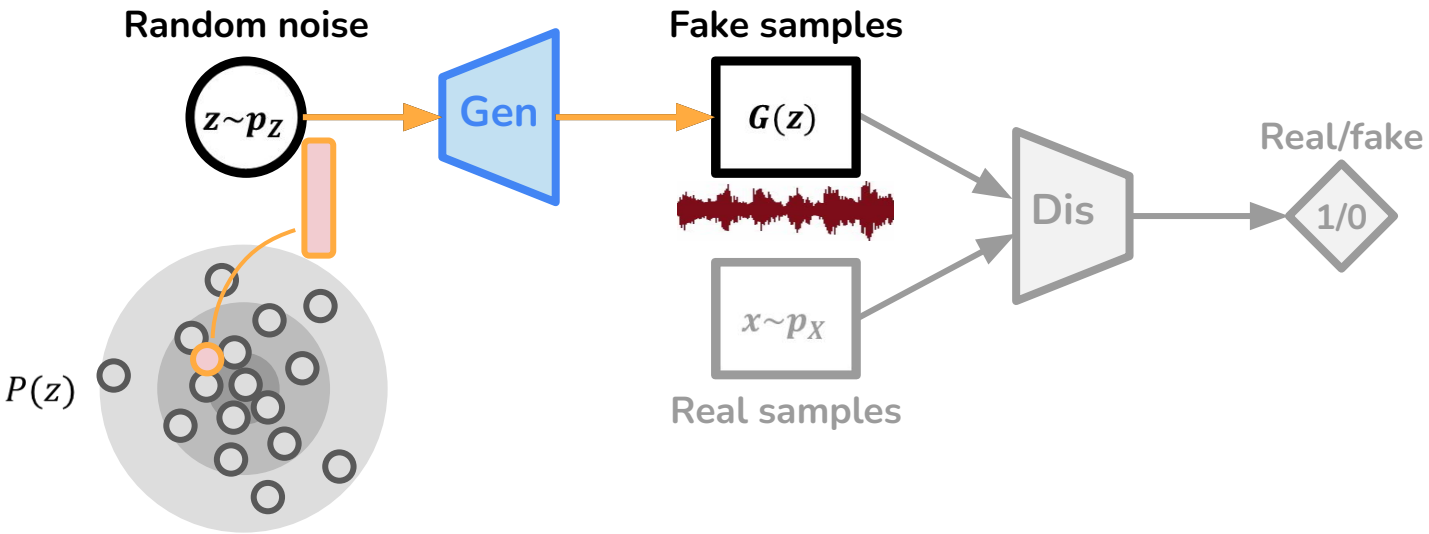
Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, "Generative Adversarial Networks," *NeurIPS*, 2014

Generative Adversarial Nets (GANs): Generation



Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, "Generative Adversarial Networks," *NeurIPS*, 2014

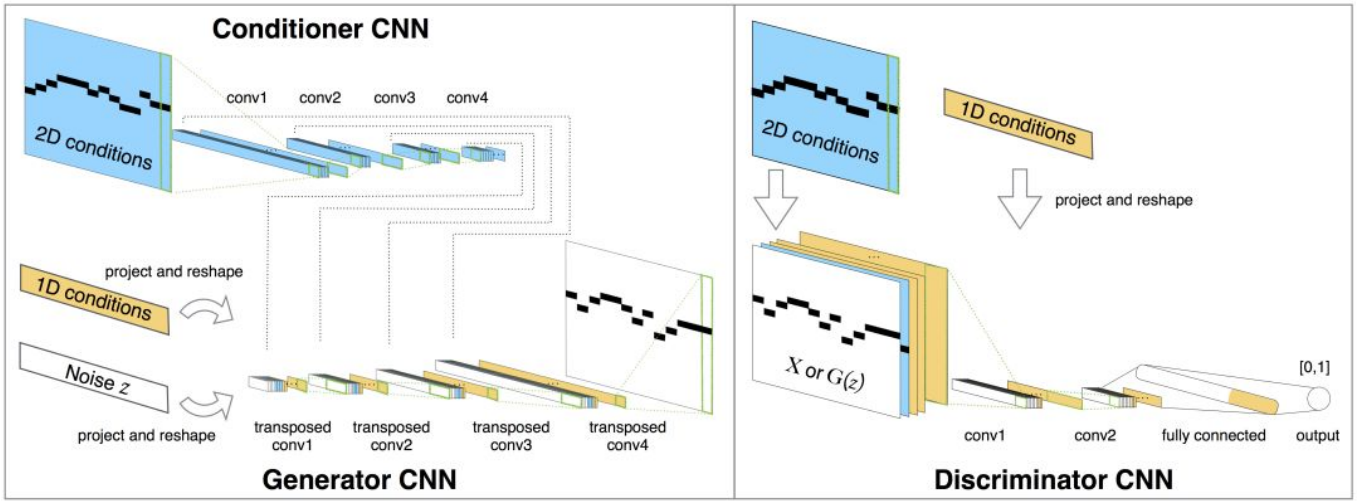
Generative Adversarial Nets (GANs): Generation



Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, "Generative Adversarial Networks," *NeurIPS*, 2014

Example: MidiNet (Yang et al., 2017)

- Convolutional GAN for **one-bar melody** generation
  - Conditioned on the previous bar (2D conditions), or on the chord (1D conditions)

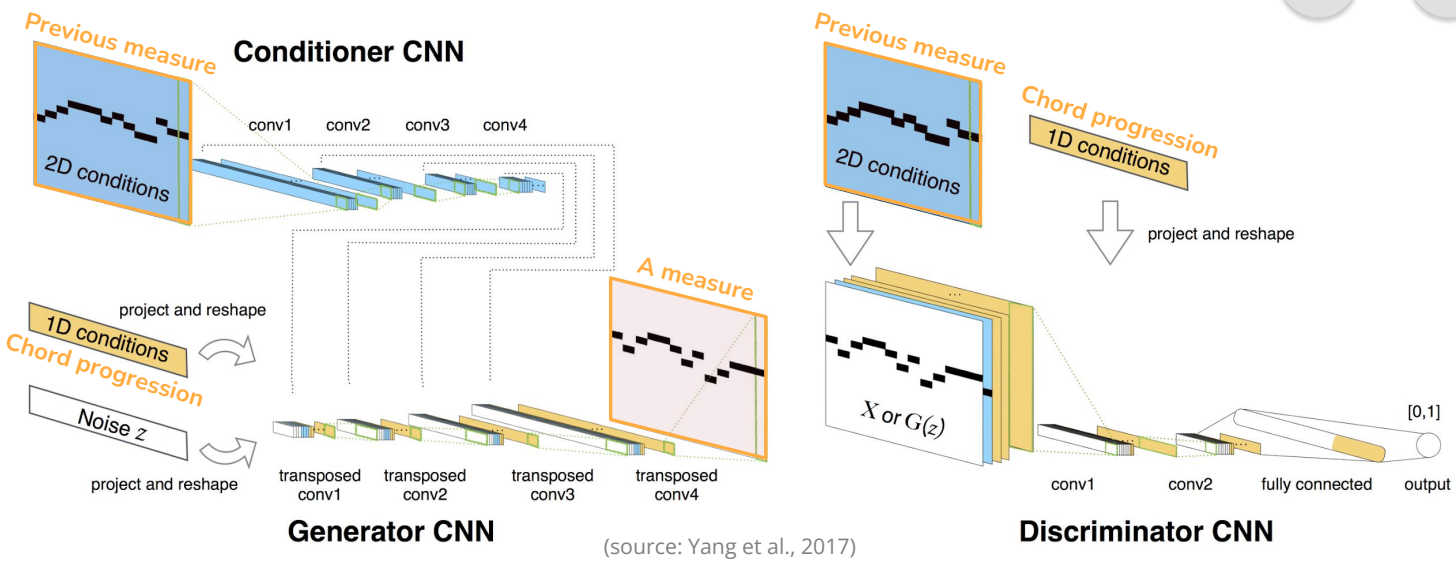


(source: Yang et al., 2017)

Li-Chia Yang, Szu-Yu Chou, and Yi-Hsuan Yang, "MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation," *ISMIR*, 2017

Example: MidiNet (Yang et al., 2017)

Examples of generated music

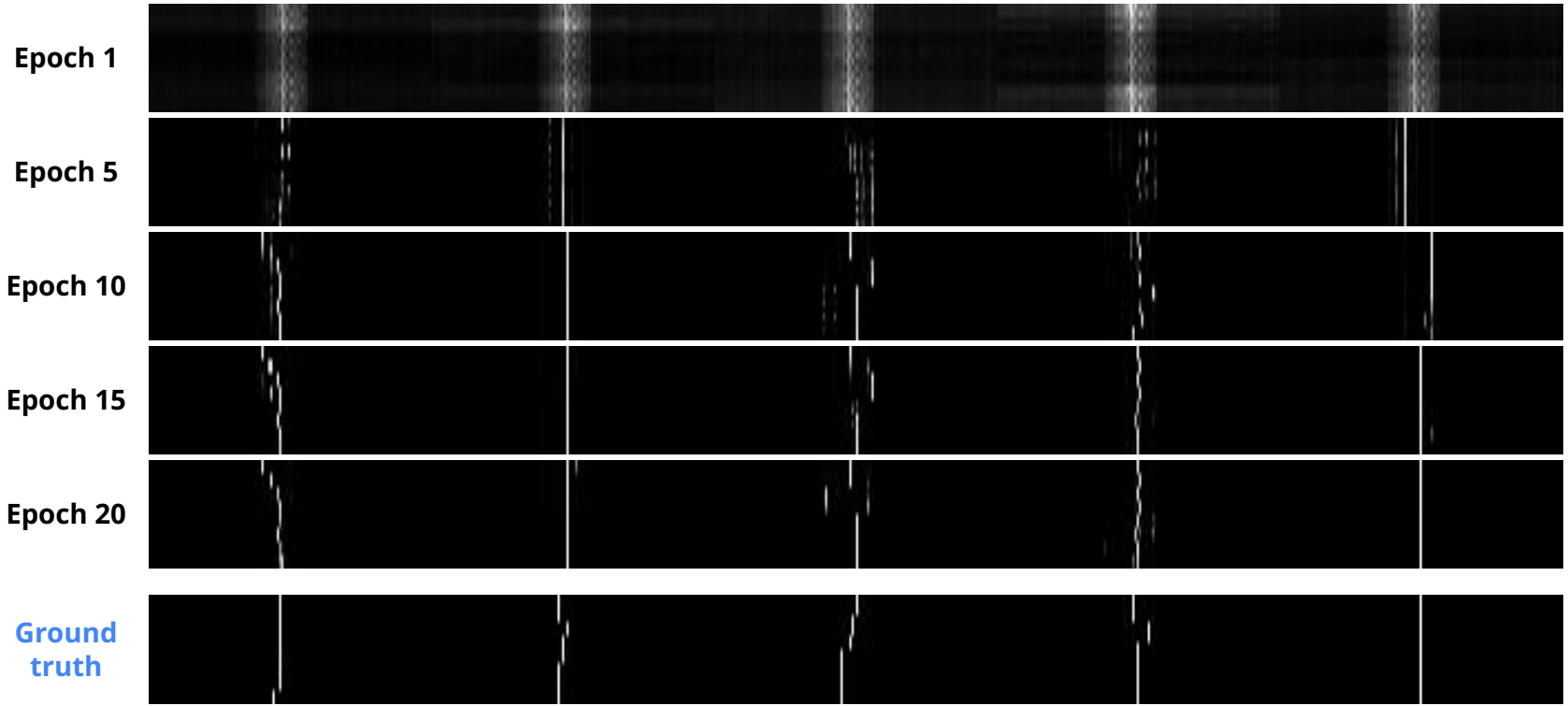


(source: Yang et al., 2017)

MidiNet generates music measure-by-measure by conditioning on the last measure generated

Li-Chia Yang, Szu-Yu Chou, and Yi-Hsuan Yang, "MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation," *ISMIR*, 2017

Example: MidiNet (Yang et al., 2017)

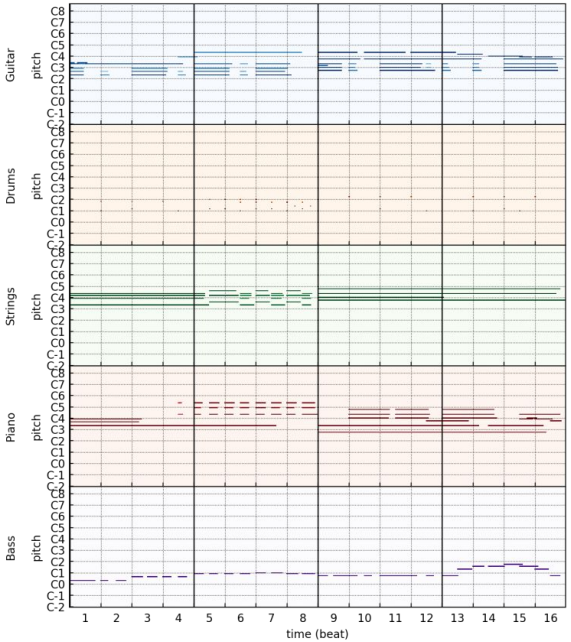


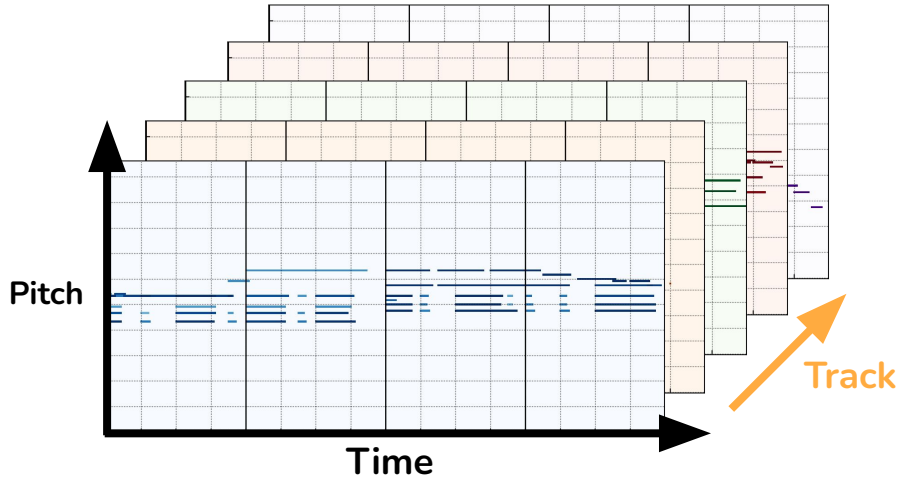
(source: Yang et al., 2017)

Li-Chia Yang, Szu-Yu Chou, and Yi-Hsuan Yang, "MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation," *ISMIR*, 2017

Multitrack piano rolls

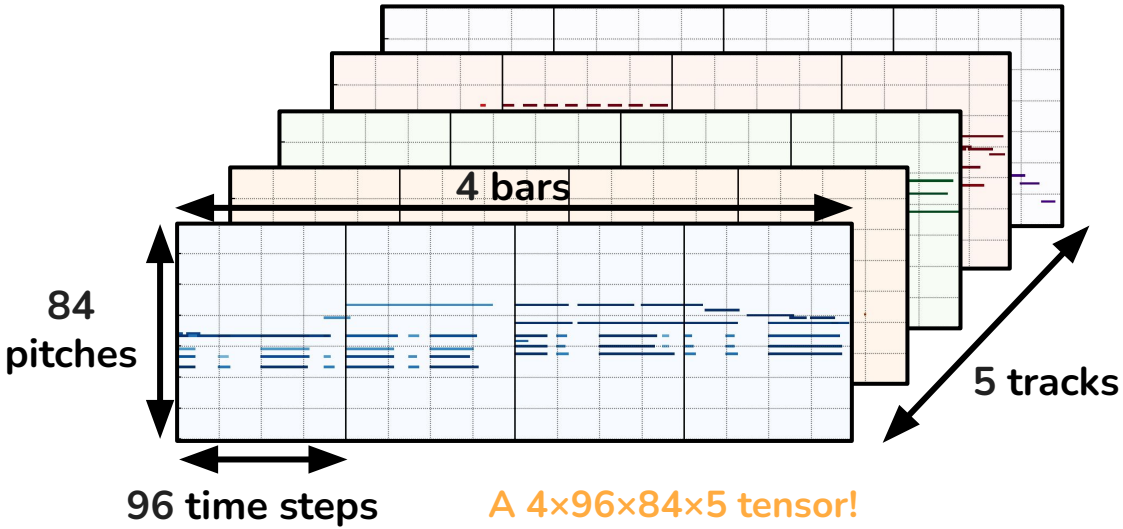
- Represent different voices / tracks in different channels of a fixed-size tensor





Example: MuseGAN (Dong et al., 2018)

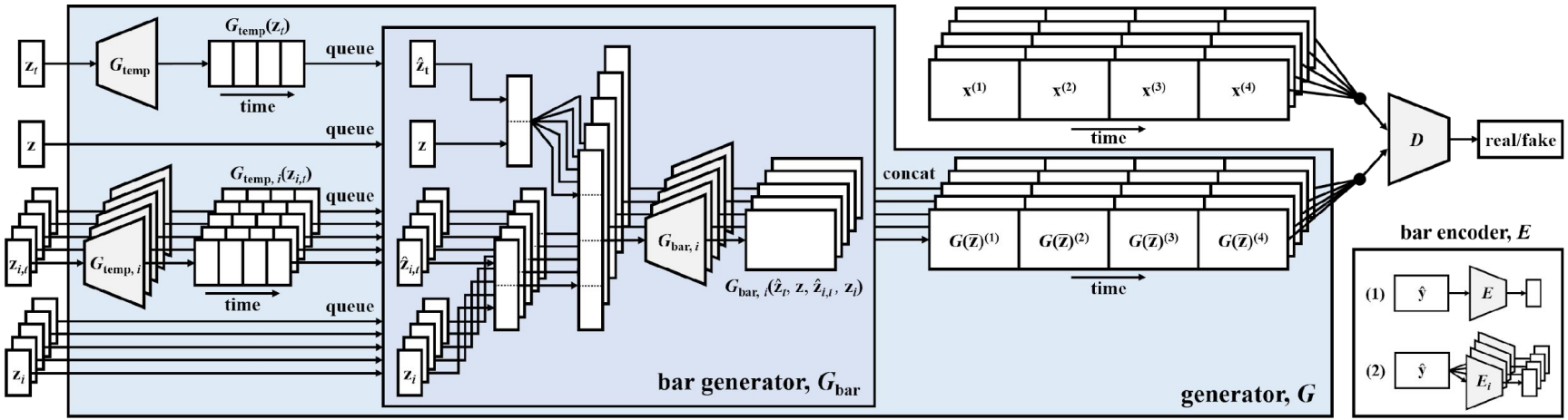
- Convolutional GAN for **multi-track** MIDI generation
  - Piani, guitar, bass, strings, drums



Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang, "MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," *AAAI*, 2018

Example: MuseGAN (Dong et al., 2018)

- Convolutional GAN for **multi-track** MIDI generation



(source: Dong et al., 2018)

Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang, "MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," *AAAI*, 2018

Example: MuseGAN (Dong et al., 2018)

Jamming model

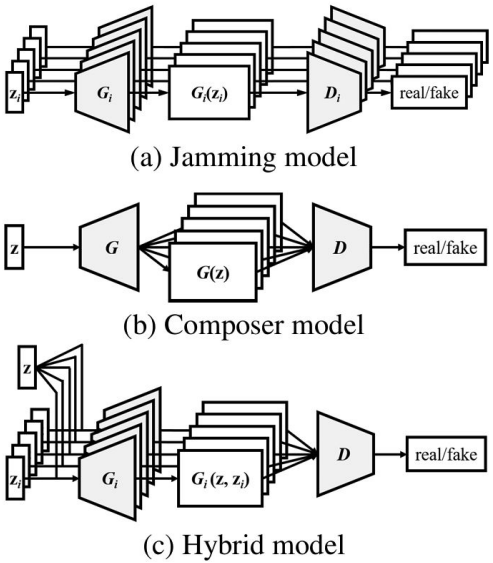
- multiple generators work independently and generate music of its own track
- a group of musicians playing different instruments without a predefined arrangement, a.k.a, jamming

Composer model

- one single generator creates a multi-channel piano-roll
- a composer who arranges instruments with knowledge of harmonic structure and instrumentation

Hybrid model

- combining the idea of jamming and composing



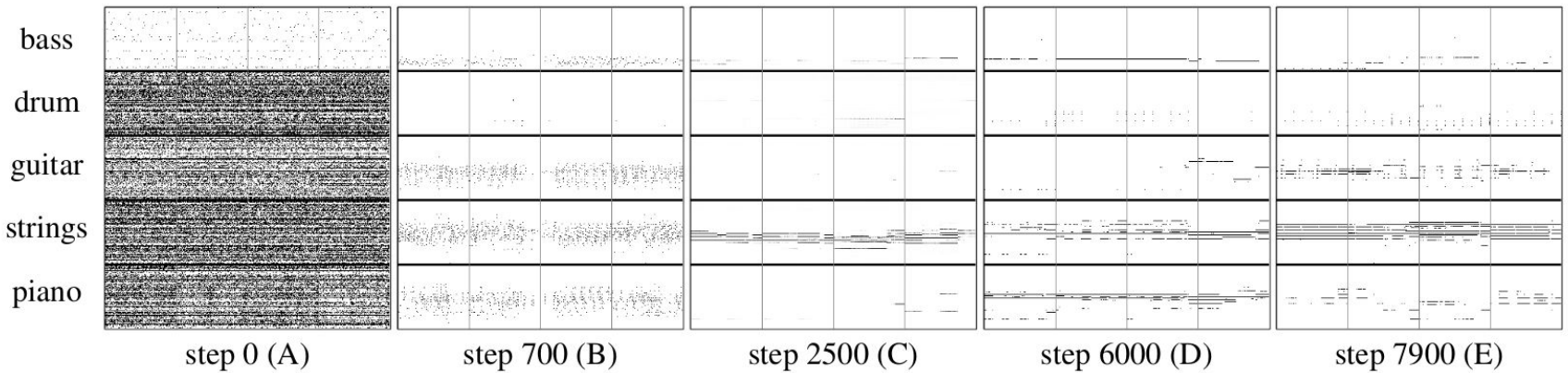
(source: Dong et al., 2018)

Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang, "MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," *AAAI*, 2018

Example: MuseGAN (Dong et al., 2018)

- Demo page: https://hermandong.com/musegan/results

Examples of generated music



(source: Dong et al., 2018)

Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang, "MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," *AAAI*, 2018

Take-homes

- Token generation isn’t the only way to generate music **symbolically**
- The “2D” view of music, which is mostly what we explored in Module 2, and we’ll see again in Module 4 is also viable for symbolic synthesis

Symbolic music generation

Appendix: Symbolic music datasets

PDMX

Description:

- 254,077 public domain MusicXML scores of multi-track multi-instrument music, collected from MuseScore
- including genre, rating, user comment information and 12M time-aligned performance directives

Download: https://zenodo.org/records/13763756

Citation:

> **PDMX: A large-scale public domain MusicXML dataset for symbolic music processing**
> Phillip Long, Zachary Novack, Taylor Berg-Kirkpatrick, Julian McAuley
> *ICASSP, 2025*
> pdf

Lakh MIDI

Description:

- 178,561 multi-track multi-instrument music in MIDI format
- 45,129 of which have been matched and aligned to entries in the Million Song Dataset, with the matching process discussed in the paper
- for the statistics of information such as instruments, lyrics and key changes in the dataset, refer to this tutorial

Download: https://colinraffel.com/projects/lmd/

Citation:

> **Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching**
> Colin Raffel
> *Columbia University, USA, 2016*
> pdf

HookTheory

Description:

- 26,175 aligned melody and harmony annotations for recordings hosted on YouTube, collected from HookTheory's TheoryTab DB
- HookTheory is a platform where users can easily create and share musical analyses of particular recordings hosted on YouTube, with Wikipedia-style editing
- including key annotations

Download: https://github.com/chrisdonahue/sheetsage

Citation:

> **Melody transcription via generative pre-training**
> Chris Donahue, John Thickstun, Percy Liang
> *ISMIR, 2022*
> pdf

POP909

Description:

- 909 pop piano arrangement in MIDI format, each with multiple versions of piano arrangements created by professional musicians
- including tempo, beat, chord, key annotations

Download: https://github.com/music-x-lab/POP909-Dataset

Citation:

> **POP909: A Pop-song Dataset for Music Arrangement Generation**
> Ziyu Wang, Ke Chen, Junyan Jiang, Yiyi Zhang, Maoran Xu, Shuqi Dai, Xianbin Gu, Gus Xia
> *ISMIR, 2020*
> pdf

Pop1k7

Description:

- 1747 4-minute pop piano music, with transcribed MIDI available

Download: https://zenodo.org/records/13167761

Citation:

> **Compound Word Transformer: Learning to Compose Full-Song Music over Dynamic Directed Hypergraphs**
> Wen-Yi Hsiao, Jen-Yu Liu, Yin-Cheng Yeh, Yi-Hsuan Yang
> *AAAI, 2021*
> pdf

MAESTRO

Description:

- 1,184 piano performance recordings and aligned MIDI, with the fine alignment between note labels and audio waveforms
- mostly classical, including composers from the 17th to early 20th century
- key strike velocities and sustain/sostenuto/una corda pedal positions are recorded

Download: https://magenta.tensorflow.org/datasets/maestro

Citation:

> **Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset**
> Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, Douglas Eck
> *ICLR, 2019*
> pdf

EMOPIA

Description:

- 1078 pop piano performance clips from 387 songs, with transcribed MIDI available, collected from YouTube
- including clip-level human-annotated emotion labels
- more datasets with emotion labels: datasets_emotion

Download: https://zenodo.org/record/5090631

Citation:

> **EMOPIA: A Multi-Modal Pop Piano Dataset For Emotion Recognition and Emotion-based Music Generation**
> Hsiao-Tzu Hung, Joann Ching, Seungheon Doh, Nabin Kim, Juhan Nam, Yi-Hsuan Yang
> *ISMIR, 2021*
> pdf

MetaMIDI

Description:

- 436,631 multi-track MIDI files, with artist and title metadata for 221,504 MIDI files, and genre metadata for 143,868 MIDI files
- MIDI files were matched against a collection of 32,000,000 30-second audio clips retrieved from Spotify, resulting in over 10,796,557 audio-MIDI matches
- for the statistics of instruments, time signature and key signature information, please refer to the repo

Download: https://zenodo.org/records/5142664#.YQN3c5NKgWo

Citation:

> **Building the MetaMIDI Dataset: Linking Symbolic and Audio Musical Data**
> Jeff Ens, Philippe Pasquier
> *ISMIR, 2021*
> pdf

SymphonyNet

Description:

- 46,359 multi-track multi-instrument symphony in MIDI format
- for the statistics of instruments, time signature and key signature information, please refer to the Dataset Introduction and Analysis part from the project website

Download: https://symphonynet.github.io/

Citation:

> **Symphony Generation with Permutation Invariant Language Model**
> Jiafeng Liu, Yuanliang Dong, Zehua Cheng, Xinran Zhang, Xiaobing Li, Feng Yu, Maosong Sun
> *ISMIR, 2022*
> pdf

GiantMIDI-Piano

Description:

- 10,855 classical piano MIDI composed by 2,786 composers
- MIDI files are transcribed from live recordings with a high-resolution piano transcription system
- more details about this dataset, such as comparison between different composers on note histogram, pitch class distribution and interval distribution, can be found in the paper

Download: https://github.com/bytedance/GiantMIDI-Piano?tab=readme-ov-file

Citation:

> **GiantMIDI-Piano: A large-scale MIDI dataset for classical piano music**
> Qiuqiang Kong, Bochen Li, Jitong Chen, Yuxuan Wang
> *TISMIR, 2022*
> pdf

JSB Chorales

Description:

- 382 four-part harmonized chorales by J. S. Bach
- the data is encoded as Python lists, needs some further processing steps to render as audio
- **Bonus**: playing with Google Doodles to generate polyphonic music in the style of Bach (a music generation model trained with this dataset, more details in the website)

Download: https://github.com/czhuang/JSB-Chorales-dataset?tab=readme-ov-file

Citation:

> **Harmonising Chorales by Probabilistic Inference**
> Moray Allan, Christopher Williams
> *NIPS, 2004*
> pdf

Nottingham

Description:

- 1k lead sheets of British and American folk tunes in ABC format or cleaned MIDI format

Download:

ABC format: https://abc.sourceforge.net/NMD/

MIDI format: https://github.com/jukedeck/nottingham-dataset?tab=readme-ov-file